Imam Mohammad Ibn Saud Islamic University
College of Computer and Information Sciences
Computer Science Department
Bachelor of Science in Computer Science
First Semester 1447 - 2025

Machine Learning Based Loan Default Prediction MLLDP

Group
M14
Name
Thamer Alshamrani
Saad Almugrin

Abstract

Loan defaults are a major source of financial risk for banks and lending institutions, leading to revenue loss and unstable credit markets. Traditional credit decisions rely on manual reviews or fixed rule-based systems, which struggle to capture the complex patterns hidden in modern lending data and often produce inconsistent outcomes. This project introduces the Machine Learning Loan Default Prediction (MLLDP) system, an end-to-end pipeline that learns these patterns directly from historical data and supports loan officers with transparent, evidence-based decisions. The methodology processes the LendingClub public dataset of over 1.3 million loans, applies an extensive data engineering pipeline, and addresses the class imbalance through native class weighting rather than synthetic oversampling. We tuned and compared four machine learning algorithms (LightGBM, XGBoost, Random Forest, and Logistic Regression), built a stacking ensemble, applied isotonic probability calibration, and selected the decision threshold by sweeping F2 scores across the validation set. The final hero model is a calibrated XGBoost at threshold 0.17, which achieves a recall of 76.14%, an F2 of 0.5846, and an AUC of 0.7309 on a fully held-out test set. To solve the "black-box" problem identified in the literature, every prediction is paired with a SHAP-based feature explanation and a list of 10 similar past borrowers retrieved from a 100,000-applicant lookup pool. The complete system is deployed as a Streamlit web application with four input methods, giving loan officers a fast and explainable tool to support real credit risk decisions.

الملخص

يُعدّ تعثّر القروض من أبرز مصادر المخاطر المالية التي تواجه البنوك والمؤسسات الإقراضية، إذ يؤدي إلى خسائر في الإيرادات وعدم استقرار في أسواق الائتمان. تعتمد القرارات الائتمانية التقليدية على المراجعات اليدوية أو الأنظمة القائمة على

قواعد ثابتة، وهي أساليب تعجز عن التقاط الأنماط المعقدة المخفية في بيانات الإقراض الحديثة وكثيراً ما تنتج قرارات غير متسقة. يقدم هذا المشروع نظام التنبؤ بتعثر القروض المبني على تعلم الآلة(MLLDP) ، وهو خط معالجة متكامل يتعلم هذه الأنماط مباشرةً من البيانات التاريخية ويزود موظفي القروض بقرارات شفافة ومدعومة بالأدلة. تعتمد المنهجية على معالجة مجموعة بيانات LendingClub العامة التي تضم أكثر من 1.3 مليون قرض، وتطبق خط معالجة موسعاً لهندسة البيانات، وتعالج اختلال توازن الفئات من خلال الترجيح المدمج للفئات (native class weighting)بدلاً من توليد عينات اصطناعية. قمنا بضبط ومقارنة أربع خوارزميات لتعلم الآلة وهي LightGBM و XGBoostو Random Forest وLogistic Regression، وبنينا نموذجاً تجميعياً مكدساً (stacking ensemble)، وطبّقنا معايرة احتمالية متساوية التوتر (isotonic calibration)، واخترنا عتبة القرار عبر مسح قيم درجة F2 على مجموعة التحقق. النموذج النهائي المختار هو XGBoost المعاير عند عتبة 0.17، الذي حقق استرجاعاً (recall) قدره 76.14% ودرجة F2 قدرها 0.5846 ومساحة تحت المنحنى (AUC) قدرها 0.7309 على مجموعة اختبار محجوزة بالكامل. ولمعالجة مشكلة "الصندوق الأسود" التي رصدتها الأدبيات، يُرفق كل تنبؤ بتفسير للسمات قائم على قيمSHAP ، إضافةً إلى قائمة بعشرة مقترضين سابقين مشابهين مستخرجة من مجموعة بحث تضم 100,000 متقدم. ويُنشر النظام الكامل كتطبيق ويب مبني باستخدامStreamlit ، يدعم أربع طرق لإدخال البيانات، ليمنح موظفي القروض أداةً سريعة وقابلة للتفسير تدعم اتخاذ قرارات حقيقية في إدارة مخاطر الائتمان.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Description
MLLDP	Machine Learning Loan Default Prediction
ML	Machine Learning
EDA	Exploratory Data Analysis
k-NN	k-Nearest Neighbors
SVC	Support Vector Classifier
DT	Decision Tree
RF	Random Forest
LR	Logistic Regression
MLP	Multi-Layered Perceptron
SVM	Support Vector Machine
XGBoost	eXtreme Gradient Boosting
SMOTE	Synthetic Minority Over-sampling Technique
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
SHAP	SHapley Additive exPlanations
VIF	Variance Inflation Factor
RFE	Recursive Feature Elimination
BNPL	Buy Now, Pay Later
P2P	Peer-to-Peer
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
LightGBM	Light Gradient Boosting Machine
AdaBoost	Adaptive Boosting
NN	Neural Networks
ADASYN	Adaptive Synthetic Sampling
FICO	Fair Isaac Corporation
KNIME	Konstanz Information Miner
F1	F1-score, harmonic mean of precision and recall
CSV	Comma-Separated Values
GBT	Gradient Boosted Trees
ROS	Random Over-Sampling
RUS	Random Under-Sampling
ENNs	Edited Nearest Neighbors
IDE	Integrated Development Environment
RAM	Random Access Memory
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
AI	Artificial Intelligence
SMOTE-NC	SMOTE for Nominal and Continuous features
SMOTE-Tomek	SMOTE combined with Tomek links

1. Introduction

1.1 Introduction

Loans are crucial to daily life and have a strong positive impact on the economy and consumer spending. However, this financial tool carries some risk, as an unexpected loan default could potentially lead to some financial problems. Historically, many loan decisions have relied on manual checks or fixed rules. This traditional approach seems inconsistent; it can lead to the approval of risky applicants who later default, while at the same time rejecting good applicants who would have repaid according to their payment plan. This results in higher financial losses and inconsistent decision making.

With the appearance of machine learning techniques and the age of big data, more advanced options than manual processing are now available for loan default prediction and classification. Several studies have explored the use of machine learning for this problem. This project proposes a generalizable system that understands lending data from public datasets and trains machine learning models to predict whether applicants are likely to default or not. The methodology focuses on creating an end-to-end data engineering and machine learning pipeline, utilizing Python tools like scikit-learn and SHAP. Figure 1.1 shows the loan default prediction system overview.

This chapter provides an overview of the project foundation. It begins by defining the core technical problem, followed by the specific aims and objectives derived from it. also talks about the project timeline and the qualifications of the project team. The chapter concludes with a summary of these points and outlines the structure of the upcoming chapter.

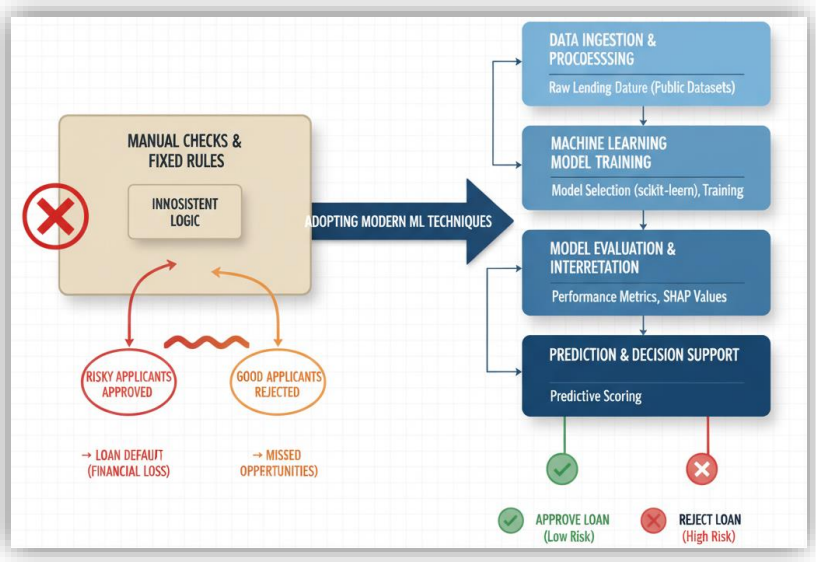

Figure 1.1: Loan Default Prediction System: From Traditional Methods to ML Pipeline

1.2 Problem definition

The reason for this project is that it's a big financial risk when people don't pay back their loans. The main problem we are trying to solve is that the old ways of approving loans don't work very well and aren't reliable. These old systems, which use manual checks or fixed rules, are bad at telling the difference between risky applicants and safe applicants, especially when they have to look at lots of complex data.

The main challenge is to build a system that can look at complex loan data and accurately predict if the person a risky applicant or safe applicant. This involves scientifically addressing challenges in data engineering (handling missing values, feature engineering) and machine learning (model selection, training, and comparison) to create a reliable prediction tool. The central research *question is*: **How can an end-to-end data engineering and machine learning pipeline be developed to accurately predict loan default?**

1.3 Aims and Objectives

1.3.1 Aim

The primary aim of this project is to develop an end-to-end data engineering and machine learning pipeline that predicts" Default" or " Not Default" for loan applications.

1.3.2 Objectives

To achieve this aim, the following tangible objectives must be completed:

- Define the project scope, establish success criteria, and select an appropriate public dataset for the study.
- Perform comprehensive data cleaning and organization, which includes handling missing values, standardizing formats, and engineering general features.
- Train and compare multiple machine learning models for "Default vs. Not Default" classification, followed by a comparative analysis to select the most reliable model.
- Produce clear summary reports detailing dataset characteristics, model performance.
- Design and develop a user-friendly web interface to allow loan officers to input applicant data and view the predictions and explanations.

1.4 Project Timeline

The project is divided into two main phases (GP1 and GP2), with clear milestones for each. The estimated time for each phase is detailed in Table 1.1 and Table 1.2

Table 1.1: Project Plan - Phase One (GP1)

Number of weeks	Milestones
3 weeks	Problem definition & proposal
6 weeks	Literature Review & Methodology Study
5 weeks	Report Writing & Submission
1 week	Discussion

Table 1.2: Project Plan - Phase Two (GP2)

Number of weeks	Milestones
5 weeks	Data collection and engineering
5 weeks	Develop and Train Machine learning models
2 weeks	Testing
2 weeks	GP2 report
1 week	Discussion

1.5 Team Qualifications

Table 1.3: Team Qualifications

	Student Names	
Task / Role	**Thamer Alshamrani**	**Saad Almugrin**
Team Leader	Yes	-
Documentation & Reporting	Yes	Yes
Data Engineering	Yes	Yes
ML Model Training	Yes	Yes
Web Application	Yes	Yes
System Testing	Yes	Yes

1.6 Conclusions

This chapter introduced the foundation for the "Machine Learning Based Loan Default Prediction" project. It began by highlighting the importance of the problem being studied: the significant financial risk associated with loan defaults and the high cost of the inconsistent decision making that results from these financial problems.

The chapter also outlined the previous attempts to solve this problem, which have historically relied on traditional manual checks or fixed, rule-based systems. As noted in the problem definition, these methods are often unreliable, work poorly, and are bad at telling the difference between safe and risky applicants when dealing with complex data.

To address these shortcomings, this project proposes a modern, data driven methodology. The plan, as defined in the aim, is to develop an end-to-end data engineering and machine learning pipeline. This approach differs from previous attempts by replacing static rules with a generalizable system that can learn from data. As detailed in the objectives, this involves comprehensive data cleaning, training and comparing multiple models, and producing clear summary reports on their performance.

The next chapter, the Literature Review, will build upon this foundation by providing a more detailed explanation of the problem background and giving a comprehensive overview of the related scientific work.

2. Literature Review

2.1 Introduction

Chapter 1 introduced the foundation of this project, defining the core problem of loan default prediction. It highlighted the unreliability of traditional, manual-based systems and established the ***project's aim: to build an end-to-end machine learning pipeline to solve this problem***.

The focus of this chapter is to provide the scientific and technical context for that aim. This chapter will give a clear understanding of the scientific background related to the problem and provide a clear overview of the work that has been done so far to solve it. It will review machine learning and explore data engineering processes involved and critically analyze existing research in loan default prediction.

This chapter is organized into three main sections. **Section 2.2** provides the necessary Background by defining fundamental concepts for the reader. **Section 2.3** presents a detailed review of the Related Work, categorizing and comparing previous studies. **Section 2.4** concludes the chapter by summarizing the findings and identifying the research gap that this project aims to fill.

2.2 Background

Loan default prediction has become a central challenge in modern financial technology due to the increasing complexity of lending processes. Loan decisions mainly affect banks and online lenders. A single misclassification like approving a risky applicant or rejecting a safe one can led to revenue loss, poor customer experiences and other problems. As mentioned before, some banks still use old, rule-based or manual systems. These systems don't work well when they have to analyze large amounts of data or complex customer behaviour.

The way lenders check for credit risk has changed a lot. In the past, they used fixed rules or just their own judgment. This was slow, expensive, and led to inconsistent decisions. The first data-driven systems used simple scoring models, but these weren't smart enough to catch complex patterns in a person's financial life. When digital banking became popular, this created a new challenge. Lenders suddenly had huge amounts of data (like income, spending, and payment history) that the old systems couldn't handle. This created a gap that machine learning filled. ML algorithms are perfect for this job because they can find complex patterns in all that data and are much better at predicting who will default on a loan.

2.2.1 Fundamental Concepts

Hereafter we present the fundamental concepts in this project:

Loan Default: A Loan Default occurs when a borrower fails to make scheduled repayments

Data Engineering and Feature Engineering*:* Data Engineering this is the process of preparing raw data for modelling, including cleaning, transforming, and handling missing values. Feature Engineering is the creation of new, predictive variables from the raw data like creating a 'debt-to-income ratio' from separate 'debt' and 'income' fields to improve model performance.

***Machine Learning*:** Machine Learning is a field of computer science where algorithms learn patterns from data rather than being explicitly programmed. In Supervised Learning, the algorithm learns from a historical dataset where the correct outcomes like past defaults are already known.

2.3 Related work

This section builds on the problem foundation established in Chapter 1. It provides a review of existing scientific literature focused on solving loan default prediction using machine learning techniques. The following studies highlight common methodologies, algorithms, and outcomes in the field, providing a background for the work undertaken in this project.

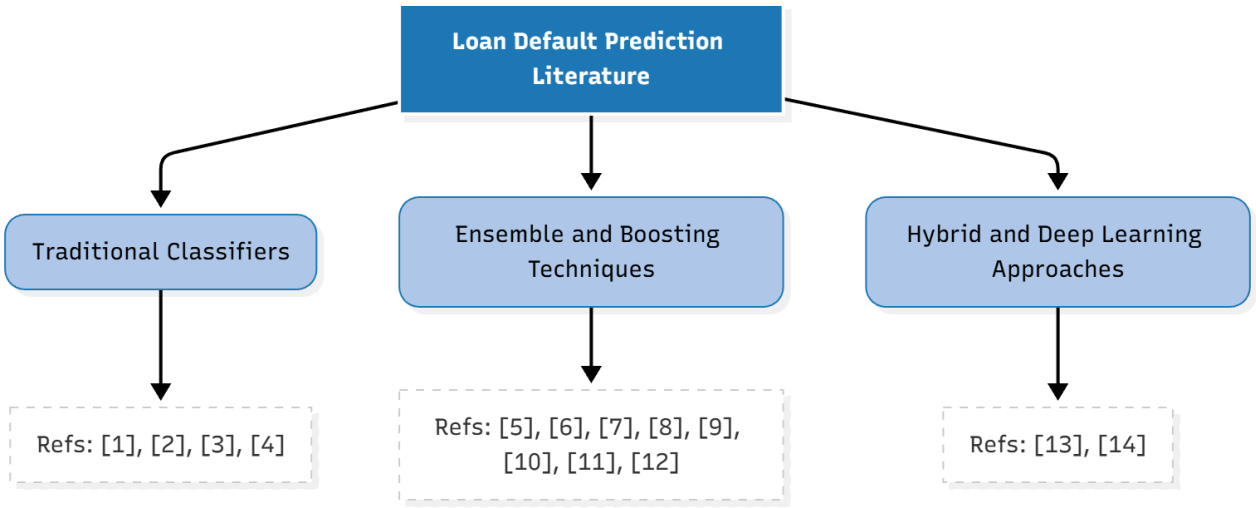

Figure 2.1: Taxonomy of surveyed research papers related to loan default prediction.

2.3.1 Comparative Studies of Traditional Classifiers

Early research in loan default prediction largely focused on establishing baselines using fundamental machine learning algorithms. These studies aimed to compare standard classifiers to understand their behavior on financial datasets.

The main focus of the research in [1] aims to improve credit risk management by comparing the accuracy of different machine learning classifiers for predicting loan defaults. The author trained and compared three models: Logistic Regression, Random Forest, and Decision Tree. The results showed a direct accuracy comparison: Random Forest achieved 89%, the Decision Tree achieved 88%, and Logistic Regression achieved 87%. The study concluded that the ensemble method, Random Forest, was the best-performing and most robust model for this task, though it also noted that all models were less effective at identifying the default cases specifically. For this analysis, the paper employed a CSV dataset from the Kaggle open-source platform. Key features included borrower demographics and financial attributes such as Income, Age, Experience, Marital Status, House Ownership, and Car Ownership.

The goal of the work in [2] was to find the best classifier for loan default by comparing several supervised data mining algorithms. The authors used J48 (Decision Tree), k-Nearest Neighbors (k-NN), Naïve Bayes, and Logistic regression. A key part of their work involved data preprocessing, especially using the SMOTE technique to address a significant class imbalance in the training data. In their comparison, k-NN (with k=3) had the highest classification accuracy at 78.38%, and Logistic regression was the runner-up at 77.31%. The study concluded that k-NN (k=3) and Logistic regression were the most effective models for this dataset, demonstrating the best overall performance. The experimental analysis relied on a private dataset provided by a loan-extending cooperative in the Philippines. It consisted of 1,000 instances; 900 were used for training and 100 for testing. The data had 29 attributes, including loan amount, interest rate, source of income, and previous default history.

The research detailed in [3] aims to create an efficient system for predicting loan approval status ("Yes" or "No"). The authors compare three machine learning models: K-Nearest Neighbours (KNN), Decision Tree, and Support Vector Classifier (SVC). The methodology involved data preprocessing to convert categorical values to numbers and using a correlation matrix for feature selection. The results showed that the Support Vector Classifier (SVC) was the most effective model, achieving the highest accuracy of 83.33%. The study concluded that SVC has the potential to be a

strong tool for forecasting loan approval. This research made use of a public dataset from Kaggle consisting of 615 rows. Key features included 'Gender', 'Married', 'Education', 'Applicant Income', and 'Property area'.

The work proposed in [4] aims to maximize the prediction performance for P2P loan defaults using the KNIME analytics platform. The authors compared five models: Logistic Regression, Naive Bayes, Random Forest, K-Nearest Neighbour (KNN), and Decision Tree. Their methodology included extensive data preprocessing and dimension reduction, using a correlation filter and backward feature elimination to reduce the dataset from 112 features down to 16. After performing hyperparameter tuning on all models, the results showed that Random Forest was the best-suited model, achieving the highest testing accuracy of 94.6% and the highest AUC score (0.985). The analysis leveraged a real-world, public dataset from Bondora, a European P2P lending platform. It originally contained over 220,000 records and 112 features.

In conclusion, while these studies established important baselines, they highlighted that single, traditional classifiers like k-NN or Decision Trees often struggle with the complexity and size of modern financial datasets compared to more advanced methods.

2.3.2 Ensemble and Boosting Techniques

To address the limitations of single classifiers, recent literature has shifted heavily towards ensemble learning. These methods combine multiple weak learners like Decision Trees to improve predictive performance and handle class imbalance more effectively.

The study in [5] aims to enhance credit risk evaluation by using machine learning, noting that traditional models often fail to capture complex financial behaviors. The authors implement a Random Forest Classifier (while also noting Gradient Boosting as a valuable algorithm). Their methodology involves comprehensive data preprocessing, feature engineering, and model evaluation using metrics like accuracy, a confusion matrix, and a classification report. The study concluded that their Random Forest model demonstrates "promising performance" and can serve as a reliable tool for financial institutions to identify high-risk applicants and make more informed decisions. The study utilized a comprehensive dataset from a CSV file containing historical loan data and borrower information. Key features included loan terms, interest rates, annual income, verification status, and loan purpose.

The research presented in [6] evaluates and compares four machine learning models XGBoost, Gradient Boosting, Random Forest, and LightGBM for loan default prediction. The methodology involved a full pipeline, including feature engineering and using SMOTE and class weighting to handle imbalanced data. The results showed that Gradient Boosting was the most effective model overall, achieving the highest accuracy (0.8887) and F1-score (0.8084). However, XGBoost demonstrated the best discriminatory power with the highest ROC AUC score (0.9714). The study was conducted using a synthetic dataset created by the authors. It was inspired by the "Credit Risk dataset on Kaggle" and enriched with variables from the "Financial Risk for Loan Approval data".

The primary objective of the study in [7] was to identify the most effective techniques for the entire model development process, with a strong focus on handling class imbalance. The authors compared six base models: Random Forest, Decision Tree, SVM, XGBoost, ADABoost, and a Multi-Layered Perceptron (MLP). They systematically tested various balancing methods, including ROS, RUS, SMOTE, ADASYN, and hybrid approaches like SMOTE + ENNs. The SMOTE + ENNs technique yielded the best-balanced dataset. The authors then proposed two ensemble methods (Voting and Stacking) to combine the base models. The final Stacking ensemble (Stacking A) achieved the highest performance, with an accuracy of 93.7%, precision of 95.6%, and recall of 95.5%. The study concluded that a stacking ensemble combined with the SMOTE + ENN balancing technique provides a robust solution for predicting credit default. The dataset employed in this study was sourced from LendingClub, specifically a stratified sample of 30% of the data from 2007-2018, resulting in 403,593 records. The dataset was imbalanced, with 80% non-defaults and 20% defaults.

The research conducted in [8] aimed to conduct a comparative study to find the best-performing classifier for loan default by comparing traditional models against more complex ensemble techniques. The authors used Apache Spark to process the big data and compared six models: K-nearest neighbor (KNN), Decision Trees (DT), Logistic Regression (LR), Neural Networks (NN), Random Forest (RF), and Gradient Boosted Trees (GBT). The results clearly showed that the ensemble methods were superior. Random Forest achieved 91.7% accuracy and 95.83% precision, while Gradient Boosted Trees achieved 91.9% accuracy and 79.68% precision. The study concluded that ensemble techniques perform better than single classifiers for predicting loan default. The analysis relied on a large, private dataset from a public Egyptian bank's database. It consisted of 2,954,168 customer loan observations from May 2005 to December 2017. The data

included customer personal, demographic, and loan details like 'Age', 'Marital status', 'Loan type', and 'Amount disbursement'.

The research in [9] compares five sophisticated machine learning models to find the best predictor for loan default. The models compared were XGBoost, Random Forest (RF), AdaBoost, k-nearest neighbors (kNN), and Multilayer Perceptrons (MLP). Because the dataset was so imbalanced, the author used AUC (Area Under the Curve) as the primary metric instead of accuracy. The results showed that RF, kNN, and MLP performed very poorly, while the boosting algorithms (XGBoost and AdaBoost) were much better. The study concluded that the AdaBoost model was the clear winner, achieving a perfect AUC score of 1.0 (or 100% accuracy) for this dataset. The experiments were conducted using a real-world, publicly accessible dataset from Xiamen International Bank, containing 132,029 records. Key features included user's basic attributes (age, job, education), lending information (loan product type, application hour), and credit report data. The dataset was noted as being highly imbalanced.

In [10], the primary aim is to apply and compare the performance of Random Forest and XGBoost for loan default prediction. The methodology heavily focused on feature engineering, first using the Variance Threshold method to remove columns with no variance, and then using the Variance Inflation Factor (VIF) method to filter out features that were highly correlated (multicollinearity). This process reduced the number of features from 771 down to 419. The results showed that both models performed almost identically, with Random Forest achieving an accuracy of 90.66% and XGBoost achieving an accuracy of 90.64%. The study concluded that both models are highly accurate for this task, with little difference between them. The empirical work was based on a dataset provided by Imperial College London, which included 105,471 records and 771 features.

Reference [11] provides a comprehensive comparison of multiple machine learning models for predicting credit default. The authors evaluated Logistic Regression, Decision Trees, Random Forest, Gradient Boosting, Support Vector Machine (SVM), XGBoost, and LightGBM. Their methodology involved detailed data preprocessing, using SMOTE to handle class imbalance, Min-Max scaling for normalization, and Recursive Feature Elimination (RFE) for feature selection. The results showed that ensemble boosting models were the most effective. LightGBM achieved the highest performance with an accuracy of 93.1% and an AUC-ROC of 0.96, followed closely by XGBoost with 92.4% accuracy and an AUC-ROC of 0.95. The study concluded that LightGBM and XGBoost are highly effective for credit risk management. The dataset used in this evaluation was

curated from publicly available financial repositories, containing 10,000 records. Key features included 'Age', 'Income', 'Credit_History_Length', 'Debt_to_Income_Ratio', and 'Loan_Amount'.

The study presented in [12] aims to identify the most critical variables for default prediction from the entire history of the LendingClub platform. The authors used the Random Forest algorithm as their primary model for both its predictive power and its ability to identify feature importance. After preprocessing, the model selected just nine key predictors, including 'last_fico_range_high', 'last_pymnt_amnt', and 'total_rec_int', as the most influential. The study also applied the SMOTE technique to address the data's class imbalance (80.64% non-default vs. 19.36% default). The final Random Forest model achieved a high F1-Macro Score of over 96% (and 98% accuracy). The study concluded that Random Forest is highly effective for this binary classification and that traditional credit scoring variables (like FICO scores) remain the most important predictors. The analysis was based on the complete public dataset from LendingClub (2007-2020), which originally contained 2,925,493 observations and 140 variables.

In conclusion, the consensus in the literature is that ensemble methods Random Forest and boosting algorithms XGBoost and LightGBM consistently outperform traditional models in terms of accuracy. However, a recurring limitation in these studies is the "black-box" nature of these complex models, which often lack of transparency.

2.3.3 Hybrid and Advanced Deep Learning Approaches

The most recent advancements involve hybrid methodologies and deep learning architectures designed to capture sequential patterns or handle specific data challenges like extreme imbalance.

The work in [13] aims to solve two key problems in online (BNPL) loan prediction: data imbalance and the need for fast, real-time credit scoring. The authors proposed a hybrid methodology, comparing 8 ML models (including Random Forest and XGBoost) combined with two balancing methods (SMOTE-Tomek and SMOTE-NC) and two feature selection methods (Logistic Regression and SHAP). The results showed that data balancing was essential and that SMOTE-NC performed better. Feature selection using SHAP was more effective and significantly reduced model run time. The study concluded that the combination of Random Forest or XGBoost with SMOTE-NC balancing and SHAP feature selection is the recommended approach. The analysis was performed on a private dataset from one of Iran's banks. It consisted of 8,289 records and 23

features. Key features included collateral_type, loanAmount, turnover, activeLoan, and job. The dataset was highly imbalanced.

The main contribution of [14] is a novel hybrid time-series model (CNN-LSTM-Attention) designed to capture the sequential nature of financial data (e.g., monthly statements). First, the authors benchmarked traditional models, finding that after hyperparameter tuning, XGBoost was the best performer with an AUC of 0.9529. They then compared this to their hybrid model. The hybrid CNN-LSTM-Attention model significantly outperformed the best traditional model, achieving an accuracy of 99.81% and an AUC of 0.9976. The study concluded that incorporating time-series analysis (treating a customer's history as a sequence) dramatically improves the precision and performance of credit default prediction. The study makes use of a public dataset from the Kaggle American Express Default Prediction challenge. This dataset contained 18 months of anonymized credit card account information from 41,989 unique customers, resulting in over 1 million data points and 191 features.

In conclusion, while hybrid and deep learning models offer marginal gains in accuracy for specific data types, they introduce significant computational complexity. For standard tabular loan data, ensemble methods remain the most practical balance of performance and efficiency.

Table 2.1: comparison table of related work

Ref.	*Description*	*Method Used*	*Dataset*	*ML Models Compared*	*Performance*	*Limitations*
[1]	Aims to improve credit risk management by comparing the accuracy of different ML classifiers.	Data preprocessing, hyperparameter tuning, model comparison.	Kaggle (CSV dataset)	Logistic Regression, Random Forest, Decision Tree.	Random Forest was the best-performing model with 89% accuracy.	Poor minority class performance. The author notes all models were "less effective at identifying the default cases specifically."
[2]	Aims to find the best classifier for loan default by comparing several supervised data mining algorithms.	J48 (DT), k-NN, Naïve Bayes, Logistic Regression. Used SMOTE for class imbalance.	Private (Philippine cooperative , 1,000 instances)	J48 (DT), k-NN, Naïve Bayes, Logistic Regression.	k-NN (k=3) had the highest accuracy at 78.38%.	Uses outdated models (k-NN) on a very small, private dataset. Results are not robust or generalizable.
[3]	Aims to create an efficient system for predicting loan approval status ("Yes" or "No").	Data preprocessing (categorical to numeric), correlation matrix for feature selection.	Kaggle (615 rows)	K-Nearest Neighbours (KNN), Decision Tree, Support Vector Classifier (SVC).	SVC was the most effective model with 83.33% accuracy.	Extremely small dataset (615 rows) makes the results unreliable and not generalizable.
[4]	Aims to maximize prediction performance for P2P loan defaults using the KNIME platform.	Extensive data preprocessing, dimension reduction, and hyperparameter tuning.	Real-world (Bondora P2P, 220k rows)	Logistic Regression, Naive Bayes, Random Forest, k-NN, Decision Tree.	Random Forest was the best-suited model with 94.6% accuracy.	Limited model comparison. Fails to test Random Forest against its main competitors
[5]	Aims to enhance credit risk evaluation using ML, as traditional models fail to capture complex behaviors.	Data preprocessing, feature engineering, Random Forest, hyperparameter tuning.	CSV (Historical loan data)	Random Forest (Gradient Boosting mentioned but not compared).	The RF model demonstrates "promising performance."	Limited comparison. Focuses on a single model (RF) without a systematic comparison to other ensemble methods.
[6]	Evaluates and compares four modern ML models for loan default prediction.	Full pipeline include feature engineering, SMOTE, and class weighting.	Synthetic Dataset (inspired by Kaggle)	XGBoost, Gradient Boosting, Random Forest, LightGBM.	Gradient Boosting was most effective (88.87% accuracy).	The study's results are based on synthetic data, not real-world bank data, limiting its practical validity.
[9]	Compares five sophisticated ML models to find the best predictor for loan default.	Feature extraction, model comparison using AUC (due to imbalance).	Real-world (Xiamen Int'l Bank, 132k rows)	XGBoost, Random Forest, AdaBoost, k-NN, MLP.	AdaBoost achieved a perfect AUC score of 1.0 (100%).	A 100% perfect score is unrealistic and a major red flag for overfitting or data leakage.
[10]	Applies and compares the performance of Random Forest and XGBoost.	Heavy feature engineering (Variance Threshold, VIF) to reduce 771 features.	Private (Imperial College London, 105k rows)	Random Forest, XGBoost.	Both models performed almost identically (approx. 90.6% accuracy).	Critical methodological gap. The entire study ignores the class imbalance problem

2.4 Conclusion

This chapter has provided the scientific and technical context for the project, fulfilling its aim as set out in the introduction. It established a clear understanding of the problem's background and provided a critical analysis of existing research in loan default prediction.

The Background section established that loan default prediction is a central challenge for financial institutions. It confirmed that traditional manual or rule-based systems can’t handle the huge amounts of data and complex customer behavior that define modern digital banking. This review affirmed that machine learning is essential to fill this gap, as its algorithms are uniquely capable of identifying the complex patterns required to accurately assess risk.

The review of Related Work analyzed numerous recent studies and confirmed a consensus on best practices. Ensemble models such as Random Forest, XGBoost, and LightGBM consistently provide the highest accuracy. Furthermore, the review highlighted that managing class imbalance, often through techniques like SMOTE, is a critical step in the data engineering pipeline.

However, this review found a ***significant research gap:*** a lack of interpretability. Many existing studies use "black-box" models while they are accurate, they cannot explain why a loan is flagged for default. We will try to fix this by using SHAP, the goal is to create a model that is not only accurate but also easy to understand. This directly fills the gap by providing a solution that is both predictive and explainable.

Building on the technical foundation and identified research gap from this chapter, Chapter 3 will present the Requirements Analysis and Specification, detailing the user and system requirements needed to build this proposed solution.

3. Requirements Analysis and Specification

3.1 Introduction

The previous chapter provided a comprehensive literature review, highlighting the importance of loan default prediction and identifying the lack of model interpretability as a key research gap. This chapter shifts the focus from the theoretical background to the practical specification of the proposed Machine Learning Based Loan Default Prediction (MLLDP) system. This chapter defines the scope of the project by detailing the User Requirements and the System Requirements. It classifies these into functional and non-functional requirements. Finally, this chapter presents the Project Management Plan, outlining the timeline for the development of lifecycles.

3.2 User Requirements

To define the requirements for the MLLDP system, the requirements elicitation process involved a thorough analysis of the datasets available in the public domain and a review of the limitations found in existing studies discussed in Chapter 2. Given the data-driven nature of this project, this approach was chosen over traditional interviews. The target users for this system are primarily financial analysts or loan officers who need a reliable tool to assist in decision-making, as well as the data engineers responsible for maintaining the model.

3.2.1 Functional Requirements

These requirements describe the specific services and functions the MLLDP system shall provide to its users. They are listed below to define what the system is expected to perform:

1-Model Integration: The web application shall automatically load the pre-trained machine learning model and necessary scalers upon startup to ensure it is ready to perform real-time predictions without requiring manual dataset uploads.

2-Applicant Data Input: The system shall provide a user-friendly web interface that allows the user (a loan officer) to manually input a new applicant's financial details, such as income, loan amount.

3-Loan Status Prediction: Upon receiving the applicant's data, the system shall process the input and display a clear classification result indicating whether the applicant is predicted to "Default" or "Not Default".

4-Prediction Explanation: To ensure transparency in decision-making, the system shall generate a visual explanation (using SHAP values) for each prediction, identifying which specific features, like high debt-to-income ratio influenced the model’s decision.

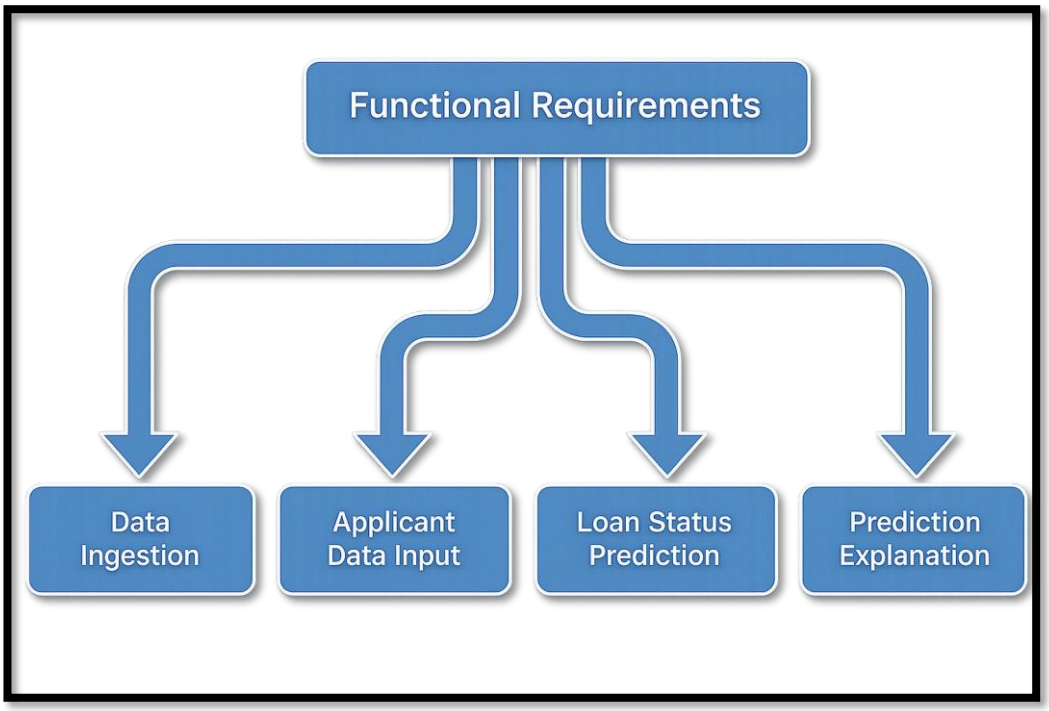

Figure 3.1: Functional Requirements

3.2.2 Non-Functional Requirements

These requirements define the system properties and constraints under which the MLLDP system must operate.

A. Product Requirements These requirements specify the expected behavior of the software in terms of efficiency, dependability, and security.

1-Efficiency Requirements: The web interface shall generate and display the prediction result and SHAP explanation within few seconds of data submission to ensure a smooth user experience.

2-Dependability Requirements: The machine learning model embedded in the system shall achieve a classification Accuracy and F1-Score to be considered reliable for decision support.

3-Usability Requirements: The web interface shall be intuitive and require no technical or programming knowledge to operate, allowing financial analysts to use the system effectively with minimal training.

4- Security Requirements: The system shall ensure data privacy by processing applicant data locally within the application session and not storing sensitive applicant data persistently after the session ends.

B. Organizational Requirements These requirements are derived from the development standards and operational policies.

1-Development Requirements: The system shall be developed using Python as the primary programming language, utilizing standard libraries such as scikit-learn for modeling and Streamlit for the web interface, ensuring standard code maintainability.

2-Operational Requirements: The system shall be deployable on a standard computing environment without requiring specialized high-performance hardware such as GPU clusters for the inference phase.

C. External Requirements These requirements arise from external factors to the system, such as ethical considerations in the financial domain.

1-Ethical Requirements (Interpretability): To comply with ethical guidelines for AI in finance and avoid "black-box" decision-making, the system must provide visual justifications for every prediction, ensuring transparency in why a loan was rejected or approved.

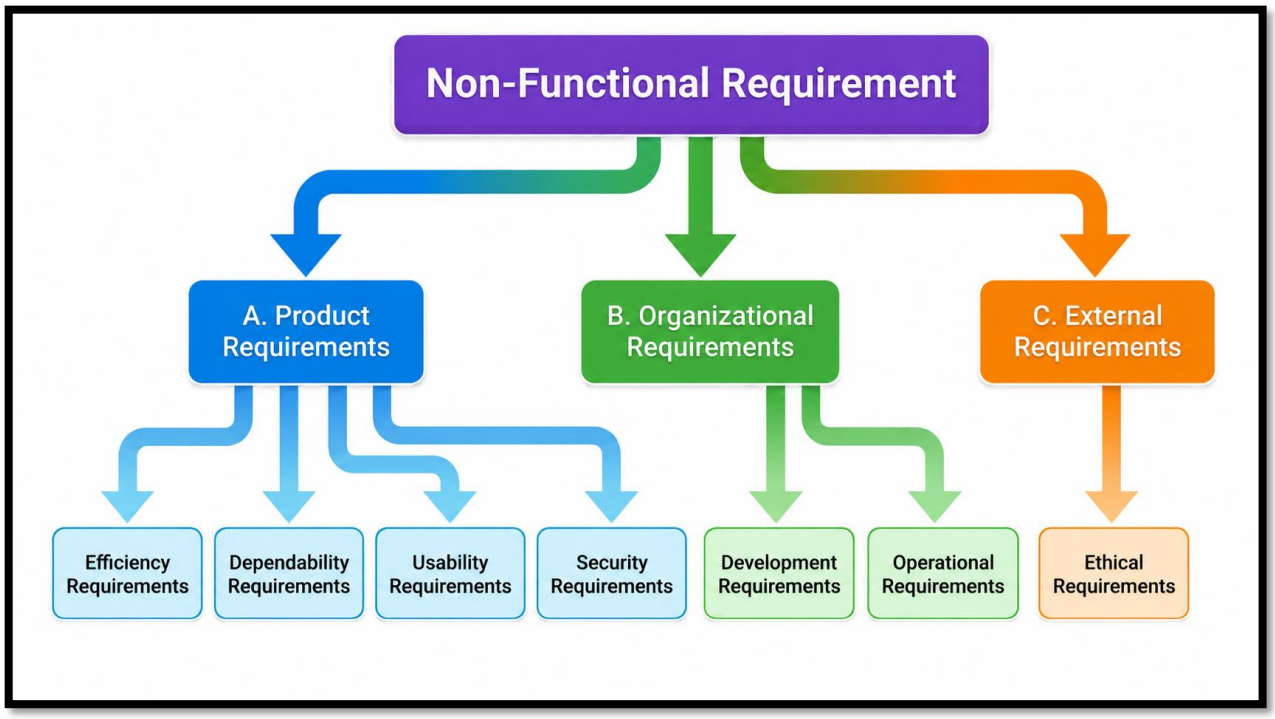

Figure 3.2: Non-Functional Requirements

3.3 System Requirements

This section elaborates on the user requirements defined in the previous section to provide a precise, detailed, and complete specification of the MLLDP system. It details the functional interactions using Use Case modelling and specifies the technical non-functional constraints required for implementation.

3.3.1 Functional Requirements

The system's functional behavior is modeled using a Use Case Diagram, which identifies the primary actors and their interactions with the system.

Actors Description: The system involves two primary actors:

1-Data Engineer: The technical user responsible for maintaining the system's backend, including processing datasets, training and Evaluating the model.

2-Loan Officer: The end-user who utilizes the web interface to input applicant details and view prediction results.

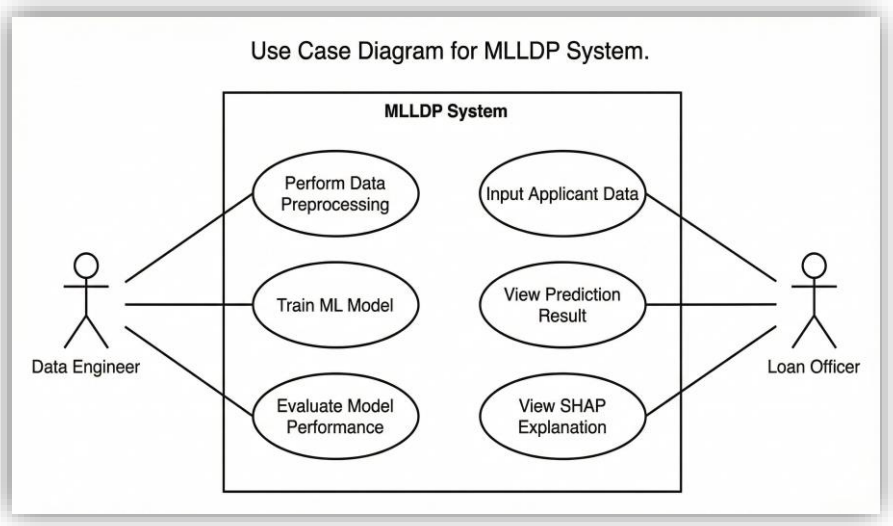

Figure 3.3: Functional Requirements Use Case Modeling

Use Case Specifications

The following table details the core use cases depicted in the diagram, mapping each interaction to its specific function.

Table 3.1: Use Case Specifications

Use Case Name	Actor	Description
Perform Data Preprocessing	Data Engineer	The actor executes the data engineering pipeline to ingest raw data, handle missing values, and encode categorical features to prepare the dataset for modeling.
Train ML Model	Data Engineer	The actor initiates the training process using the prepared data to build the classification model and saves the trained model for the web application.
Evaluate Model Performance	Data Engineer	The actor reviews the model's performance metrics like Accuracy and F1-Score to ensure it meets high reliability before deployment.
Input Applicant Data	Loan Officer	The actor accesses the web interface and manually enters the new applicant's financial details like income and loan amount into the input form.
View Prediction Result	Loan Officer	The system processes the input using the pre-trained model and displays the classification result ("Default" or "Not Default") to the actor.
View SHAP Explanation	Loan Officer	The system generates and displays a SHAP visual plot to explain the reasoning behind the specific prediction, for example highlighting high debt as a cause.

3.3.2 Non-Functional Requirements

this section specifies the detailed technical requirements and environmental constraints necessary for the development and deployment of the MLLDP system.

A. Software Requirements

The system shall be built using the following open-source software stack. These tools are selected for their compatibility, community support, and efficiency in data science pipelines.

Programming Language: Python 3.9 (or higher) is required due to its extensive support for data manipulation and machine learning libraries.

Machine Learning Libraries:

1-Scikit-Learn: For implementing classification algorithms and evaluation metrics.

2-Pandas & NumPy: For efficient data ingestion, cleaning, and vectorization of the dataset.

3-SHAP: For generating interpretability plots to explain model decisions.

Web Framework:

1-Streamlit: it should be used to develop the frontend interface. It is chosen for its ability to convert Python scripts into interactive web apps without requiring HTML/CSS knowledge.

Version Control: Git and GitHub shall be used for source code management and team collaboration.

IDE: Jupyter Notebook and VS Code.

B. Hardware Requirements

The system has distinct hardware requirements for the two main phases of the lifecycle:

Development and Training Phase:

1-Processor: Minimum Intel Core i5 (10th Gen) or equivalent AMD Ryzen.

2-RAM: Minimum 8GB (16GB recommended) to handle the in-memory processing of the datasets during training.

3-Storage: At least 20GB of free disk space for datasets and model artifacts.

Deployment and Inference Phase:

1-Server Side: The Streamlit application is lightweight and can run on a standard local machine or a basic cloud instance such as Streamlit Cloud with 1GB RAM.

2-Client Side: The user needs only a modern web browser such as Google Chrome, Microsoft Edge, or Firefox and a stable internet connection.

C. Performance Constraints

To ensure the system meets the efficiency goals defined in:

1-Inference Latency: The system must utilize the pre-trained model to ensure that the prediction time from button click to result display is within a few seconds under normal load.

2-Model Size: The serialized model file size should be optimized to ensure fast loading times upon application startup.

3.4 Project Management Plan

The project timeline is divided into two phases (GP1 and GP2) following the System Development Life Cycle. The schedule ensures that the theoretical foundation and methodology are fully established in GP1, while the heavy implementation, model training, and deployment are executed in GP2.

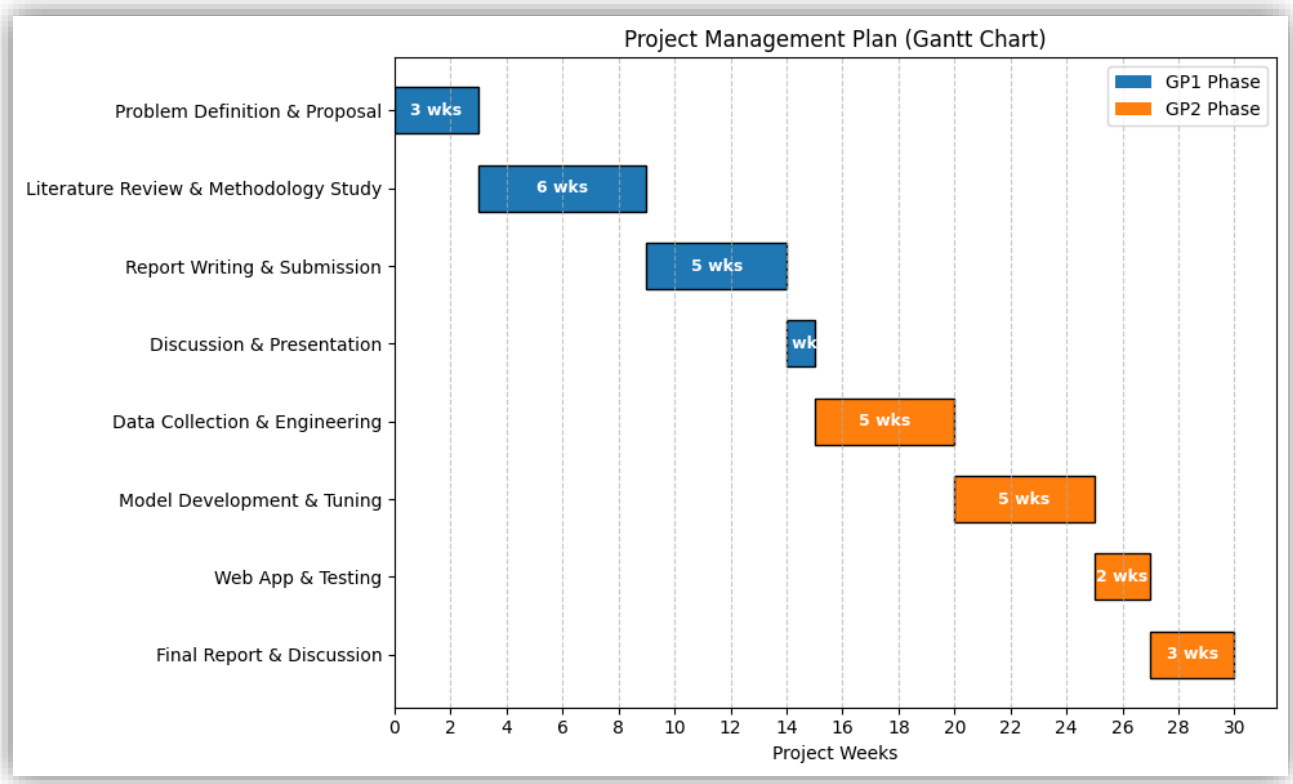

Figure 3.4: Project Gantt Chart representing the timeline for GP1 and GP2.

4. Design Chapter

4.1 Introduction

Following the requirements analysis presented in the previous chapter, this chapter focuses on the conceptual design and structural organization of the Machine Learning Based Loan Default Prediction (MLLDP) system. While the previous chapter defined what functions the system must provide, this chapter establishes the logical blueprint and system components required to fulfill those needs.

4.2 System Architecture

The MLLDP system is built using a Machine Learning Pipeline Architecture. This is the standard pattern used for production machine learning systems because it captures the fundamental split between **training time** and **inference time**. In our system, training is done once in an offline environment to produce a set of saved artifacts, and inference runs each time a user submits a new applicant for prediction. The two pipelines do very different work, but they share a common set of saved files that connect them. This design separates the heavy machine learning training from the live web application, which makes the system fast at inference time and easy to maintain.

The architecture is organized into three main parts: the **Training Pipeline**, the **Shared Storage** of saved artifacts, and the **Inference Pipeline**. The complete architecture is shown in Figure 4.1.

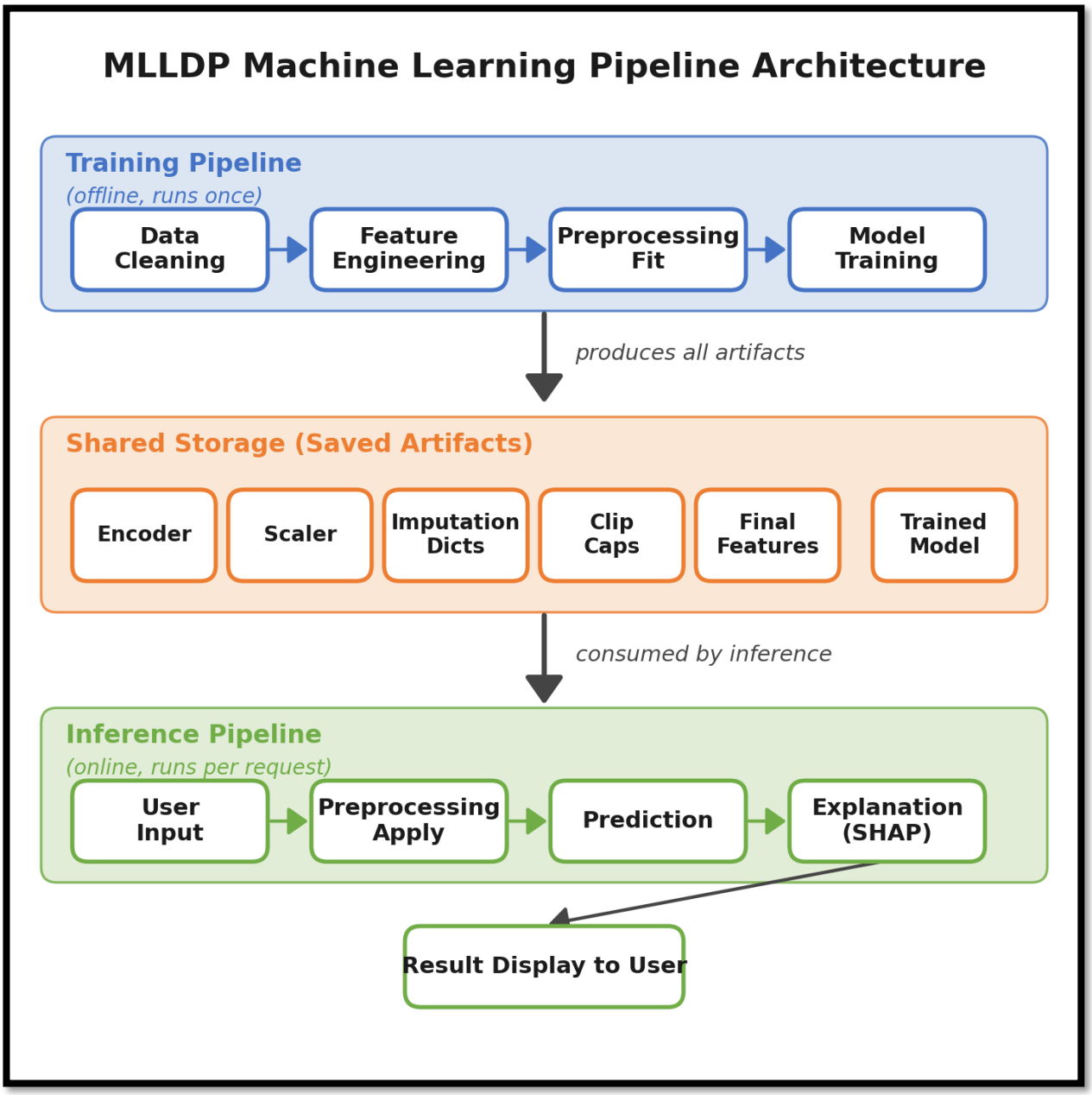

Figure 4.1: MLLDP Machine Learning Pipeline Architecture

4.2.1 Training Pipeline (Offline)

The Training Pipeline is the offline part of the system. It runs once on the historical Lending Club dataset and is responsible for building all the components that the live application later needs. As shown in Figure 4.1, this pipeline is divided into four stages:

1- Data Cleaning: This stage takes the raw dataset and removes the rows and columns that are not useful for the model. It also handles data quality issues such as data leakage columns and broken values that would mislead the training.

2- Feature Engineering: This stage creates new features from the existing columns to make the data more meaningful for the model.

3- Preprocessing Fit: This stage splits the data into training, validation, and test sets, then fits the preprocessing tools such as the encoders, the scaler, and the imputation dictionaries on the training set only. This ensures that the validation and test sets stay independent and do not leak any information into the training process.

4- Model Training: This stage trains the final classification model on the prepared training set and selects the best feature configuration through validation experiments.

The output of the Training Pipeline is not a single file but a collection of saved artifacts. These artifacts move into the Shared Storage in the middle of the architecture and become the bridge to the Inference Pipeline.

4.2.2 Shared Storage (Saved Artifacts)

The Shared Storage holds all the files that the Training Pipeline produces and the Inference Pipeline consumes. This is the most important part of the architecture because it is the only point of communication between the two pipelines. It contains six main artifacts:

1- Encoder: The fitted categorical encoder that converts text into numeric form.

2- Scaler: The fitted feature scaler that brings continuous numerical features to a common scale.

3- Imputation Dicts: The dictionaries that hold the fill values (medians and modes) used to handle missing values in the user input.

4- Clip Caps: The dictionary that holds the upper and lower percentile caps used to clip extreme values in numerical features.

5- Final Features: The locked list of input features that the model expects, in the exact order it was trained on.

6- Trained Model: The final classification model used to predict default probability for new applicants.

Storing these artifacts ensures that any new applicant input is processed in exactly the same way as the training data was processed. Without this, the model would receive inputs in a different format than what it was trained on, and the predictions would be unreliable.

4.2.3 Inference Pipeline (Online)

The Inference Pipeline is the online part of the system. It runs on demand inside the web application every time a loan officer submits an applicant for prediction. Unlike the Training Pipeline, this pipeline does not learn anything new from the data. It only uses the saved artifacts to produce a result. As shown in Figure 4.1, this pipeline is divided into four stages:

1- User Input: The loan officer enters the applicant's details into the web interface, such as the requested loan amount, the income and other financial information.
2- Preprocessing Apply: The system loads the saved artifacts from Shared Storage and applies the same transformations that were used during training: filling missing values, clipping outliers, encoding categorical fields, and scaling continuous fields.
3- Prediction: The preprocessed input is passed to the trained model, which returns the probability of default for that applicant.
4- Explanation (SHAP): The system uses SHAP to break down the prediction and identify which specific features pushed the score up or down for this particular applicant. This produces a visual explanation that the user can understand.

Each stage of the Inference Pipeline uses specific artifacts from Shared Storage to do its work. The User Input stage uses the Final Features list to know which input fields to ask the user. The Preprocessing Apply stage uses the Encoder, the Scaler, the Imputation Dictionaries, and the Clip Caps to transform the input in the same way as the training data was transformed. The Prediction stage uses the Trained Model to compute the default probability. The Explanation stage uses the same Trained Model to generate the SHAP values. The final result, including the prediction and the explanation, is displayed back to the user through the web interface.

4.2.4 Design Justification

We chose the Machine Learning Pipeline Architecture for four main reasons. First, it is the standard pattern used in real production machine learning systems. It fits the way ML systems naturally work, where training and inference are two separate jobs. Second, it keeps the heavy training work apart from the live web application. This makes the web app fast and responsive for the user. Third, the saved artifacts act as a bridge between the two pipelines. They make sure that any new user input is processed in the same way as the training data. Fourth, the architecture has a dedicated SHAP stage built into the Inference Pipeline. This makes the model explainable, which solves the "black-box" problem we discussed in the literature review of Chapter 2.

4.3 Data Design

The MLLDP system uses a single historical dataset as its primary data source, the LendingClub public lending dataset. This section describes the source dataset and the logical schema that organizes its attributes into meaningful categories.

4.3.1 Dataset Overview

The core data source for this project is the LendingClub dataset, sourced from the largest peer-to-peer lending platform in the United States. This dataset is widely used in financial technology research as a benchmark for credit risk analysis.

Nature of Data: The dataset represents real unsecured personal loans issued between 2007 and 2018, capturing the complete lifecycle of a loan from initial application to final repayment status.

Volume and Dimensionality: The dataset contains over 2 million loan records and more than 150 distinct attributes covering financial metrics and borrower demographics.

Data Source: The data is retrieved from public repositories (Kaggle), which aggregate LendingClub's loan files into a consolidated format suitable for data science applications[1]

4.3.2 Data Schema

The dataset attributes are organized into a logical schema centered around a single aggregate entity called the Loan Application Record. Three external categories contribute to this central entity: Borrower Info, Loan Details, and Credit History. Figure 4.2 shows the conceptual data model.

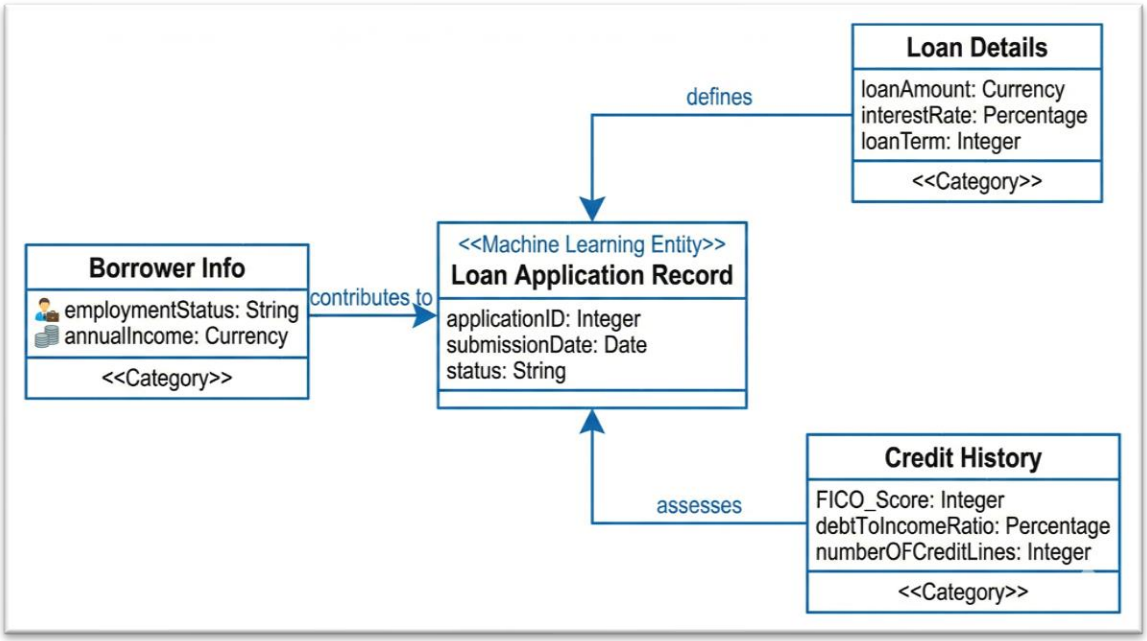

Figure 4.2: Conceptual Data Model

[1] NOTE: https://www.kaggle.com/datasets/wordsforthewise/lending-club?select=accepted_2007_to_2018Q4.csv.gz

4.4 Modular Decomposition

The MLLDP system follows a pipeline data-flow model rather than an object-oriented design. The system is decomposed into a small number of functional modules where each module takes a specific input, transforms it, and produces a specific output that the next module consumes. This style fits the project naturally because the work is sequential by nature: data flows from raw form, through cleaning and engineering, into a model, and finally into a human-readable explanation. The five modules are shown in Figure 4.3.

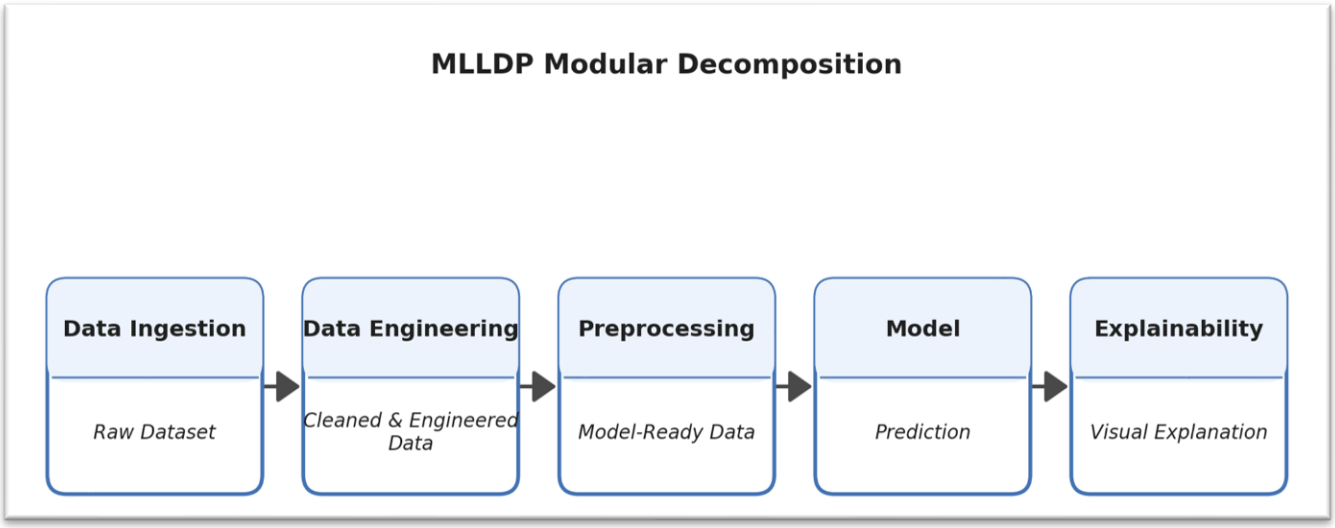

Figure 4.3: Modular Decomposition of the MLLDP system

1- Data Ingestion Module: Loads the source dataset from its storage location and produces the raw dataset as its output.

2- Data Engineering Module: Takes the raw dataset, removes leakage and bad values, and creates new informative features. Its output is a cleaned and engineered dataset.

3- Preprocessing Module: Takes the cleaned dataset and applies the steps needed to make it model-ready: missing value imputation, outlier clipping, categorical encoding, and feature scaling. Its output is a model-ready dataset.

4- Model Module: Takes the model-ready dataset and produces a default-probability prediction. The same module is responsible for both training the model on historical data and using the trained model on new applicants.

5- Explainability Module: Takes the prediction and produces a visual explanation that identifies which features pushed the score up or down for the specific applicant.

4.5 System Organization

The previous sections described the components of the system and the way they are arranged. This section describes how the system actually behaves at runtime by tracing the data as it moves between the modules and the shared data store. Because the MLLDP system follows a pipeline data-flow model, we use Data Flow Diagrams (DFDs) to capture this behavior. The Training Pipeline and the Inference Pipeline run at different times and exchange data only through the Saved Artifacts data store, so we describe each pipeline in its own diagram.

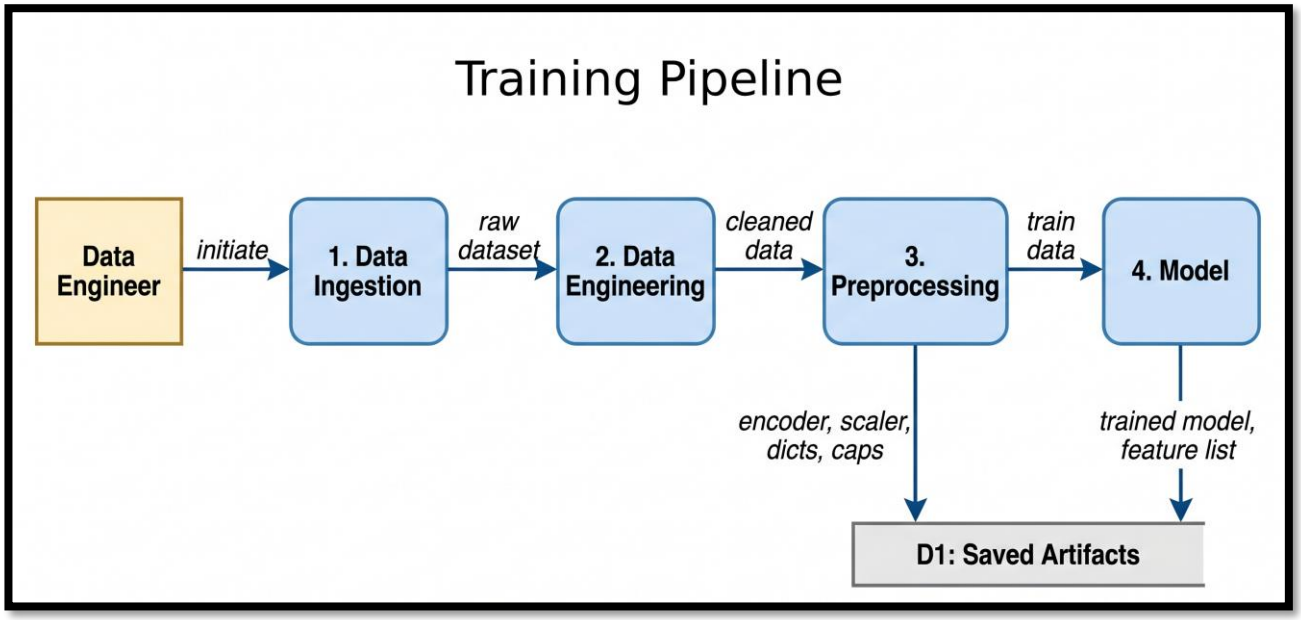

Figure 4.4: Data Flow Diagram of the Training Pipeline.

The Training Pipeline shown in Figure 4.4 runs once, offline, and is initiated by the Data Engineer. The **Data Ingestion** module reads the raw LendingClub dataset and forwards it to the **Data Engineering** module, which removes leakage columns, drops bad rows, and creates new ratio features. The cleaned dataset enters the **Preprocessing** module, which fits the encoder, the scaler, the imputation dictionaries, and the clip caps on the training set and writes them to the Saved Artifacts data store. The same module also produces the model-ready training data that is passed to the **Model** module. The Model module trains the final classifier and writes the trained model and the locked feature list back to the Saved Artifacts store. At this point the offline work is complete.

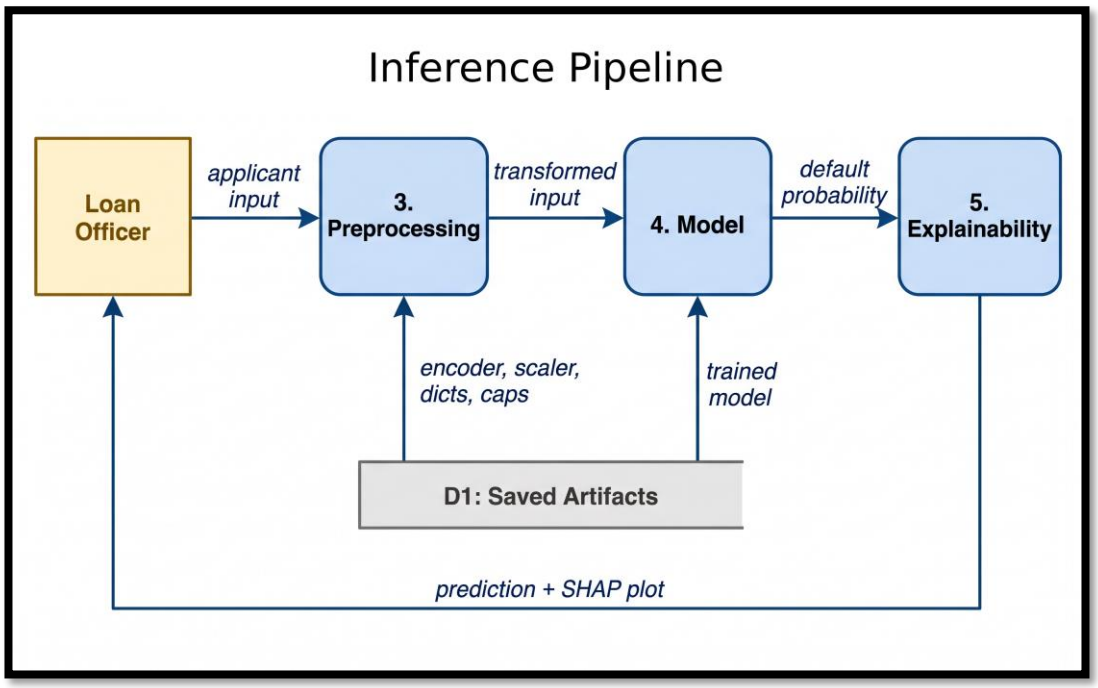

Figure 4.5: Data Flow Diagram of the Inference Pipeline.

The Inference Pipeline shown in Figure 4.5 runs on demand inside the web application every time a Loan Officer submits an applicant. The **Preprocessing** module reads the saved encoder, scaler, dictionaries, and clip caps from the Saved Artifacts store and applies the same transformations that were used during training. The transformed input is passed to the **Model** module, which reads the trained model from the store and returns the default probability. The **Explainability** module takes this probability and generates a SHAP explanation that identifies which features pushed the score in each direction. The prediction together with the SHAP plot is returned to the Loan Officer through the web interface.

4.6 Graphical User Interface Design

The user interface is designed to be minimalistic and intuitive, focusing on the needs of the Loan Officer. The design ensures that the prediction process is straightforward, requiring no technical expertise to operate.

4.6.1 Main Input Interface

The primary interface is designed to be approachable and efficient. It features a centred layout that guides the user's eye directly to the data entry points. Key design elements include:

Input Fields: Clearly labeled text boxes for essential financial indicators, specifically Annual Income ($), Loan Amount ($), Interest Rate (%), and Employment Length (Years). Each field includes placeholder text to guide the user on the expected input format.

Action Button: A prominent "Predict Risk" button is centered at the bottom, serving as the trigger to initiate the inference pipeline.

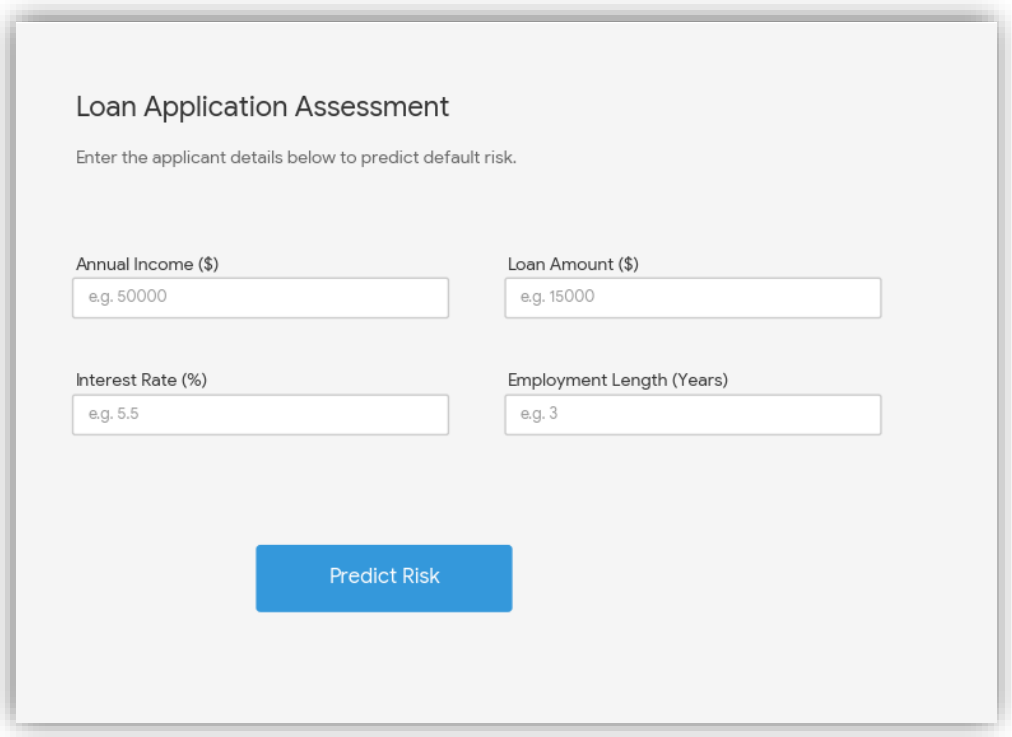

Figure 4.6: Proposed Main Input Interface

4.6.2 Prediction Result Interface

Once the prediction is triggered, the interface updates to display the results. The design prioritizes immediate visual communication of risk:

High-Contrast Status Banner: The most critical information is displayed at the top in a large, colored banner. In the case of a default prediction, a red background with a warning icon alerts the officer to "HIGH RISK DETECTED".

Explainability Visualization (SHAP): To support the decision, the system displays a Waterfall Chart titled "Why did the model make this prediction?". This visual clearly distinguishes between:

Risk Factors (Red): Features pushing the prediction toward default.

Positive Factors (Blue): Features lowering the risk of default.

This layout allows the officer to explain the rejection or approval rationale to the applicant confidently.

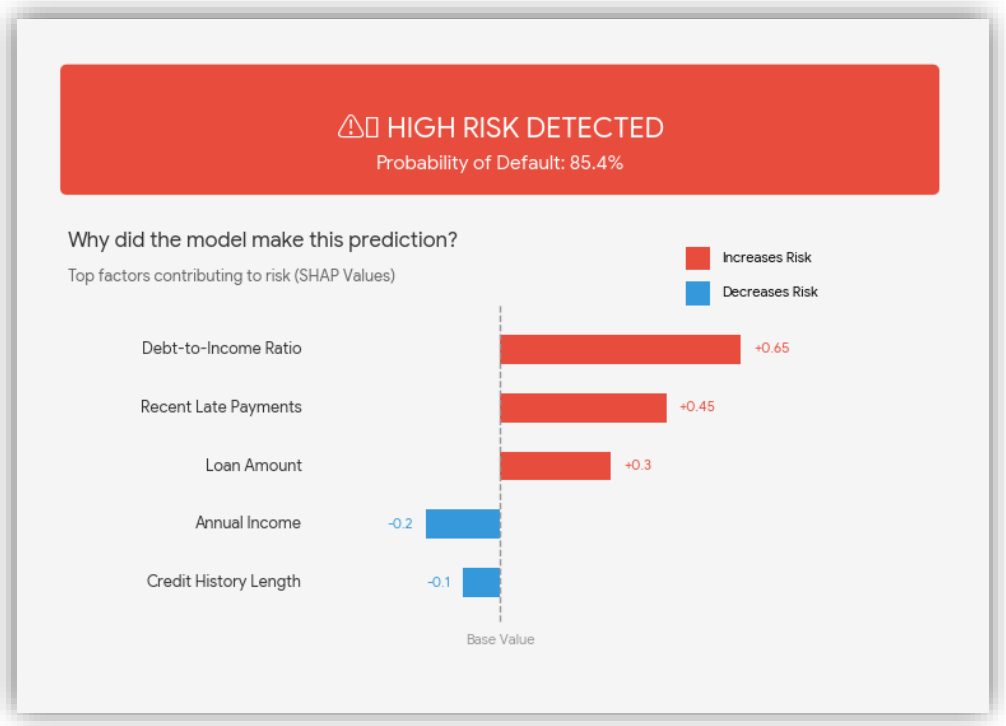

Figure 4.7: Prediction Result and Explanation Interface

5. Implementation

5.1 Implementation Requirements

This section describes the hardware and software that we used to implement the current phase of the MLLDP project. It also explains the programming language and the main tools that we chose. The goal is to show why each of these choices is suitable for a machine learning project that works with a large financial dataset, and to prove that our system can run on a normal computer without needing expensive hardware.

5.1.1 Hardware Requirements

We picked the hardware for this project based on the need to work with a large lending dataset that contains over 1.3 million rows and more than one hundred features. Our personal laptops did not have enough memory to handle this, so we decided to use Google Colab Pro, a paid cloud-based platform from Google that gives us access to virtual machines with more RAM than the free version. We chose Colab Pro because it gives us enough computing power for heavy data tasks without paying for strong physical hardware. The required hardware is described below:

1- Cloud Computing Environment: We used Google Colab Pro with the High-RAM runtime as our main development environment. The High-RAM option gives us a virtual machine with around 50 GB of RAM, which is needed to load the full Lending Club dataset and process it in memory without crashing.

2- Client-Side Machine: Each one of us only needs a normal laptop with at least 8 GB of RAM, a modern processor such as Intel Core i5 or similar, and a stable internet connection to open Colab through a web browser.

3- Cloud Storage: We used Google Drive as the main storage for the raw dataset and the cleaned files. It gives us enough free space and allows both of us to open the files from any device.

5.1.2 Software Requirements

We selected our software stack to make sure the tools are compatible, easy to use, and well supported by the community. All of the software we picked is open-source or free to use, which matches our goal of keeping the project low-cost and easy to reproduce. The required software is listed below:

1- Google Colab (Jupyter Notebook Environment): We used this as our main development environment. It runs code cell by cell, which helped us check the output after every step. This was very useful during the exploratory data analysis and the cleaning phase.

2- Google Chrome (Web Browser): We used it to open Google Colab and to manage files in Google Drive. Any modern browser such as Microsoft Edge or Firefox would also work.

3- Google Drive: We used it as a cloud storage to keep the raw dataset, the work in progress, and the final cleaned data. This way we never lost our work between sessions.

5.1.3 Programming Language

The main programming language that we used for this project is Python 3.9. We chose Python for several reasons that make it a good fit for machine learning and data science projects:

1- Rich Ecosystem of Libraries: Python has a wide range of open-source libraries for data handling, analysis, and visualization, so we did not have to build these functions ourselves.

2- Readability and Simplicity: Python has a clean and simple syntax, which makes the code easier to write, read, and fix.

3- Industry Standard: As shown in the related work in Chapter 2, Python is the most used language in both academic and industrial loan default prediction studies.

5.1.4 Tools and Technologies

This section presents the main tools and libraries that we used during the current implementation phase. We picked each tool based on how common it is in the machine learning community and how directly it helped us with data loading, cleaning, and exploratory analysis. These tools are the same standard stack used in most of the related work. The tools are listed below:

1- Opendatasets: We used this small helper library as the first step to download the Lending Club dataset directly from Kaggle into our Google Colab notebook using Kaggle API credentials. This saved us from manually uploading a large compressed file to the notebook.

2- Pandas: After downloading the dataset, we used Pandas as the main library for handling the data. We used it to load the Lending Club CSV file, calculate the percentage of missing values, drop the columns that we did not need, and do the initial exploratory data analysis. Its DataFrame structure is the standard way to work with tabular data in Python.

3- NumPy: We used NumPy for numerical operations and for working with arrays. NumPy is the base that Pandas is built on, so it is a core part of our data processing pipeline.

4- Matplotlib: We used Matplotlib to create simple plots and charts during the exploratory data analysis phase. It helped us visualize the distribution of important features and check the shape of the data.

5- Seaborn: We used Seaborn on top of Matplotlib to make more informative statistical plots. Seaborn made it easier for us to visualize the relationships between features and to show the cleaning results in a clear way.

6- Scikit-Learn: We used Scikit-Learn for many parts of the data preparation and baseline modeling work. We used it for splitting the data into training, validation, and test sets with ***train_test_split***, for encoding the categorical columns with ***OneHotEncoder***, for scaling the numerical features with ***StandardScaler***, and for training the baseline models such as ***LogisticRegression*** and ***RandomForestClassifier***. We also used its metric functions like ***accuracy_score***, ***precision_score***, ***recall_score***, ***f1_score***, and ***roc_auc_score*** to evaluate model performance.

7- LightGBM: We used LightGBM as one of the main gradient boosting libraries in our baseline algorithm comparison. LightGBM is well-known in the machine learning community for being fast and accurate on tabular data. It eventually became our champion model in the feature selection and final baseline phases.

8- XGBoost: We used XGBoost as a second gradient boosting library in our baseline algorithm comparison. Including two strong boosting libraries side by side gave us more confidence that the patterns we saw in the feature importance analysis were real and not just specific to one algorithm.

9- Joblib: We used Joblib to save the fitted encoders and the dictionaries we built during data preparation, such as the one-hot encoder, and the median and mode dictionaries. Saving these files lets us reuse the exact same preprocessing later in the Streamlit web app, so that any new applicant input is encoded in the same way as the training data.

We chose this technology stack for two main reasons. First, these tools are the standard in data science, and the same combination of Python, Pandas, and Matplotlib is used in most of the related work that we reviewed in Chapter 2. Second, they are all free, well-documented, and fully compatible with Google Colab, so our whole pipeline runs without any licensing or setup problems.

5.2 Implementation Details

This section describes the implementation work of the MLLDP project. It covers how the system was set up, the structure of the data we worked with, and the main procedures we followed. It also discusses the unexpected problems we faced and how we solved them.

5.2.1 Deployment and Installation

Since we used Google Colab as our development environment, the deployment and installation process was simple. Colab comes with Python and most data science libraries already installed, so we did not need to install them manually. The only extra setup steps we had to do are listed below:

1- Google Colab Pro Subscription: Since the free version of Google Colab did not give us enough RAM to load the full Lending Club dataset, we subscribed to Google Colab Pro and selected the High-RAM runtime. This subscription was the first step in setting up our development environment.
2- Kaggle Dataset Download: The Lending Club dataset is hosted on Kaggle, so we used the Opendatasets library to download it directly into the Colab environment. This required entering our Kaggle API credentials (username and key) when the download command runs for the first time.
3- Data Dictionary Upload: We uploaded the Lending Club Data Dictionary file (***LCDataDictionary.xlsx***) manually to the Colab session so that we could read it in our code and use it to understand the meaning of each column.
4- Low Memory Flag: When loading the dataset using Pandas, we had to set the parameter ***low_memory=False*** in the ***pd.read_csv()*** function. This was necessary because the dataset contains mixed data types in some columns, and without this flag, Pandas would show warning messages during loading.

5.2.2 Data Structures Description

The main data structure used throughout the implementation is the Pandas DataFrame. A DataFrame is a two-dimensional table where each column represents a feature like loan amount and each row represents a single loan application record. We stored the entire dataset in a single DataFrame variable called ***df***, and after filtering it to keep only the relevant loan statuses, we continued working with the same variable. After the train, validation, and test split (60/20/20), we worked with six separate structures: ***X_train***, ***X_val***, and ***X_test*** DataFrames holding the features, and ***y_train***, ***y_val***, and ***y_test*** Series holding the target labels.

The original dataset that we downloaded from Kaggle is a compressed CSV file named (***accepted_2007_to_2018Q4.csv.gz***). It contains historical loan data from the Lending Club platform, covering the period from 2007 to the fourth quarter of 2018. The full file has over 2.2 million rows and 151 columns. After loading and filtering it to keep only the rows where the loan status is **"Fully Paid"** or **"Charged Off"**, we ended up with a dataset of about 1.34 million rows and 151 columns to start working with. The columns were a mix of numerical features and text-based features.

5.2.3 Procedures Description

The data preparation phase of our implementation was organized into the following sections inside our notebook. Each section focuses on a specific task in the pipeline. Below is a description of what we did in each section along with the most important code cells:

1- Data Loading and Initial Inspection

In this section we downloaded the Lending Club dataset from Kaggle using the Opendatasets library, then loaded it into a Pandas DataFrame. After upgrading to Google Colab Pro with the High-RAM runtime, we loaded the full dataset of over 2.2 million rows without any memory issues. After loading, we printed the first 10 rows to make sure the data looks correct.

Code cells:

```
!pip install opendatasets -q
import opendatasets as od
dataset_link = "https://www.kaggle.com/datasets/wordsforthewise/lending-club"
od.download(dataset_link)
```

```
file_path = '/content/lending-club/accepted_2007_to_2018Q4.csv.gz'
df = pd.read_csv(file_path , low_memory=False)
print("Displaying the first 10 rows from the dataset")
df.head(10)
```

2- Exploring the Target Variable (loan_status)

In this section we focused on the target column ***loan_status***, which is what our model will need to predict. We first printed the value counts to see all the different statuses that exist in the data. We found that the column has nine unique statuses. Some of them represent ongoing loans, like *Current, Late (31-120 days), Late (16-30 days),* and *In Grace Period*. Others represent finalized loans, like *Fully Paid, Charged Off,* and *Default*. Since our goal is to predict whether a loan will default or not, we filtered the dataset to keep only rows where the status is **"Fully Paid"** or **"Charged Off"**. These two statuses make up the bulk of the finalized loans and map directly to our prediction target. We excluded the other categories for different reasons: the ongoing statuses (Current, Late, In Grace Period) describe loans that have not finished yet, the *Default* category had only 40 rows which is too small to be useful, and the *"Does not meet the credit policy"* rows were issued under a deprecated lending policy and would skew the data. After filtering, the dataset contained 1,345,310 rows split between 1,076,751 Fully Paid loans and 268,559 Charged Off loans, an imbalance of about 4 to 1. We then visualized the filtered distribution using a simple bar chart to see the class balance.

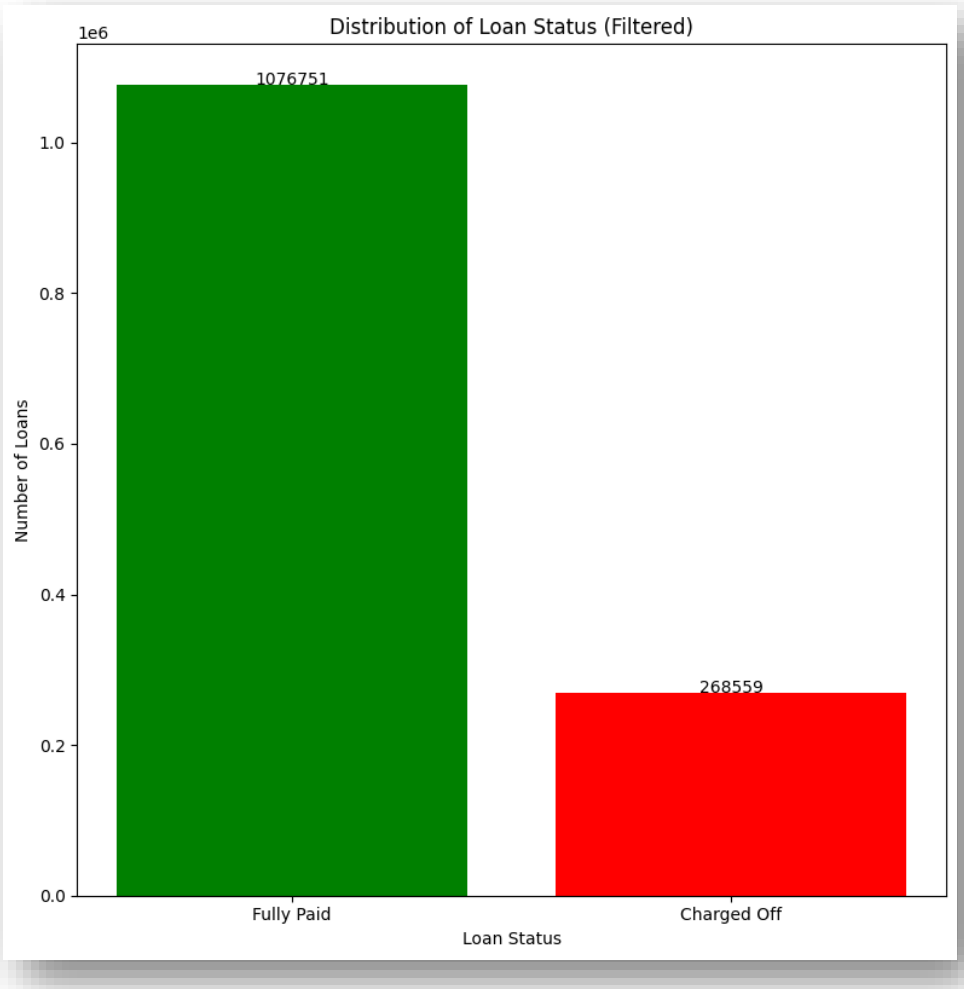

Figure 5.1: Distribution of Loan Status bar chart

3- Exploring the Dataset Features

In this section we explored the remaining columns in detail to understand what kind of data we have. First, we separated the columns into two lists based on their data type: numerical columns and text columns. Then we looped through each text column to see how many unique values it has and what those values look like. After that, we looped through each numerical column to count the unique values and find out which columns are completely empty or which ones have only one value since those columns are useless for prediction. We also checked for negative values in numerical columns and for duplicate columns that have the exact same data in every row. Finally, we printed the percentage of missing values for each column.

4- Initial Data Cleaning

Based on what we found during the exploration, we performed the first round of cleaning in three steps. The dataset entered this section with 151 columns and 1,345,310 rows.

Step 1 dropped every column with more than 50% missing values, because if more than half the data is missing the column gives the model very little to learn from. This step removed **58 columns**. Most of these were related to joint applications, hardship plans, debt settlements, and second-applicant fields, all of which are only filled in for a small fraction of loans. The column count went from 151 to 93.

Step 2 dropped columns that have only one unique value across all rows, since a constant column gives the model zero information. This step removed **5 columns**: ***pymnt_plan***, ***out_prncp***, ***out_prncp_inv***, ***policy_code***, and ***hardship_flag***. The column count went from 93 to 88.

Step 3 dropped three specific columns we identified during exploration. ***id*** and ***url*** are unique identifiers with one different value per row, so they carry no predictive information. ***funded_amnt*** was dropped because it carries almost the same information as ***loan_amnt*** in the vast majority of rows, with a small fraction differing when a loan was only partially funded. Although our multicollinearity filter in Phase 8 was specifically designed to catch this kind of redundancy and would have dropped ***funded_amnt*** automatically, removing it here kept the early dataset cleaner and cut one redundant column before any downstream phase had to touch it. The column count went from 88 to **85**, ready for the advanced cleaning phases.

Code cell:

```
cols_to_drop = ['id', 'url', 'funded_amnt']
df = df.drop(columns=cols_to_drop)
```

5- Advanced Data Cleaning, Feature Engineering, and Machine Learning Preparation

Before we made any advanced changes to the dataset, we first did a full column review using the official Lending Club Data Dictionary. We loaded the dictionary file (***LCDataDictionary.xlsx***) and looped through every remaining column in our DataFrame to print its name next to its official description. This step gave us a clear understanding of what each column actually means and helped us spot which columns are leakage, which ones need cleaning, and which ones can be combined into stronger features. The decisions in all following phases came directly from this column review.

After the column review we organized the advanced work into 11 phases. Each phase has a clear and limited goal which made it easier to track our progress and check our work. The 11 phases are described below.

Phase 1: Manual Cleanup

In this phase we removed columns that we already identified as problematic during the column review with the data dictionary, and we fixed two row-level issues that we found in the same review. The dropped columns fall into three categories, each with a different reason for removal.

The first category is **post-decision data leakage**. These are columns that were only filled in after the loan was approved or after the borrower started repaying. If we had kept them, the model would have cheated by using future information to make predictions, since none of this data would be available at the time of a real new application.

The second category is **free-text and constant columns**. These do not give the model any usable signal. Two of them (***title*** and ***emp_title***) contain thousands of unique free-text values, and ***application_type*** becomes a single constant value "Individual" after we drop the joint application rows in this same phase.

The third category is **non-transferable identifiers**. ***Grade*** and ***sub_grade*** are Lending Club's own internal risk labels, which depend on a proprietary grading system that does not exist in Saudi Arabia. ***Addr_state*** and ***zip_code*** use US-specific geography, and we also wanted to avoid using a person's location to judge their creditworthiness, which is problematic from a fair lending perspective. The full list of dropped columns is shown in Table 5.1.

Table 5.1: Columns Dropped in Phase 1

Category	Columns to Drop
Post-decision data leakage	funded_amnt_inv, total_pymnt, total_pymnt_inv, total_rec_prncp, total_rec_int, total_rec_late_fee, recoveries, collection_recovery_fee, last_pymnt_d, last_pymnt_amnt, last_credit_pull_d, last_fico_range_high, last_fico_range_low, debt_settlement_flag, disbursement_method, initial_list_status
Free-text and constant columns	title, emp_title, application_type
Non-transferable identifiers	grade, sub_grade, addr_state, zip_code

In total we dropped 23 columns in this phase, which brought us from 85 columns down to 62.

We also fixed two row-level issues. First, we removed all the joint application rows because the income and DTI for these rows only showed the values of the main applicant after we had already dropped the joint columns, which would have confused the model. This removed 25,800 rows. Second, we replaced any negative ***dti*** value with NaN because we found out that Lending Club uses negative numbers like -1 as a placeholder when they could not calculate the real DTI. Keeping the negative values would have made the median lower than its real value during imputation in Phase 4.

Phase 2: Row-wise Feature Engineering

In this phase we worked on the columns one row at a time. The strict rule was that every change had to use values from one single row only. We did not use any value that came from another row like a median or a mean. If we had used those values we would have created data leakage, because we had not split the data yet at this point. The phase was divided into two parts: Part A for cleaning existing columns and Part B for creating new features.

Part A: Cleaning Existing Columns

Some of the remaining columns were stored in a messy text format that the model could not use directly. In this part we converted three of these columns into clean numeric values. First, we converted the ***term*** column from text values like "36 months" into the number 36 by removing the word "months". Second, we converted the ***emp_length*** column from text values like "10+ years", "< 1 year", and "5 years" into real numbers, where "10+ years" became 10 and "< 1 year" became 0. Before doing this conversion, we created a new flag column called ***emp_length_was_missing*** to remember which rows had a missing value, because a missing employment length often means the borrower is unemployed and the model should know that.

Third, we converted the ***earliest_cr_line*** column into a new column called ***credit_history_months***. The original column contained a date string like "Aug-2003" which the model could not use. We measured the time in months from this date up to the ***issue_d*** date (the loan issue date). We did not measure up to today's date because the loan officer at the time of approval could only see the credit history that existed up to the issue date. After creating ***credit_history_months***, we dropped both ***earliest_cr_line*** and ***issue_d*** since the new column already contained the information we needed.

Part B: Creating New Features

In this part we created seven new ratio columns by combining existing values to show credit risk patterns more clearly. The original columns by themselves did not always show the full picture. For example, a high income alone does not tell us whether a loan is affordable, but the installment divided by the monthly income does. All seven calculations were still row-wise, so they were safe to do before the train, validation, and test split.

We added a small +1 to every denominator in our formulas to avoid division by zero. For very large numbers like income, the +1 is so small that it does not change the result. For small numbers, it just protects the calculation from crashing. For three of the seven ratios, the source columns could be NaN for borrowers with no credit history. Since the +1 only protects against zero and not NaN, we filled the resulting NaN values in those ratios with 0. This was still a row-wise operation because every row received the same constant 0, with no statistics taken from other rows. The seven new features are listed in Table 5.1.

Table 5.2: Engineered Features in Phase 2 Part B

	Feature Name	Formula	What It Represents
1	installment_to_income_ratio	installment / ((annual_inc / 12) + 1)	How much of the borrower's monthly income goes to this loan payment.
2	loan_to_income_ratio	loan_amnt / (annual_inc + 1)	The size of the loan compared to the borrower's yearly income.
3	revol_bal_to_income_ratio	revol_bal / (annual_inc + 1)	The borrower's existing revolving debt compared to their income.
4	fico_avg	(fico_range_low + fico_range_high) / 2	A single clean FICO score that replaces the two correlated bound columns.
5	delinquency_rate	num_accts_ever_120_pd / (total_acc + 1)	Serious delinquencies as a percentage of the borrower's total accounts.
6	debt_to_credit_ratio	total_bal_ex_mort / (tot_hi_cred_lim + 1)	The borrower's non-mortgage debt compared to their total credit limit.
7	loan_amnt_to_total_credit	loan_amnt / (tot_hi_cred_lim + 1)	The size of the requested loan compared to the borrower's existing total credit.

After creating ***fico_avg*** in feature 4, we also dropped the original ***fico_range_low*** and ***fico_range_high*** columns because they were now replaced by the cleaner combined version.

Phase 3: Train, Validation, and Test Split

In this phase we split the dataset into three separate sets before doing any further preprocessing. This step was the most important boundary in our pipeline. After this point, the test set was completely locked. We were not allowed to fit any encoder, calculate any statistic, or use any value from the test set during the rest of the work. This rule protected the test set from data leakage so that the final evaluation would give us an honest estimate of how the model performs on truly unseen data.

Before splitting, we converted the target column ***loan_status*** from text into binary numbers, where 0 represented **"Fully Paid"** and 1 represented **"Charged Off".** This made the column ready for any classification algorithm. We then separated the features (X) from the target (y).

We used a **60/20/20 split** instead of the more common **80/20 split** because we needed three sets, not two. This required a two-step process. In the first step, we split the data into 80% (training plus validation combined) and 20% (test). In the second step, we split the 80% portion into 75% training and 25% validation, which gave us the final **60/20/20 ratio**. Both splits were stratified on the target variable to keep the same class ratio in all three sets. We also used a fixed random state value of 42 so that the split would be the same every time we re-ran the notebook, which made our results reproducible. After the split, the training set had 791,706 rows, the validation set 263,902 rows, and the test set 263,902 rows.

Each of the three sets had a clear and limited purpose. Table 5.3 lists what each set was used for, and Figure 5.2 shows the visual proportions of the split.

Table 5.3: Train, Validation, and Test Split Purposes

Set	Percentage	Purpose
Training	60%	Fitting the models and calculating any statistics like medians and scalers.
Validation	20%	Comparing models, tuning hyperparameters, and picking the winner.
Test	20%	Final evaluation, kept locked until the very end.

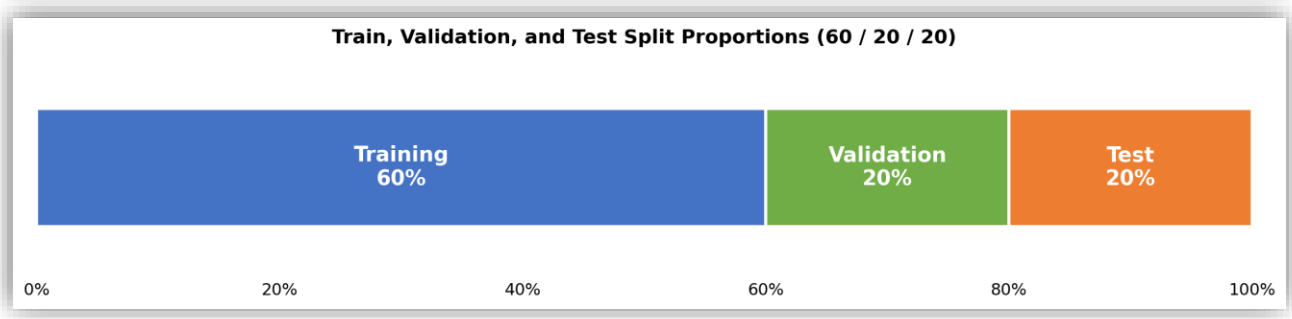

Figure 5.2: Train, Validation, and Test Split Proportions

Phase 4: Missing Values Imputation

In this phase we filled in the missing values that were still left in the dataset. The strict rule for this step was that we calculated all the fill values from the training set only. Then we applied those same values to fill the empty cells in all three sets: training, validation, and test. The validation and test sets never contributed to calculating any fill value, which protected our pipeline from data leakage. We used two different methods depending on the type of the column. For numerical columns we used the median, because loan data has a lot of outliers, like a small number of borrowers with very high incomes, and the median is not affected by these extreme values the way the mean is. For categorical columns we used the mode, which is just the most common value in the column. This is a safe default for filling in a missing category.

We stored the medians and modes from the training set, then applied them to the training, validation, and test sets in one consistent step. We also calculated a fill value for every numerical column even if it had zero empty cells at this point. This way, if the validation or test set later had a surprise empty cell in any column, we already had a fill value ready to use without breaking the pipeline. After the imputation step finished, we verified the result by counting the total number of empty cells in all three sets to confirm that nothing was missed.

Phase 5: Outlier Handling

In this phase we handled extreme values in some of our numerical columns by clipping them to a reasonable range. This step was needed because the dataset contained data quality issues like income values of just one dollar and DTI values above 999 percent. These extreme values would have damaged models that are sensitive to scale, like Logistic Regression so we needed to fix them before any model training. We did this step before feature scaling so that the scaler in the next phase would not be confused by the broken extreme values.

Before applying the clipping, we built a simple outlier scanner that looped through every numerical column in the training set. The scanner flagged a column as a clipping candidate when its maximum value was more than two times its 99th percentile, since this ratio is a clear sign that the column has extreme values that do not represent the rest of the distribution. The nine columns we ended up clipping all came out of this scanner's candidate list.

We used a method called **winsorization**, which means we replaced any value below the 1st percentile with the value at the 1st percentile, and any value above the 99th percentile with the value at the 99th percentile. The strict rule for this phase was that we calculated the percentile cap thresholds from the training set only. We then applied the same caps to all three sets, meaning the training set, the validation set, and the test set. If we had calculated the caps from the full dataset, the validation and test sets would have leaked information into the training process.

We applied this clipping to nine columns in total. Three of them were raw columns from the original dataset: ***annual_inc***, ***dti***, and ***revol_bal***. The other six were engineered ratios that we created earlier in **Table 5.2:** ***installment_to_income_ratio***, ***loan_to_income_ratio***, ***revol_bal_to_income_ratio***, ***delinquency_rate***, ***debt_to_credit_ratio***, and ***loan_amnt_to_total_credit***. We excluded ***fico_avg*** from the clipping because FICO scores are already naturally bounded between 300 and 850 by definition, so the column cannot have outliers.

Phase 6: Categorical Encoding

In this phase we converted every remaining text column in our dataset into numeric format because machine learning models cannot work with text values directly. After the column drops and numeric conversions in the previous phases, we were left with three categorical columns that still needed to be encoded: ***home_ownership***, ***purpose***, and ***verification_status***. None of these columns has a natural order between their values, so it would not make sense to map them to numbers like 1, 2, 3. For this reason we used one-hot encoding for all three of them.

With one-hot encoding, each unique value in the column becomes its own new column with values of 0 or 1. For example, the ***home_ownership*** column was replaced by six separate columns: ***home_ownership_RENT***, ***home_ownership_MORTGAGE***, ***home_ownership_OWN***, ***home_ownership_OTHER***, ***home_ownership_NONE***, and ***home_ownership_ANY***. Each row had a 1 in the column matching its original value and 0 in all the others.

The strict rule for this phase was that we fitted the OneHotEncoder on the training set only, then used it to transform the training, validation, and test sets. This protected the pipeline from data leakage. We used the OneHotEncoder from Scikit-Learn with the option ***handle_unknown='ignore'*** so that any new category appearing in the validation or test set that the training set had not seen would be safely encoded as all zeros instead of crashing the program.

Phase 7: Feature Scaling

In this phase we put all of our continuous numerical columns on the same scale. This step was needed because some machine learning models like **Logistic Regression** rely on the size of feature values when calculating distances and weights. Without scaling, large columns like ***annual_inc*** would dominate small columns like ***dti***, and the model would learn the wrong patterns. Tree-based models like **XGBoost** and **Random Forest** do not care about the scale of features, but we still scaled the data once for all algorithms so that the comparison between them in the later phases would be fair.

We used StandardScaler from Scikit-Learn, which transforms each column so that its mean becomes 0 and its standard deviation becomes 1. We chose this method for two reasons. First, it is the textbook default for feature scaling, which makes it a safe and well-known choice. Second, StandardScaler can be sensitive to outliers, but we had already clipped the extreme values in Phase 5, so this concern was no longer a problem.

The strict rule for this phase was the same as the previous ones: we fitted the scaler on the training set only, then used it to transform the training, validation, and test sets. The validation and test sets never contributed to calculating the mean or the standard deviation, which protected the pipeline from data leakage.

We did not scale every column. The binary columns, like the one-hot encoded category columns and the ***emp_length_was_missing*** flag, were skipped because their values were already 0 or 1. Scaling them would have destroyed their original meaning. We detected the binary columns dynamically by checking which columns had only two unique values that were exactly 0 and 1, instead of hardcoding the list. This made our code flexible to work even if the column structure changed.

Phase 8: Multicollinearity Filter

In this phase we removed pairs of features that carried almost the same information. When two features are highly correlated, they confuse the model in three ways. First, the coefficients of models like Logistic Regression become unstable. Second, the feature importance gets split between the duplicate features. Third, we waste training time on redundant signal. So we needed to identify these pairs and drop one feature from each pair.

We calculated the absolute correlation matrix on the training set continuous columns only. We used the absolute value because both very positive and very negative correlations mean redundancy. We then walked through the upper triangle of the matrix to find every pair of features with a correlation above 0.9, which is our threshold for deciding that two columns are saying almost the same thing. We did not include the one-hot encoded columns in this check because dropping one of them would break the encoding scheme of the original categorical column. Our scan found five pairs of features above the 0.9 threshold, as shown in Figure 5.3.

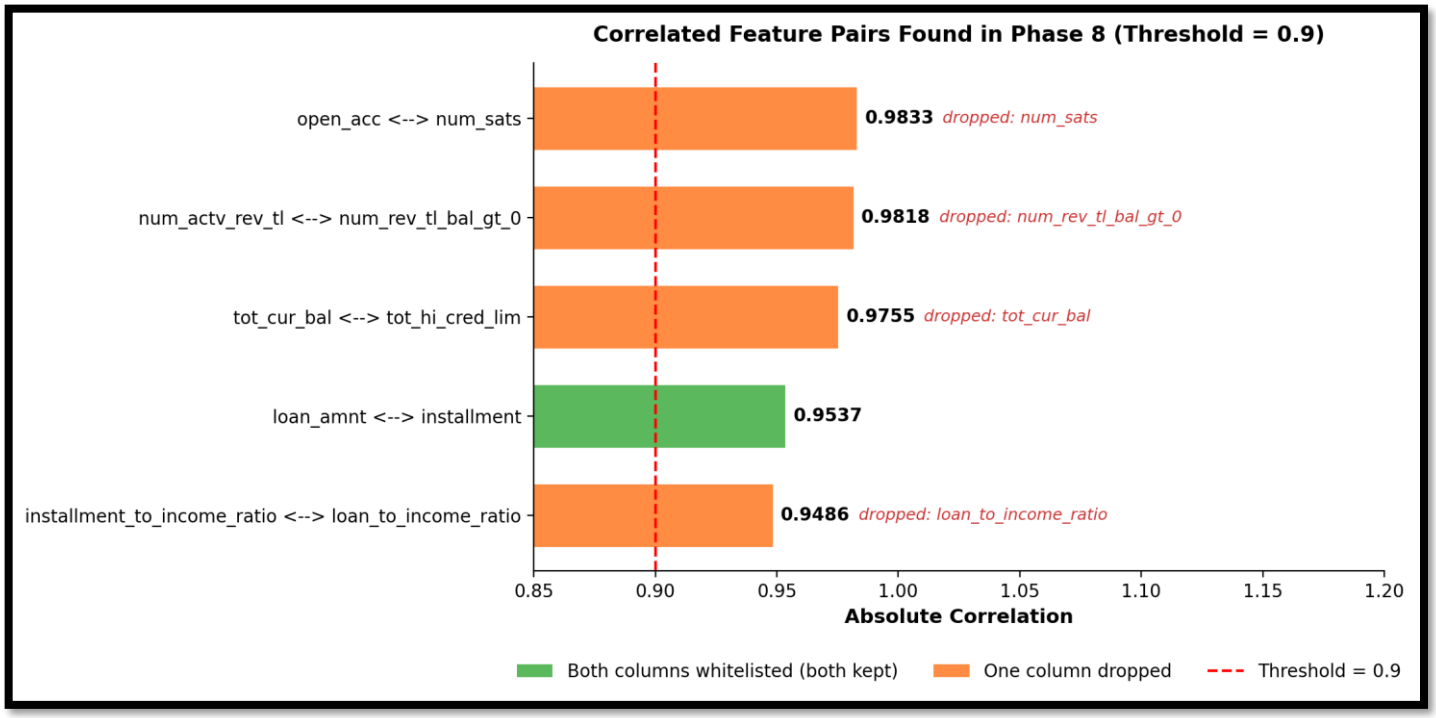

Figure 5.3: Correlated Feature Pairs Found in Phase 8

For each correlated pair, we needed to decide which column to keep and which one to drop. We did not want to use pure correlation math alone, because some columns are too important to drop even if they correlate with something else. For example, ***int_rate*** is the bank's core risk pricing signal, so it must always be kept. To handle this, we built a whitelist of six business-critical columns that are always protected from being dropped: ***int_rate***, ***installment***, ***installment_to_income_ratio***, ***fico_avg***, ***dti***, and ***loan_amnt***.

Using the whitelist together with the target correlation as a fallback, we built a 3-priority decision rule. The rule is summarized in Table 5.4.

Table 5.4: Rules for Dropping Correlated Features

Priority	Condition	Decision
1	One column is whitelisted, the other is not	Keep the whitelisted column, drop the other
2	Both columns are whitelisted	Keep both, skip the pair
3	Neither column is whitelisted	Drop the column with lower target correlation

Out of the five pairs we found, one pair had both columns inside the whitelist, so we kept both columns in that pair without dropping anything. This pair is shown in green in Figure 5.3. The remaining four pairs had at least one non-whitelisted column, so we dropped one column from each pair using the priority rule. These four pairs are shown in orange in Figure 5.3. We removed the four dropped columns from all three sets, the training, validation, and test sets, in one consistent step.

Phase 9: Baseline Algorithm Comparison

In this phase we ran a clean comparison of four representative classical machine learning algorithms on the full preprocessed feature set. The goal was to compare how the main algorithm families perform on our problem before moving to the feature importance analysis in Phase 10. We chose four algorithms that cover the main families used for tabular credit data: ***XGBoost*** and ***LightGBM*** as two strong gradient boosting libraries, ***RandomForest*** as a bagging method, and ***LogisticRegression*** as a simple linear baseline.

A challenge we had to handle in this phase was the imbalance between the two classes in our training set. We had 634,377 fully paid loans (class 0) and 157,329 charged off loans (class 1), which gives an imbalance ratio of about 4 to 1. If we did not address this, the models would be biased toward predicting **"fully paid"** most of the time, because that is the easy way to get a high accuracy on this kind of data. We chose to use the native class weighting parameter of each algorithm instead of generating synthetic samples through **SMOTE**, because class weighting is a cleaner production-quality approach that does not invent fake training examples. For ***XGBoost*** we set ***scale_pos_weight*** to the imbalance ratio, and for the other three algorithms we used ***class_weight='balanced'***.

We trained each of the four models on the training set and then evaluated them on the validation set. The test set was kept locked, as required by the rules we set in Phase 3. We measured five metrics: *accuracy, precision, recall, F1 score, and AUC*. Before showing the results, it is helpful to briefly explain what each of these metrics means. The five metrics are listed below:

1- Accuracy: The percentage of all predictions that the model got right. Accuracy is easy to understand but it can be misleading on imbalanced data.

2- Precision: Out of all the loans the model predicted as **"default"**, the percentage that actually defaulted. High precision means the model has very few false alarms.

3- Recall: Out of all the actual defaulters in the data, the percentage that the model successfully caught. High recall means the model is good at catching the real defaulters and not missing them.

4- F1 Score: The harmonic mean of precision and recall. F1 gives a single number that balances the two and is very useful when one metric is high and the other is low.

5- AUC (Area Under the ROC Curve): Measures how well the model can separate the two classes across all possible decision thresholds. An AUC of 0.5 means the model is guessing, and an AUC of 1.0 means perfect separation. The results of the four models on these five metrics are shown in Table 5.5 and Figure 5.4.

Table 5.5: Validation Set Results for the Four Baseline Algorithms

Algorithm	Accuracy	Precision	Recall	F1 Score	AUC
XGBoost	0.6642	0.3292	0.6648	0.4404	0.7272
LightGBM	0.6549	0.3244	0.6808	0.4395	0.7277
RandomForest	0.8032	0.5574	0.0461	0.0852	0.7115
LogisticRegression	0.6642	0.3266	0.6497	0.4347	0.7191

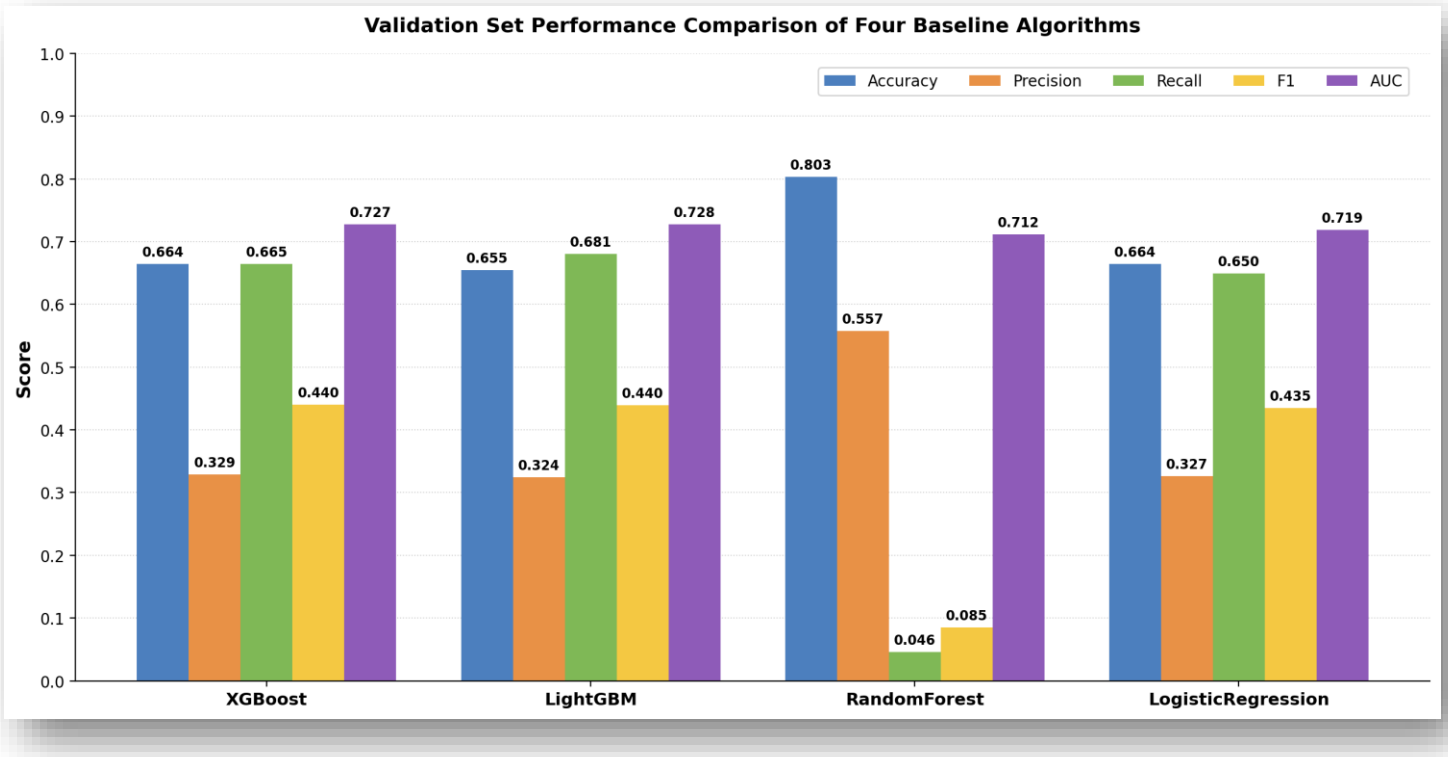

Figure 5.4: Validation Set Performance Comparison of Four Baseline Algorithms

XGBoost did well on every metric. It scored an accuracy of 0.6642, a precision of 0.3292, a recall of 0.6648, an F1 score of 0.4404, and an AUC of 0.7272. The recall of 0.6648 means ***XGBoost*** caught about 66 percent of the real defaulters in the validation set. This is a good result for a model that has not been tuned yet. The F1 score of 0.4404 was the highest in the table. This shows ***XGBoost*** had the best balance between catching defaulters and avoiding false alarms. The precision of 0.3292 looks low, but this is expected on imbalanced data when class weighting is used. The model is trained to predict "default" more often than the natural rate. So it catches more true defaulters, but it also raises more false alarms.

LightGBM performed similarly to ***XGBoost***, with both models close on F1 and AUC. It scored an accuracy of 0.6549, a precision of 0.3244, a recall of 0.6808, an F1 score of 0.4395, and an AUC of 0.7277. The recall of 0.6808 was the highest in the table. This means ***LightGBM*** was the best at catching the real defaulters. The AUC of 0.7277 was also the highest, just a little higher than ***XGBoost***. This means ***LightGBM*** was also the best at separating defaulters from non-defaulters across all decision thresholds. The F1 score was very close to ***XGBoost*** too. The accuracy and precision were a bit lower than ***XGBoost***. This means ***LightGBM*** gave up a small amount of accuracy to catch more defaulters.

RandomForest gave us the most surprising result. It had the highest accuracy at 0.8032 and the highest precision at 0.5574, but its recall was only 0.0461 and its AUC was 0.7115, the lowest in the table. These numbers are not a bug. They are a known problem called the *accuracy paradox*, which happens when a model is tested on *imbalanced data*. Our training set is about 80 percent fully paid loans, so a model that just predicts "fully paid" for almost every applicant will be right 80 percent of the time without learning anything useful. The accuracy of ***RandomForest*** is almost exactly 80 percent, which shows this is what the model is doing. The recall of 0.0461 confirms it: out of every 100 real defaulters, ***RandomForest*** caught fewer than 5. The high precision of 0.5574 only describes the small number of cases where the model did predict "default" at all, so it does not mean the model is good. We used native class weighting on all four models, but ***RandomForest*** responds less to it than the gradient boosters. ***RandomForest*** trains many trees in parallel and averages them, which weakens the effect of the weights. ***XGBoost*** and ***LightGBM*** train one tree at a time and apply the class weights directly in the loss function at each step, which is why they reacted to class weighting much more.

LogisticRegression was our simple linear baseline. It scored an accuracy of 0.6642, a precision of 0.3266, a recall of 0.6497, an F1 score of 0.4347, and an AUC of 0.7191. These numbers were close to the two gradient boosters but a bit lower on the metrics that matter most, recall and AUC. This was exactly what we expected from a linear baseline. The job of ***LogisticRegression*** in this comparison was to check that the more complex boosting models were really worth their extra complexity. If ***XGBoost*** and ***LightGBM*** had not beaten ***LogisticRegression*** on recall and AUC, there would have been no reason to use the heavier models. The boosters did beat the linear baseline, even if only by a small amount. This gave us confidence that the boosting family was the right choice for our task.

Putting all four models side by side shows a clear pattern. ***XGBoost*** and ***LightGBM*** were almost tied at the top on every metric that matters for catching defaulters. ***LogisticRegression*** came in third, close behind the boosters but a bit weaker. ***RandomForest*** looked the best on accuracy and precision but failed on the metrics that matter for credit risk. This pattern matches studies from Chapter 2, especially [9] and [11], which both found that boosting algorithms outperform bagging algorithms on imbalanced credit data. For the rest of the project, we needed a model that could catch defaulters, because accepting a person who defaults is much worse than rejecting a safe applicant. So recall and AUC were the most important metrics. Based on these priorities and the results in Table 5.5 and Figure 5.4, ***LightGBM*** came out as the strongest model with the highest recall and the highest AUC. We carried this finding into Phase 10, where we explored feature importance across all four algorithms before locking in the final feature set.

Phase 10: Baseline Training and Cross-Algorithm Feature Importance

In this phase we looked at which features each of the four baseline models from Phase 9 considered most important. The goal was to cross-check feature importance across different model families before we picked a ranking to use for feature selection in Phase 11. Each algorithm was trained on the full set of 82 preprocessed features, and we extracted the importance scores after training. The figures in this phase show only the top 20 features per algorithm to keep them readable in the report, but the importance values were calculated across all 82 features which you can find in the notebook.

We used all four algorithms because each one measures importance in a different way, and seeing the same feature appear at the top across multiple algorithms is a strong signal that the feature carries real predictive value. Each algorithm also played a specific role in this cross-check:

1- LightGBM was already the strongest model from the Phase 9 comparison, with the highest recall and the highest AUC. Including it here was natural because its ranking was the one we were most likely to trust for the feature selection step in Phase 11.
2- XGBoost was the other strong booster from Phase 9. Including it gave us a second tree-based perspective that uses a completely different importance measure (gain instead of split-count). If a feature ranked high in both LightGBM and XGBoost, that was double evidence the feature really mattered.
3- Random Forest brought a bagging perspective to balance the two boosters. Boosting trees build on each other and can over-rely on a small set of features. Random Forest trains many independent trees in parallel, so its ranking is built from many independent votes. This gave us a different angle even though Random Forest performed poorly on recall in Phase 9.
4- Logistic Regression was the only linear model in the comparison. Its importance ranking shows what a simple additive model picks up, which is a useful sanity check against the tree-based rankings. If a feature appeared at the top in both a linear model and the tree models, that was the strongest possible signal.

After choosing the four algorithms, we extracted their importance scores. Each algorithm uses a different mathematical definition of importance, which is summarized below:

1- LightGBM (Split-Count): Counts how many times each feature was picked as a split point across all trees. A feature that is used often as a split point is considered important.
2- XGBoost (Gain): Measures the average improvement that each split brought to the model. A feature with high gain is one whose splits made the biggest contribution to reducing the loss.
3- Random Forest (Gini): Measures how much each feature reduced impurity (Gini index) across the entire forest. A feature that consistently produces purer child nodes is considered important.
4- Logistic Regression (|Coefficient|): Since Logistic Regression has no native feature importance attribute, we used the absolute value of each coefficient as a proxy. This works in our case because all features were scaled to mean 0 and standard deviation 1 in Phase 7, so the coefficient magnitudes are directly comparable.
The top 20 features from each algorithm are shown in Figures 5.5, 5.6, 5.7, and 5.8.

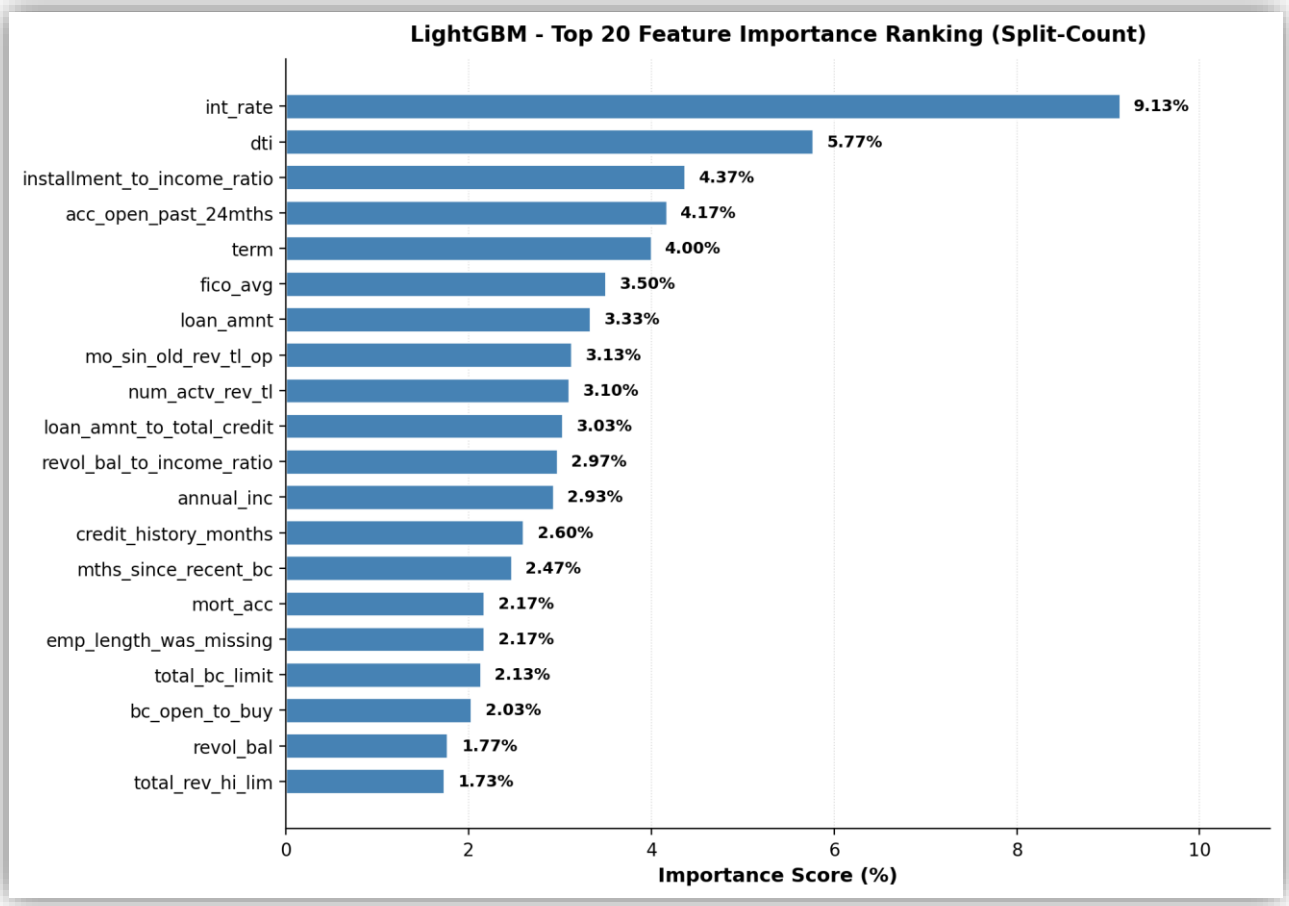

Figure 5.5: LightGBM Top 20 Feature Importance Ranking (Split-Count)

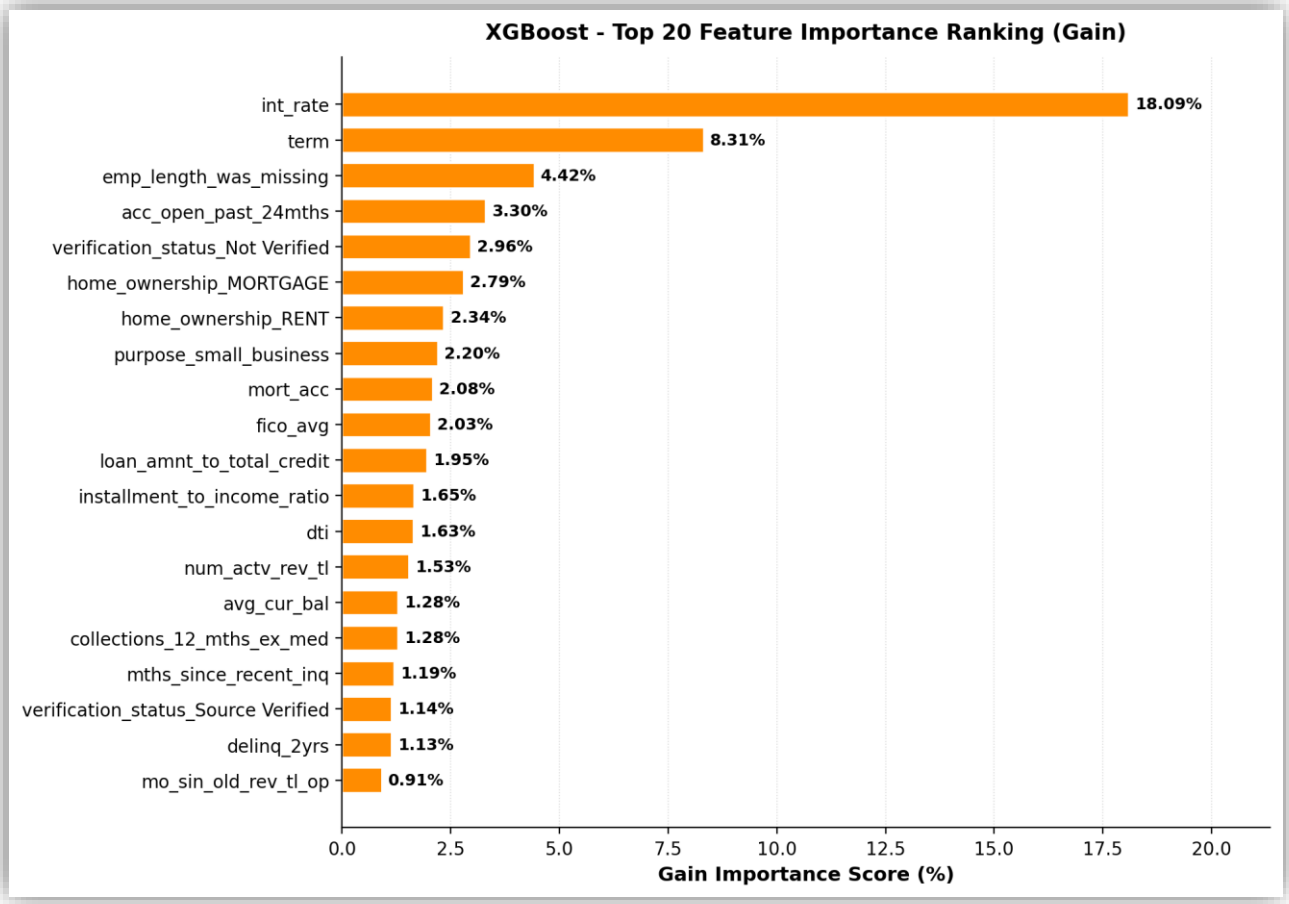

Figure 5.6: XGBoost Top 20 Feature Importance Ranking (Gain)

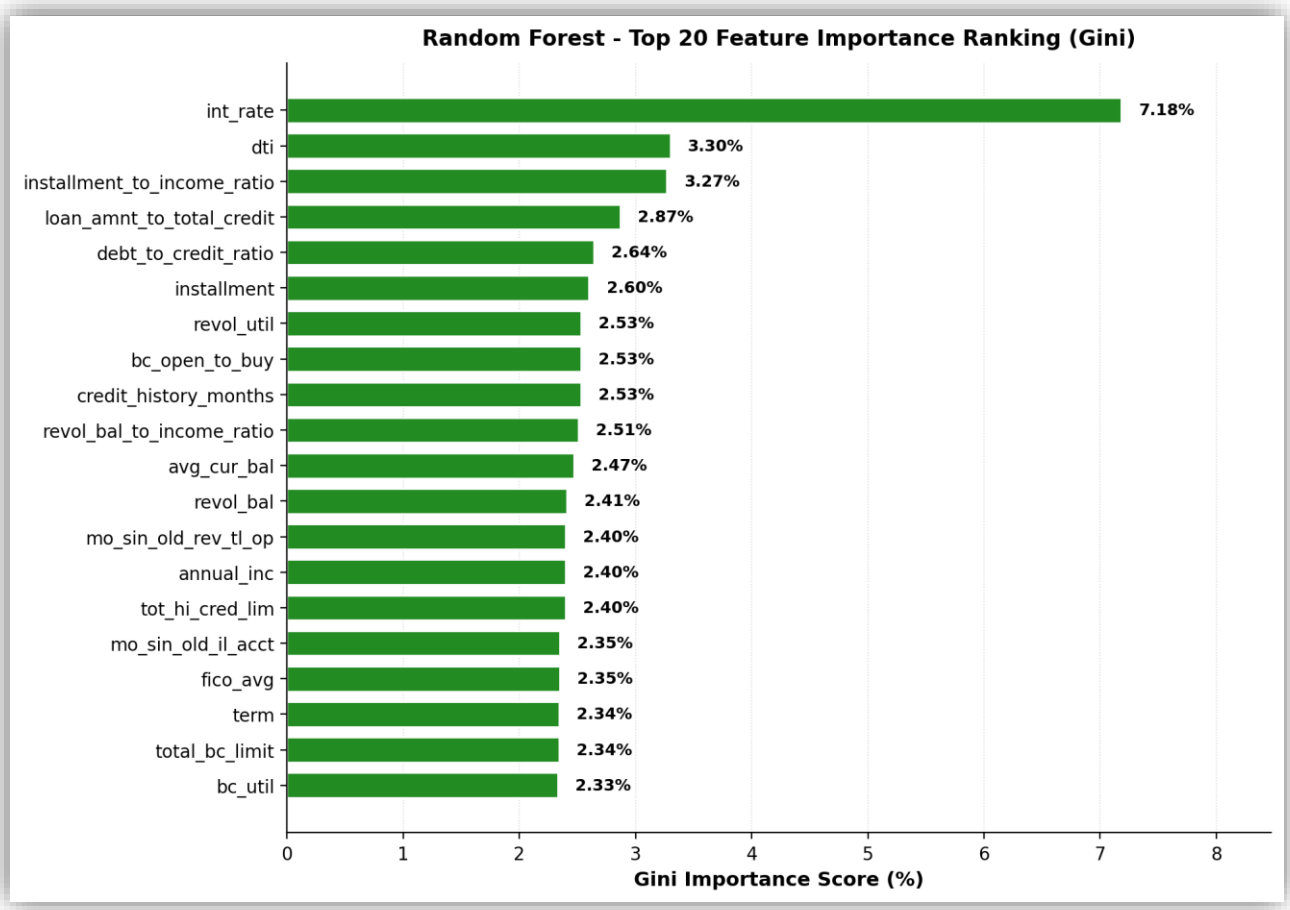

Figure 5.7: Random Forest Top 20 Feature Importance Ranking (Gini)

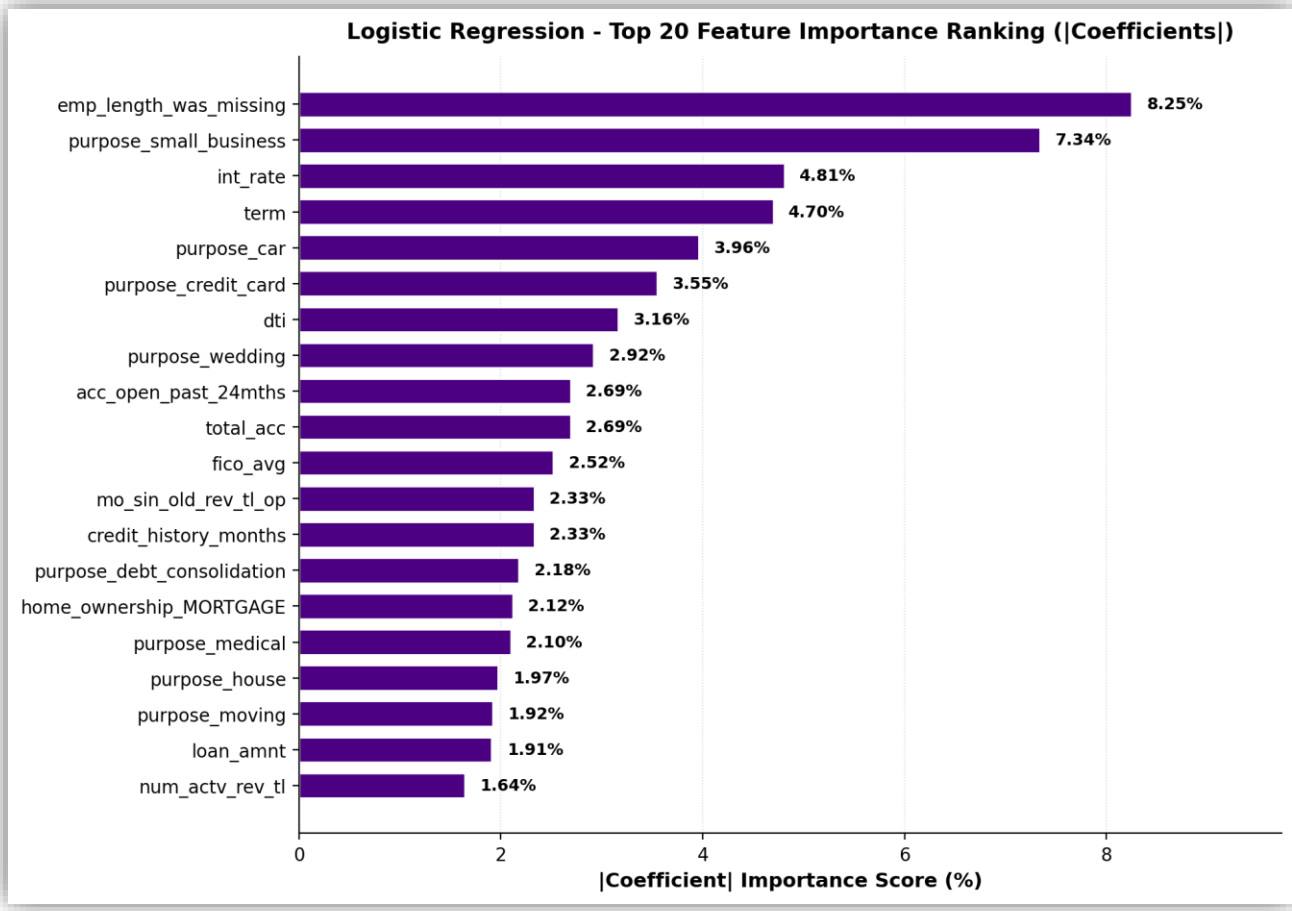

Figure 5.8: Logistic Regression Top 20 Feature Importance Ranking (|Coefficients|)

Looking at the four figures together, a few clear patterns appear. ***Int_rate*** is the most important feature in three out of the four algorithms (LightGBM, XGBoost, and Random Forest), and it is also in the top three for Logistic Regression. This is a strong signal that the interest rate is the single most predictive feature in our dataset, which makes sense because LendingClub already encodes the borrower's overall risk into the interest rate they assign.

The four algorithms differ in how they spread out the importance. LightGBM has a smooth distribution where the top feature (***int_rate***) sits at 9.13% and the rest decrease gradually. XGBoost is heavily concentrated: ***int_rate*** alone holds 18.09% of the total importance, and the top 5 features hold around 37%. This concentration is a known behaviour of gain-based importance and it can hide mid-tier features that still carry real signal.

Random Forest favours continuous features and engineered ratios. Several of our Phase 2 ratios show up in the top 10 (***installment_to_income_ratio***, ***loan_amnt_to_total_credit***, ***debt_to_credit_ratio***, ***revol_bal_to_income_ratio***). Gini importance tends to be biased toward continuous features with many unique values, which is why binary categorical signals like ***home_ownership_RENT*** or ***purpose_small_business*** appear lower in this ranking than they do in the other algorithms.

Logistic Regression's top 20 includes several rare ***purpose_X*** categories such as ***purpose_wedding***, ***purpose_car***, ***purpose_credit_card***, and ***purpose_house***. Linear coefficients can be inflated for rare categories because a small number of training rows is enough to fit a large coefficient. The flag column ***emp_length_was_missing*** also sits at the top of this ranking, which suggests the linear model treats a missing employment length as a meaningful risk signal.

Based on this cross-comparison, LightGBM produced the most balanced and informative ranking. Its split-count importance distributes scores smoothly across the top 20 instead of concentrating on one or two features, and it treats both continuous and categorical features fairly. For these reasons we used LightGBM's ranking as the basis for the feature count sweep in Phase 11, where we tested different cutoffs of top-N features to find the smallest set that still produced a strong model.

Phase 11: Final Clean Feature Lock-in

In this phase we used the ***LightGBM*** ranking from Phase 10 to find the smallest feature set that still kept the predictive signal. From Phase 10's cross-algorithm comparison, ***LightGBM*** had the most balanced and informative ranking, so we used it as the basis for the feature selection sweep. The goal was to find an *elbow point*: the point where adding more features no longer meaningfully improves the model.

To make sure our choice of feature count was robust across different model families, we ran the sweep across three algorithms: ***LightGBM***, ***XGBoost***, and ***LogisticRegression***. For each cutoff size, we trained all three algorithms on the top-N features from LightGBM's ranking and measured performance on the validation set. If all three algorithms agreed on where the elbow point is, we could be confident the cutoff was not overfitted to one algorithm.

We excluded ***RandomForest*** from this sweep for a specific reason. As we saw in Phase 9, RandomForest at default hyperparameters suffers from the accuracy paradox on imbalanced data, which means its recall stays very low regardless of the feature count. Including it in the sweep would not have helped find the elbow point. We bring RandomForest back into the comparison in Section 6.

We tested 10 cutoffs: 10, 15, 20, 25, 30, 35, 40, 45, 50, and 82 (the full feature set). The results across the three algorithms are shown in Table 5.6 and Figure 5.9.

Table 5.6: Validation Set Recall and AUC across the Phase 11 Sweep

N	LightGBM Recall	LightGBM AUC	XGBoost Recall	XGBoost AUC	Logistic Regression Recall	Logistic Regression AUC
10	0.6719	0.7172	0.6675	0.7166	0.6318	0.7087
15	0.6744	0.7209	0.6645	0.7194	0.6376	0.7114
20	0.6765	0.7232	0.6634	0.7211	0.6396	0.7137
25	0.6796	0.7249	0.6638	0.7237	0.6427	0.7151
30	0.6799	0.7251	0.6629	0.7236	0.6435	0.7154
35	0.6805	0.7265	0.6649	0.7257	0.6462	0.7172
40	0.6816	0.7269	0.6661	0.726	0.6477	0.7175
45	0.6811	0.7273	0.6643	0.726	0.6481	0.7177
50	0.6812	0.7271	0.6665	0.7265	0.6493	0.7181
82	0.6808	0.7277	0.6648	0.7272	0.6497	0.7191

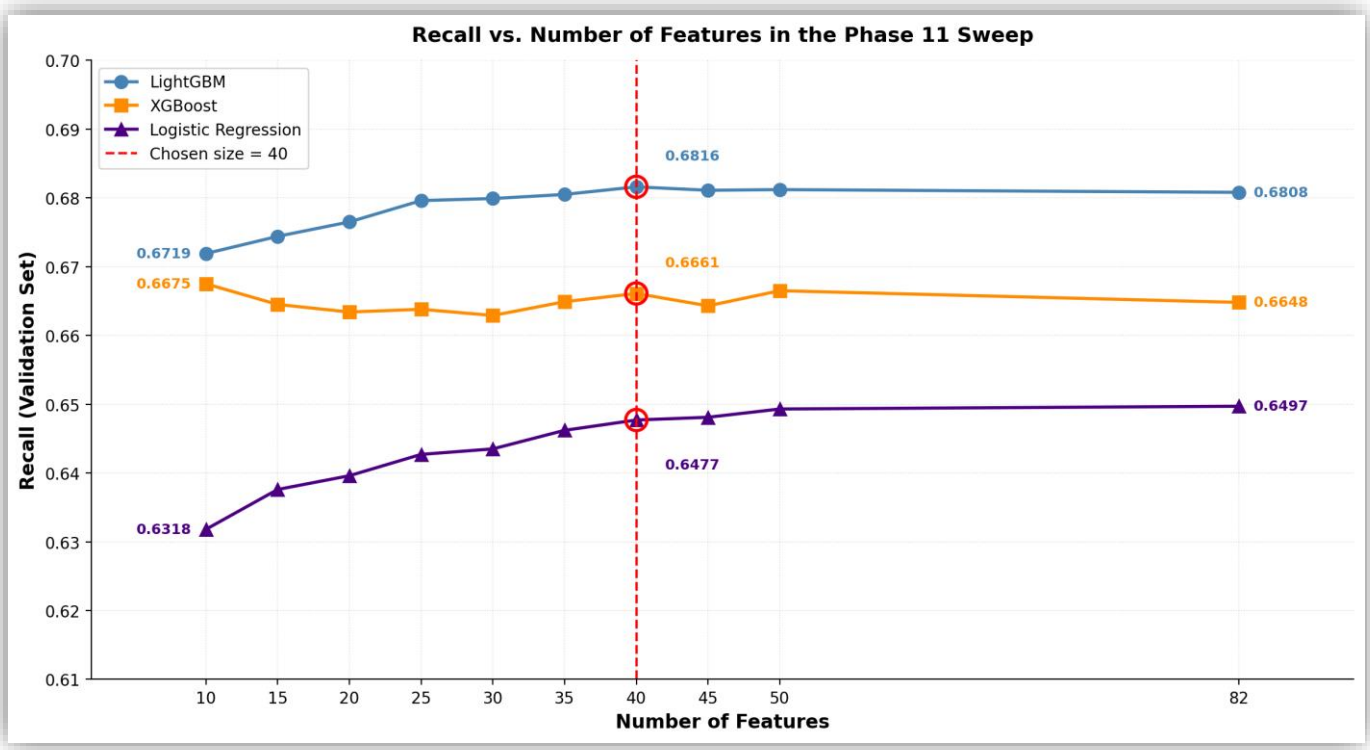

Figure 5.9: Recall vs. Number of Features in the Phase 11 Sweep

Looking at Table 5.6 and Figure 5.9, each algorithm tells a slightly different story but they all support the same conclusion. ***LightGBM*** improves steadily as more features are added, peaks at 40 features with a recall of 0.6816, and then stays essentially flat for the rest of the range. ***XGBoost*** is more variable across cutoffs but stays within a narrow band of about 0.663 to 0.668 the whole time, with no clear improvement from adding more features. ***LogisticRegression*** keeps climbing slowly, but the largest gains happen between 10 and 40 features, and after that the curve flattens.

The common pattern is that 40 features captures almost all the predictive signal in the data. ***LightGBM*** literally peaks at 40, and the small differences between 40 and 82 features are well within the noise we would expect from a single train-validate split. Going from 40 to 82 features more than doubles the model complexity but only buys an AUC improvement of 0.0008 for ***LightGBM*** and 0.0012 for ***XGBoost***, neither of which would change a real lending decision.

Because the lines flatten after 40 features for the two tree-based algorithms, we picked 40 as our final number. This is the smallest number that gives us essentially the best results across the algorithms we care about, which keeps the model simpler and easier to interpret for the loan officer.

We then locked in the top 40 features from **LightGBM's** ranking. These features carry forward to the hyperparameter tuning, the SHAP analysis, and the final Streamlit deployment. The full list of the 40 features and a short description for each one is shown in Table 5.7.

Table 5.7: Final 40 Features Used in the Production Model

	Feature Name	Description
1	*int_rate*	Interest rate on the loan.
2	*dti*	Ratio of the borrower's total monthly debt payments to their monthly income.
3	*installment_to_income_ratio*	Engineered ratio: monthly installment divided by monthly income. Measures how much of the income goes to this loan payment.
4	*acc_open_past_24mths*	Number of credit trades opened in the past 24 months.
5	*term*	Number of payments on the loan in months. Values are either 36 or 60.
6	*fico_avg*	Engineered feature: the average of the lower and upper FICO score range.
7	*loan_amnt*	The listed amount of the loan applied for by the borrower.
8	*mo_sin_old_rev_tl_op*	Months since the oldest revolving account was opened.
9	*num_actv_rev_tl*	Number of currently active revolving trades.
10	*loan_amnt_to_total_credit*	Engineered ratio: loan amount divided by the borrower's total existing credit limit.
11	*revol_bal_to_income_ratio*	Engineered ratio: revolving balance divided by annual income.
12	*annual_inc*	The self-reported annual income provided by the borrower at registration.
13	*credit_history_months*	Engineered feature: number of months between the earliest credit line and the loan issue date.
14	*mths_since_recent_bc*	Months since the most recent bankcard account was opened.
15	*mort_acc*	Number of mortgage accounts.
16	*emp_length_was_missing*	Engineered binary flag: 1 if the original employment length was missing, 0 otherwise.
17	*total_bc_limit*	Total bankcard high credit limit.
18	*bc_open_to_buy*	Total open to buy on revolving bankcards.
19	*revol_bal*	Total credit revolving balance.
20	*total_rev_hi_lim*	Total revolving high credit limit.
21	*emp_length*	Employment length in years (0 means less than one year, 10 means ten or more).
22	*bc_util*	Ratio of total current balance to high credit limit for all bankcard accounts.
23	*num_il_tl*	Number of installment accounts.
24	*total_acc*	Total number of credit lines currently in the borrower's credit file.
25	*mths_since_recent_inq*	Months since the most recent credit inquiry.
26	*revol_util*	Revolving line utilization rate (the amount of credit being used vs all available revolving credit).
27	*installment*	The monthly payment owed by the borrower if the loan originates.
28	*num_rev_accts*	Number of revolving accounts.
29	*debt_to_credit_ratio*	Engineered ratio: total non-mortgage debt divided by total credit limit.
30	*avg_cur_bal*	Average current balance across all accounts.
31	*home_ownership_RENT*	One-hot encoded flag: 1 if the borrower's home ownership status is "RENT", 0 otherwise.
32	*delinq_2yrs*	Number of 30+ days past-due delinquencies in the borrower's credit file in the past 2 years.
33	*mo_sin_old_il_acct*	Months since the oldest bank installment account was opened.
34	*purpose_small_business*	One-hot encoded flag: 1 if the borrower's loan purpose is "small business", 0 otherwise.
35	*num_bc_sats*	Number of satisfactory bankcard accounts.
36	*percent_bc_gt_75*	Percentage of all bankcard accounts that are above 75% of their limit.
37	*num_tl_op_past_12m*	Number of accounts opened in the past 12 months.
38	*mo_sin_rcnt_tl*	Months since the most recent account was opened.
39	*home_ownership_MORTGAGE*	One-hot encoded flag: 1 if the borrower's home ownership status is "MORTGAGE", 0 otherwise.
40	*num_actv_bc_tl*	Number of currently active bankcard accounts.

6- Machine Learning Hyperparameter Tuning

After we locked the final 40 features in Phase 11 of the previous section, we still had one big task left: finding the best possible model for the actual prediction. The default settings of any machine learning library are designed to work for general use cases, but they are almost never the best settings for a specific problem like ours. Hyperparameter tuning is the process of searching for the settings that give the best results on our data. We tuned **four** different algorithms one by one, then combined the three tree-based ones into a **fifth** model called a stacking ensemble, then calibrated the top four candidates to get honest probabilities, and finally picked one as the hero model that the Streamlit web app would use.

This section continues directly from where Section 5 ended. The 40 features and the train, validation, and test split from Phase 3 of Section 5 are reused here without any change. The training set is used to tune and train the models. The validation set is used to compare the tuned models against each other and to pick the best one. The test set is still locked and untouched, exactly as we promised in Section 5. We will only open the test set in Section 7 for the final evaluation.

We organized the work in this section into 12 phases. Phase 1 sets up the tuning environment. Phase 2 prepares the cross-validation strategy. Phases 3, 4, 5, and 6 tune ***LightGBM***, ***XGBoost***, ***RandomForest***, and ***LogisticRegression*** one by one. Phase 7 builds a *stacking ensemble* that combines the three tuned tree-based models. Phase 8 compares all candidates at many decision thresholds on uncalibrated probabilities. Phase 9 calibrates the four candidates that move forward so the predicted probabilities become honest. Phase 10 redoes the comparison on the calibrated probabilities. Phase 11 picks the hero model and the final decision threshold. Phase 12 prints the production summary that documents what the Streamlit app will actually use.

Phase 1: Setup ML Tuning Environment

Before any tuning could start, we had to make three setup decisions that shaped the rest of the section. First, we used the **Optuna** library with the **TPE** (Tree-structured Parzen Estimator) sampler instead of the simpler grid search method. TPE is a Bayesian search algorithm that learns from each trial it tries. It uses the results of past trials to decide which combination of hyperparameters to try next, which finds good settings in much fewer trials than grid search. Second, we mounted Google Drive so that every model and every artifact would be saved permanently. Saving every result to Google Drive meant that if Google Colab disconnected, our progress was safe and we could continue from the last checkpoint without retraining.

Third, we defined a custom *F2 scorer* using ***fbeta_score*** with ***beta=2***. The F2 score is similar to the F1 score that we used in Section 5, but it weights *recall* twice as much as *precision*. This was a deliberate choice for our problem. As we explained in Section 5, accepting a person who defaults is much worse than rejecting a safe applicant. F2 captures that priority directly. We used F2 as the optimization target for **all four algorithms we tuned *LightGBM***, ***XGBoost***, ***RandomForest***, and ***LogisticRegression***, so their cross-validation scores are directly comparable.

Phase 2: Prepare Cross Validation and Final Features

Phase 2 prepared the data and the evaluation strategy that all four tuning phases would share. We loaded the 40 locked features from Phase 11 of Section 5 and used them to subset the training, validation, and test sets. The shapes confirmed everything was in order: 791,706 rows for the training set, and 263,902 rows for both the validation and test sets, each with exactly 40 columns.

We then configured the cross-validation strategy. Cross-validation is a way to test a model on multiple slices of the training data instead of just one. The training data is split into folds, the model is trained on most of the folds, and tested on the remaining one. This process repeats for every fold. The result is a more stable estimate of how the model will perform on new data, and it lowers the chance of picking hyperparameters that just happen to look good on a single split. We used **stratified** cross-validation, which keeps the class ratio of about 80 percent fully paid and 20 percent charged off in every fold. This was important because our data is imbalanced and a non-stratified split could leave some folds with very few defaulters.

We configured two different splitters. For ***LightGBM***, ***XGBoost***, and ***LogisticRegression*** we used 5-fold stratified cross-validation. For ***RandomForest*** we used 3-fold cross-validation instead. ***RandomForest*** is much slower than the boosting algorithms because it has to train hundreds of full decision trees in parallel, so we reduced the number of folds from 5 to 3 to keep the total amount of work practical.

Finally, we instantiated the ***f2_scorer*** that we described in Phase 1, and we computed the imbalance ratio between class 0 (fully paid) and class 1 (charged off) in the training set. The ratio came out to approximately 4.03, meaning there are about 4 fully paid loans for every 1 charged off loan. We will explain how each algorithm uses this ratio in its own phase.

Phase 3: LightGBM Hyperparameter Tuning

Phase 3 was the first of four tuning phases. ***LightGBM*** went first because it is the fastest of the three algorithms, which made it a good place to start. We ran 100 ***Optuna*** trials with 5-fold cross-validation. Each trial picked one combination of hyperparameters, trained ***LightGBM*** five times (one per fold), and returned the average F2 score across the five folds. ***Optuna*** then used the result to decide which combination to try next.

We chose the **F2 score** as the optimization target for ***LightGBM***. The F2 score is the harmonic mean of precision and recall with extra weight on recall. It still rewards a model that catches a lot of defaulters, but it also penalizes the model if precision falls too low. We made this choice because optimizing for raw recall has a known risk: the search algorithm can drift toward an extreme model that predicts almost every applicant as a defaulter, since that gives perfect recall on paper while making the model useless in practice. F2 protects against this by keeping precision in the picture. We used the same F2 target for ***XGBoost***, ***RandomForest***, and ***LogisticRegression*** in the following phases, so all four tuned models are directly comparable.

We also set ***class_weight='balanced'*** on every trial. This is the simple way to handle the imbalance ratio of 4.03 that we computed in Phase 2. With this setting, ***LightGBM*** automatically applies a weight that is proportional to the inverse class frequency, which means the charged off class gets about 4 times more weight than the fully paid class during training. We also fixed the random seed to 42 for the cross-validation splitter, the model itself, and the ***Optuna*** sampler so that the entire tuning run was reproducible.

After 100 trials, the best cross-validation F2 score achieved was 0.5589. We then trained one final ***LightGBM*** model on the full training set using the best hyperparameters that ***Optuna*** found, and we evaluated it on the validation set as a sanity check. This validation evaluation is independent from the cross-validation work done during tuning, since the validation set was not touched during any trial. The results are shown in Table 5.8.

Table 5.8: Tuned LightGBM Performance on the Validation Set

Metric	***Value***
Accuracy	0.6571
Precision	0.3263
Recall	0.6814
F1 Score	0.4413
F2 Score	0.5596
AUC	0.7288

The tuned ***LightGBM*** model reached a validation F2 score of 0.5596, very close to the 0.5589 mean F2 score observed during cross-validation. The gap between the two scores is only 0.0007, which is a strong consistency signal. It meant the tuned model was not overfitting to the cross-validation folds, and it was ready to be compared against the other tuned candidates later in this section.

Phase 4: XGBoost Hyperparameter Tuning

Phase 4 was the second tuning phase. We ran 100 ***Optuna*** trials with 5-fold cross-validation, the same setup we used for ***LightGBM*** in Phase 3. Each trial trained ***XGBoost*** five times (one per fold) and returned the average F2 score across the five folds. We used the same F2 target so that the ***XGBoost*** results would be directly comparable to ***LightGBM***.

XGBoost does not have a ***class_weight='balanced'*** option like ***LightGBM*** does. Instead, ***XGBoost*** has its own parameter called ***scale_pos_weight***. This parameter takes a single number that tells the model how much extra weight to apply to the positive class. We set it to 4.03, which is the imbalance ratio we computed in Phase 2. With this setting, ***XGBoost*** treats every charged off loan as if it were 4.03 fully paid loans during training. We also fixed the random seed to 42 for the model and the ***Optuna*** sampler so the tuning run was reproducible.

After 100 trials, the best cross-validation F2 score achieved was 0.5579. We then trained one final ***XGBoost*** model on the full training set using the best hyperparameters that ***Optuna*** found, and we evaluated it on the validation set as a sanity check. The results are shown in Table 5.9.

Table 5.9: Tuned XGBoost Performance on the Validation Set

Metric	***Value***
Accuracy	0.6603
Precision	0.328
Recall	0.6765
F1 Score	0.4418
F2 Score	0.5579
AUC	0.7296

The tuned ***XGBoost*** model reached a validation F2 score of 0.5579, which exactly matched the cross-validation F2 score of 0.5579. This was a strong consistency signal that the model was learning real patterns and would generalize well. The recall of 0.6765 means that ***XGBoost*** caught about 68 percent of the real defaulters in the validation set, which is similar to what we saw with ***LightGBM*** but with a slightly different precision-recall balance. We saved the tuned ***XGBoost*** model and added it to the list of candidates for the later comparison work in this section.

Phase 5: Random Forest Hyperparameter Tuning

Phase 5 was the third tuning phase. ***RandomForest*** is the slowest of the algorithms we used because, as we explained in Section 5, it has to train hundreds of full decision trees in parallel for every single trial. To keep the total amount of work practical, we made two adjustments compared to the previous phases. First, we ran 50 ***Optuna*** trials instead of the 100 we used for ***LightGBM*** and ***XGBoost***. Second, we used 3-fold cross-validation instead of 5-fold, as we already configured in Phase 2.

We used the **F2 score** as the optimization target for ***RandomForest***, the same metric we used in all the previous tuning phases. Since all algorithms are tuned with the same target, their cross-validation scores can be directly compared at the end. We also kept the ***class_weight='balanced'*** setting that ***RandomForest*** uses by default, the same setup we used for ***LightGBM***. With this setting, every tree in the forest applies inverse-frequency weighting to the two classes during training. We also fixed the random seed to 42 for the cross-validation splitter, the model, and the ***Optuna*** sampler so the tuning run was reproducible.

After 50 trials, the best cross-validation F2 score achieved was 0.5463. We then trained one final ***RandomForest*** model on the full training set using the best hyperparameters that ***Optuna*** found, and we evaluated it on the validation set as a sanity check. The results are shown in Table 5.10.

Table 5.10: Tuned RandomForest Performance on the Validation Set

Metric	***Value***
Accuracy	0.6428
Precision	0.3137
Recall	0.6715
F1 Score	0.4276
F2 Score	0.5468
AUC	0.7136

The tuned ***RandomForest*** model reached a validation F2 score of 0.5468, very close to the cross-validation F2 score of 0.5463, which is the same consistency signal we saw in Phases 3 and 4. The validation recall of 0.6715 is a huge improvement over the baseline ***RandomForest*** from Section 5, which only caught about 5 percent of the defaulters because of the *accuracy paradox*. After tuning, ***RandomForest*** now catches about 67 percent of the defaulters.

However, even after this improvement, it is still slightly behind ***XGBoost*** (recall 0.6765) and ***LightGBM*** (recall 0.6814) on every metric. This was the result we expected based on our literature review: boosting algorithms outperform bagging algorithms on imbalanced credit data, even when the bagging model is properly tuned. With the three tree-based algorithms now tuned, we moved on to Phase 6

Phase 6: Logistic Regression Hyperparameter Tuning

Phase 6 was the fourth tuning phase. ***LogisticRegression*** is a linear model. Unlike the three tree-based models we tuned in Phases 3, 4, and 5, it cannot capture nonlinear interactions between features on its own. We still included it as a fourth tuned candidate so we would have a strong linear baseline to compare the tree models against, similar to what we did in the baseline algorithm comparison in Phase 9 of Section 5.

We ran 100 ***Optuna*** trials with 5-fold cross-validation, the same setup we used for ***LightGBM*** and ***XGBoost***. Each trial trained ***LogisticRegression*** five times one per fold and returned the average F2 score across the five folds. We used the same F2 target so that the ***LogisticRegression*** results would be directly comparable to the three tree models. We also set ***class_weight='balanced'*** so that the imbalance ratio of 4.03 from Phase 2 is handled the same way it is in the tree models. We fixed the random seed to 42 for the cross-validation splitter, the model, and the ***Optuna*** sampler so the tuning run was reproducible.

After 100 trials, the best cross-validation F2 score achieved was 0.5406. We then trained one final ***LogisticRegression*** model on the full training set using the best hyperparameters that ***Optuna*** found, and we evaluated it on the validation set as a sanity check. The results are shown in Table 5.11.

Table 5.11: Tuned LogisticRegression Performance on the Validation Set

Metric	*Value*
Accuracy	0.6639
Precision	0.3260
Recall	0.6476
F1 Score	0.4337
F2 Score	0.5409
AUC	0.7175

The tuned ***LogisticRegression*** model reached a validation F2 score of 0.5409, very close to the cross-validation F2 score of 0.5406. The gap is only 0.0003, which means the model was not overfitting. But when we compare the validation results to the three tree models from Phases 3, 4, and 5, ***LogisticRegression*** is the weakest on the two metrics that matter most for our problem. Its recall of 0.6476 is the lowest of all four tuned models, so it catches fewer real defaulters than any of the tree models. Its AUC of 0.7175 is ahead of ***RandomForest*** but behind both gradient boosters. ***LogisticRegression*** is competitive on precision (0.3260, very close to ***LightGBM*** at 0.3263), but in our problem we care more about catching defaulters than avoiding false alarms.

The accuracy of 0.6639 looks like a strength because it is the highest among the four tuned models. But this is a softer version of the *accuracy paradox* we explained for ***RandomForest*** in Phase 9 of Section 5. ***LogisticRegression*** predicts "default" less often than the tree models, so it gets more of the easy 80 percent of fully paid loans right. The price for this is that it misses more real defaulters, which is what the low recall shows. The high accuracy hides this cost, so we cannot use it to judge the model on its own.

This result matches what we expected: linear models cannot capture the nonlinear patterns in credit data as well as tree-based models can, even when the linear model is properly tuned. So we included ***LogisticRegression*** in the validation set comparison as a benchmark but did not move it forward to the probability calibration stage. The three tree models and the stacking ensemble built from them are the four candidates we calibrate and compare in the rest of this section.

Comparing the Four Tuned Models

Before we move on to the stacking ensemble, it is useful to put the four tuned models side by side on the same validation set. Each one was trained with the same F2 scoring, the same data, and the same class weighting. So a direct comparison is fair and lets us see which family is performing best on our task. The results are shown in Table 5.12.

Table 5.12: Validation Set Performance of the Four Tuned Models

Model	Accuracy	Precision	Recall	F1 Score	F2 Score	AUC
LightGBM	0.6571	0.3263	0.6814	0.4413	0.5596	0.7288
XGBoost	0.6603	0.328	0.6765	0.4418	0.5579	0.7296
Random Forest	0.6428	0.3137	0.6715	0.4276	0.5468	0.7136
Logistic Regression	0.6639	0.326	0.6476	0.4337	0.5409	0.7175

LightGBM wins on recall (0.6814) and F2 (0.5596), the two metrics that matter most for our problem. ***XGBoost*** wins on F1 (0.4418) and AUC (0.7296), and is a very close second on recall and F2. ***RandomForest*** lags slightly behind both gradient boosters on every metric. ***LogisticRegression*** is the weakest on recall, F2, and AUC, which confirms that linear models cannot capture the nonlinear patterns in our data as well as tree-based models can.

This comparison tells us two things. First, the three tree-based models are clearly the strong candidates for production. Second, ***LightGBM*** and ***XGBoost*** are almost tied at the top, so we cannot pick a single winner from these four alone. To move forward we need a deeper comparison that goes beyond the default decision threshold of 0.5. We do that in the next phases, but first we build a stacking ensemble that combines the three tree-based models into one composite model.

Phase 7: Stacking Ensemble

Phase 7 introduced a fifth candidate model alongside the four tuned base models. A **stacking ensemble** is a method that combines the predictions from multiple base models using a second model called a *meta-learner*. The goal is to capture the strengths of each base model in a single composite model that can outperform any of them individually. We followed this approach because reference [7] in our literature review (Akinjole et al.) used a similar stacking method and reported a meaningful improvement over their individual models.

Our stacking ensemble combined the three tree-based models we already tuned: ***LightGBM***, ***XGBoost***, and ***RandomForest***. The meta-learner was a ***LogisticRegression*** model with ***class_weight='balanced'***. We chose ***LogisticRegression*** as the meta-learner because it is simple and interpretable, which kept the risk of overfitting low. A complex meta-learner can over-rely on patterns in the base predictions and end up worse than its inputs.

We did not include the tuned ***LogisticRegression*** from Phase 6 as a fourth base model in the stacking ensemble. Stacking benefits from diverse base learners that capture different patterns in the data. The three tree-based models we chose already cover two different families: ***LightGBM*** and ***XGBoost*** use sequential boosting, while ***RandomForest*** uses parallel bagging. Adding ***LogisticRegression*** as a fourth base would not add a meaningfully different opinion because the meta-learner that combines the base predictions is itself a ***LogisticRegression***. The linear base would end up partly duplicating the work of the linear meta-learner, which often weakens stacking instead of helping it.

The stacking process worked in two steps. First, each of the three base models was retrained five times using 5-fold cross-validation on the training set. For every fold, the model trained on four folds and produced probability predictions on the fifth fold. The result of this step was a set of *out-of-fold predictions*, which means predictions for every training row that came from a model that did not see that row during training. This is important because it stops the meta-learner from seeing predictions that come from a model that already memorized the answer. Second, the ***LogisticRegression*** meta-learner trained on these out-of-fold predictions and learned how to combine them into a final probability. Each base model was also retrained one final time on the full training set so that it would be ready for inference on new applicants.

We then evaluated the stacking ensemble on the validation set at the default threshold of 0.5. The results are shown in Table 5.13.

Table 5.13: Stacking Ensemble Performance on the Validation Set

Metric	***Value***
Accuracy	0.659
Precision	0.3274
Recall	0.6789
F1 Score	0.4417
F2 Score	0.5588
AUC	0.7296

The stacking ensemble reached a validation F2 score of 0.5588, which was very close to the F2 scores of the three tree-based models from Phases 3, 4, and 5. The AUC of 0.7296 was tied with ***XGBoost*** for the highest in this section so far. Stacking did not produce a dramatic improvement over the tree-based models, but it did not hurt either. With four candidate models now ready ***LightGBM***, ***XGBoost***, ***RandomForest***, and the *stacking ensemble*, we moved on to a deeper comparison in the next phases, where we would test every candidate at many different decision thresholds and compare them on calibrated probabilities.

Phase 8: Uncalibrated Comparison Across Thresholds

So far we have evaluated every tuned model at the default decision threshold of 0.5. This threshold means the model predicts "default" only if it is at least 50 percent sure. For a balanced dataset this default is reasonable, but for our problem the default is not optimal. As we explained in Section 5 and again in Phase 1 of this section, accepting a person who defaults costs us a lot more than rejecting a safe applicant. So we should be willing to lower the threshold and predict "default" even when the model is less than 50 percent sure, to catch more real defaulters.

To see how each model behaves across different thresholds, we ran a sweep on the validation set. For each of our four candidates ***LightGBM***, ***XGBoost***, ***RandomForest***, and the *stacking ensemble*, we tested 19 different thresholds between 0.20 and 0.50. We used larger steps in the lower range (0.05 between 0.20 and 0.30), medium steps in the middle (0.02 between 0.30 and 0.40), and small steps near the default (0.01 between 0.40 and 0.50). This gave us finer detail near the most likely sweet spot. We did not include the tuned ***LogisticRegression*** from Phase 6 in this sweep because it is a baseline benchmark only and does not advance to the calibration or hero selection stages.

Figure 5.10 shows the F2 score of each model across the 19 thresholds. The red circles mark the F2 peak for each model. The grey dashed line shows the default threshold of 0.50 for reference.

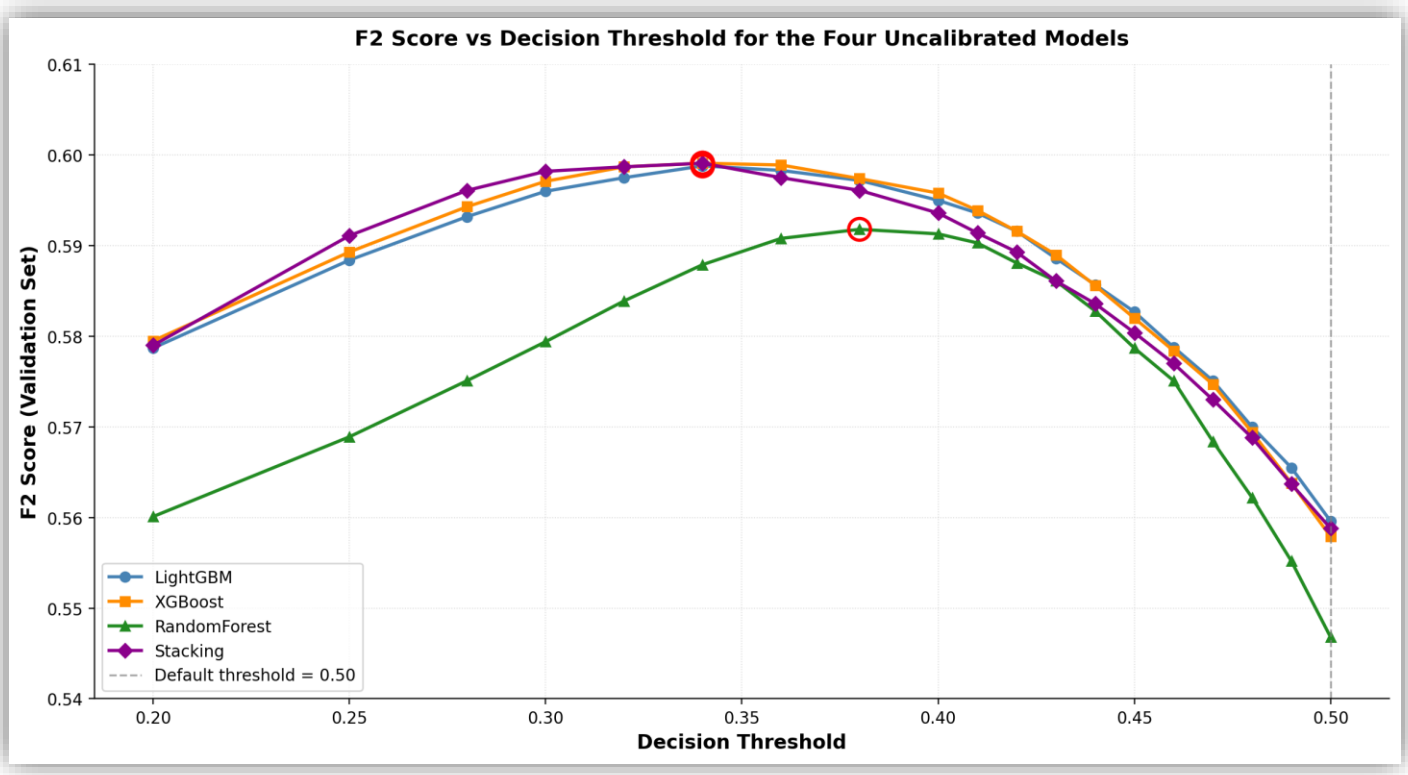

Figure 5.10: F2 Score vs Decision Threshold for the Four Uncalibrated Models

Two clear patterns appear in Figure 5.10. First, every model gets a higher F2 score at a lower threshold than at the default 0.50. The three tree-boosters and stacking all peak at threshold 0.34 with F2 scores around 0.599, while ***RandomForest*** peaks slightly later at threshold 0.38 with F2 of 0.5918. Second, ***LightGBM***, ***XGBoost***, and the stacking ensemble track each other very closely across the whole range, while ***RandomForest*** stays consistently a bit below the other three. Table 5.14 summarizes the difference between each model's F2 peak and its F2 at the default threshold of 0.50.

Table 5.14: F2 Peak vs Default Threshold for the Four Uncalibrated Models

MODEL	PEAK F2	THRESHOLD AT PEAK	F2 AT DEFAULT 0.50	GAIN FROM THRESHOLD TUNING
LIGHTGBM	0.5988	0.34	0.5596	+ 0.0392
XGBOOST	0.5991	0.34	0.5579	+ 0.0412
RANDOM FOREST	0.5918	0.38	0.5468	+ 0.0450
STACKING	0.5991	0.34	0.5588	+ 0.0403

The gain from threshold tuning is between 0.039 and 0.045 across all four models, which is a meaningful improvement on a metric like F2. This tells us that the default threshold of 0.50 is the wrong choice for our problem, and we will pick a custom threshold for the production model in Phase 11.

Phase 9: Probability Calibration

By the end of Phase 8 we had four candidate models that were good at **ranking** applicants from safest to riskiest. But the actual probability values they produced were not honest. ***LightGBM*** and ***RandomForest*** were trained with ***class_weight='balanced'*** and ***XGBoost*** was trained with ***scale_pos_weight=4.03***. These settings tell each model to predict "default" more often than the natural rate, which pushes the predicted probabilities up. The same effect carries over to the stacking ensemble because its inputs come from the same biased base models. So when a tuned model says "70 percent risk," it is really claiming a higher risk than what exists in the population.

This is fine when we only care about ranking, but it is a problem for the Streamlit web app. The loan officer will see the predicted risk as a percentage, and that percentage must mean what it says. To fix this, we used probability calibration on the four candidates.

We used **isotonic regression** as the calibration method. Isotonic regression is a non-parametric technique that learns a monotonic mapping from the raw model output to a corrected probability. It does not assume any particular shape for the miscalibration, which makes it flexible. The alternative is sigmoid calibration (also known as Platt scaling), which fits a sigmoid curve and is the right choice for smaller datasets. With 791,706 training rows in our case, isotonic regression had enough data to fit a stable mapping without underfitting, so we chose isotonic.

We applied calibration to all four candidates: ***LightGBM***, ***XGBoost***, ***RandomForest***, and the *stacking ensemble*. We did not calibrate the tuned ***LogisticRegression*** from Phase 6 because it does not advance to the hero selection stages. For each candidate, we used the ***CalibratedClassifierCV*** wrapper from Scikit-Learn with 5-fold cross-validation. The wrapper splits the training set into 5 folds, trains a fresh base model on four folds, fits the isotonic mapping on the predictions of the fifth fold, and then averages the five resulting calibrated models into one. This protects the calibration from overfitting because the isotonic mapping is always fit on data the base model has not seen during training.

To verify the calibration worked, we measured the mean predicted probability of each calibrated model on the validation set and compared it to the actual default rate. The actual default rate was 0.1987. The calibrated ***LightGBM*** produced a mean predicted probability of 0.1985, the calibrated ***XGBoost*** produced 0.1986, the calibrated ***RandomForest*** produced 0.1986, and the calibrated stacking ensemble produced 0.1986. All four calibrated models matched the actual default rate within 0.0002. This confirmed that the calibration worked correctly on every candidate. With honest probabilities now in hand, we re-ran the threshold sweep from Phase 8 on the calibrated probabilities in Phase 10 to find the real best operating point.

Phase 10: Calibrated Comparison Across Thresholds

After calibration, we re-ran the threshold sweep from Phase 8 on the calibrated probabilities. This is the comparison we actually use to pick the hero model in Phase 11, because the probability values are now honest. We tested 25 thresholds between 0.10 and 0.50 for each of the four calibrated candidates (***LightGBM***, ***XGBoost***, ***RandomForest***, and the calibrated stacking ensemble). We expanded the lower end of the range compared to Phase 8 because we expected calibration to move the optimal threshold downward.

Figure 5.11 shows the F2 score of each calibrated model across the 25 thresholds. The red circles mark the F2 peak for each model. The grey dashed line shows the default threshold of 0.50 for reference.

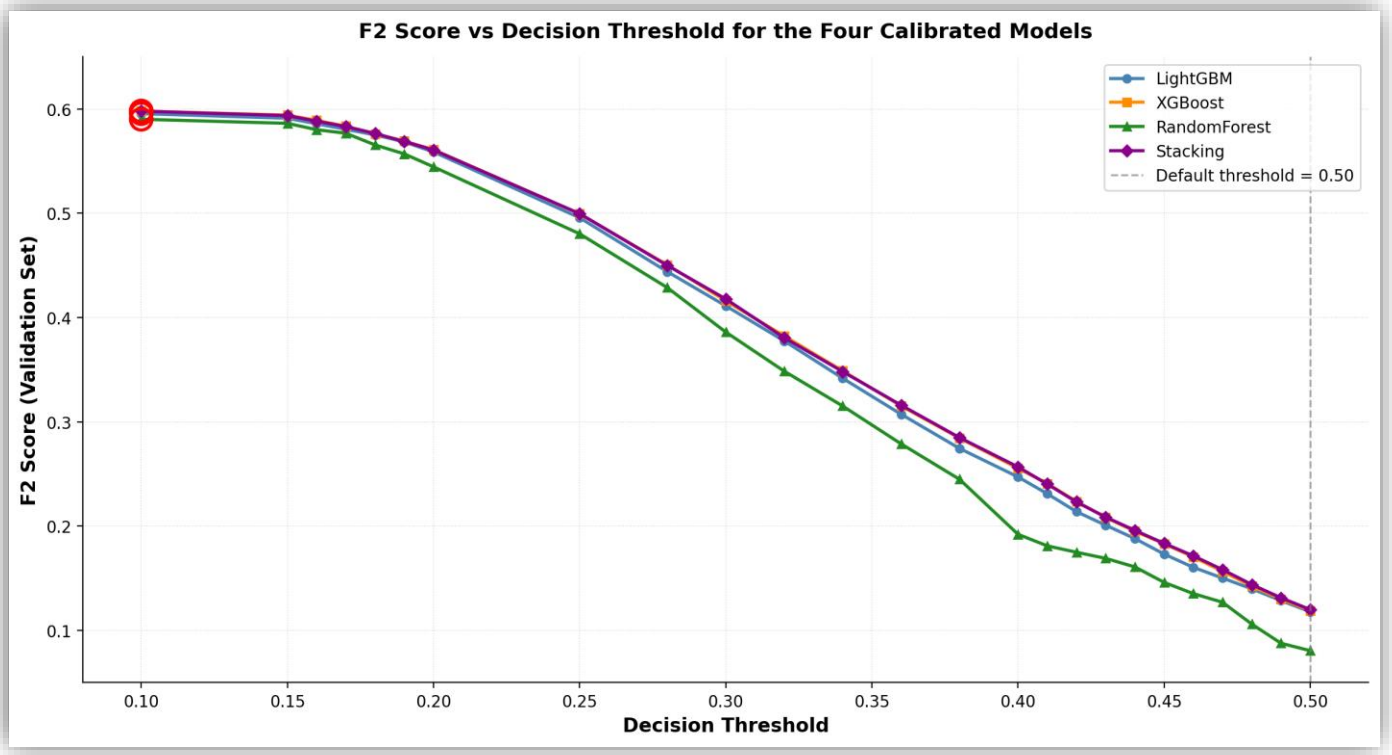

Figure 5.11: F2 Score vs Decision Threshold for the Four Calibrated Models

Figure 5.11 shows that calibration changed the shape of the F2 curve dramatically. In the uncalibrated sweep from Phase 8 the F2 curve was a wide hill that peaked around threshold 0.34. In the calibrated sweep the F2 curve is a steady decline that peaks at the lowest threshold tested. All four calibrated models peak at threshold 0.10, with F2 scores of 0.5955 for LightGBM, 0.5979 for XGBoost, 0.5901 for RandomForest, and 0.5979 for the stacking ensemble. Table 5.15 compares the uncalibrated and calibrated peaks side by side.

Table 5.15: Uncalibrated vs Calibrated F2 Peaks for the Four Models

Model	Uncalibrated Peak Threshold	Uncalibrated F2	Calibrated Peak Threshold	Calibrated F2
LightGBM	0.34	0.5988	0.1	0.5955
XGBoost	0.34	0.5991	0.1	0.5979
Random Forest	0.38	0.5918	0.1	0.5901
Stacking	0.34	0.5991	0.1	0.5979

The peak thresholds moved from around 0.34 to 0.10, but the peak F2 scores stayed nearly identical. This is the result we expected. Calibration is a monotonic transformation, so it does not change the ranking of applicants by risk. It only rescales the probability values so they match the real default rate in the population. The model's ability to separate defaulters from non-defaulters is unchanged. This is also visible in the AUC values, which barely moved between the uncalibrated and calibrated sweeps. ***XGBoost*** went from AUC 0.7296 to 0.7300, ***Stacking*** went from 0.7296 to 0.7297, ***RandomForest*** went from 0.7136 to 0.7137, and ***LightGBM*** went down from 0.7288 to 0.7263. The changes in AUC are all under 0.003, which confirms calibration preserved the ranking quality.

What did change is which threshold gives that ranking. Before calibration, a probability of 0.7 meant something inflated by the class weighting. After calibration, a probability of 0.7 actually means a 70 percent likelihood of default in the real population. The Streamlit web app shows this number to the loan officer, so it has to be honest. The price for this honesty is that the optimal threshold is now around 0.10 instead of 0.34. But this is just a relabeling of the same decision, so the underlying model behavior is identical.

The calibrated F2 peak sits at threshold 0.10, which is the lowest threshold we tested. At this threshold, the calibrated ***XGBoost*** catches 92 percent of the real defaulters with a precision of 0.2486. This is a very aggressive operating point that would flag almost every applicant as a default risk. The same applies to the other three models. In practice, an operating point this aggressive creates too many false alarms for the loan officer to handle, so we did not pick the threshold purely from the F2 peak. We made the final hero selection in Phase 11 based on a balance between recall and precision, not on the F2 peak alone. This is the only place in the section where we deviate from pure metric optimization, and we explain the reasoning in Phase 11.

Phase 11: Hero Model Selection

By the end of Phase 10 we had four calibrated candidates and a full map of their behavior across 25 thresholds. The job of Phase 11 was to pick the single best combination of model and threshold to deploy in the Streamlit web app. This combination is the "hero" model.

The F2 peak from Phase 10 sits at threshold 0.10 for every model, which would catch around 92 percent of the real defaulters. We did not pick this point because the precision drops to 0.25 at threshold 0.10, meaning only 1 in 4 applicants flagged as "default" actually defaults. In practice this would create too many false alarms for the loan officer to handle, who would quickly lose trust in the model's predictions. So we needed a threshold that catches a strong fraction of defaulters without flagging too many safe applicants by mistake.

We targeted a recall of at least 0.75, which means catching at least 75 percent of the real defaulters. This is a strong recall for a credit risk model and matches the priority we set in Section 5. We then looked for the highest precision among the thresholds that met this recall target. Threshold 0.17 was the sweet spot for every model. Table 5.16 compares the four calibrated candidates at threshold 0.17.

Table 5.16: Validation Set Performance of the Four Calibrated Models at Threshold 0.17

Model	*Accuracy*	*Precision*	*Recall*	*F1 Score*	*F2 Score*	*AUC*
LightGBM	0.6003	0.2999	0.7579	0.4297	0.5805	0.7263
XGBoost	0.6044	0.3027	0.7596	0.4329	0.5835	0.73
Random Forest	0.5761	0.288	0.7695	0.4191	0.5767	0.7137
Stacking	0.6047	0.3027	0.7589	0.4328	0.5831	0.7297

XGBoost wins on accuracy, precision, F1, F2, and AUC. It is the runner-up on recall, only 0.01 behind ***RandomForest***. The stacking ensemble is essentially tied with ***XGBoost*** on every metric but does not beat it on any of them. So the extra complexity of stacking did not earn us a real improvement, and we picked ***XGBoost*** as the hero model. This is consistent with our literature review, which noted that boosting algorithms perform very well on imbalanced credit data and that ensembles are not guaranteed to improve over a strong individual model.

With the model picked, we still had to confirm the threshold. We compared ***XGBoost*** at five thresholds spanning the full range we tested. The results are in Table 5.17.

Table 5.17: Calibrated XGBoost at Different Thresholds

Threshold	Accuracy	Precision	Recall	F1 Score	F2 Score
0.1	0.4309	0.2486	0.9217	0.3916	0.5979
0.15	0.5577	0.2853	0.8145	0.4226	0.5941
0.17	0.6044	0.3027	0.7596	0.4329	0.5835
0.2	0.6562	0.326	0.6843	0.4417	0.561
0.5	0.8067	0.58	0.0993	0.1695	0.119

The pattern is clear. Threshold 0.10 catches 92 percent of defaulters but the precision is only 0.25. Threshold 0.50 has high precision (0.58) but the recall collapses to under 10 percent, which means the model misses 9 out of every 10 defaulters. Threshold 0.17 sits at the right balance: recall stays above 0.75, precision is 0.30, and the F2 score of 0.5835 is only 0.014 below the peak F2 at threshold 0.10. So we paid a small F2 cost in exchange for much better precision and a more practical operating point.

Based on Tables 5.16 and 5.17, we locked in the calibrated ***XGBoost*** at threshold 0.17 as the hero model. We saved three artifacts to Google Drive: the calibrated model itself, the name of the model, and the threshold value. These three artifacts are what the Streamlit web app loads to make predictions on new applicants. Phase 12 prints the final production summary.

Phase 12: Hero Model Production Summary

Phase 12 was the closing phase of the section. We took the locked configuration from Phase 11 (calibrated ***XGBoost*** at threshold 0.17) and produced the final summary that documents what the Streamlit web app actually uses. The summary has three parts: the production configuration, the full performance profile on the validation set, and the confusion matrix.

The production configuration is short: algorithm ***XGBoost***, calibration method isotonic regression with 5-fold cross-validation, decision threshold 0.17. These three pieces are saved to Google Drive as three small artifact files (***hero_model.pkl***, ***hero_name.pkl***, and ***hero_threshold.pkl***). The Streamlit web app loads these three files on startup and uses them for every new prediction.
The full performance profile on the validation set is shown in Table 5.18.

Table 5.18: Hero Model Performance on the Validation Set

Metric	*Value*
Accuracy	60.44%
Precision	30.27%
Recall	75.96%
F1 Score	43.29%
F2 Score	58.35%
AUC	73.00%

The hero model catches about 76 percent of the real defaulters at a precision of 30 percent. The F2 score of 58.35% reflects this balance, since F2 weights recall twice as much as precision. The AUC of 73 percent shows the underlying ranking quality of the model, which is independent of the chosen threshold.

To understand what these numbers mean in absolute terms, we produced the confusion matrix in Figure 5.12. The left panel shows the raw counts of the four outcomes on the validation set. The right panel shows the same counts as a percentage of each actual class.

Before we read the confusion matrix, here are the four cells. A **true positive (TP)** is a real defaulter the model correctly flagged. A **false negative (FN)** is a real defaulter the model wrongly approved. A **true negative (TN)** is a safe applicant the model correctly approved. A **false positive (FP)** is a safe applicant the model wrongly flagged.

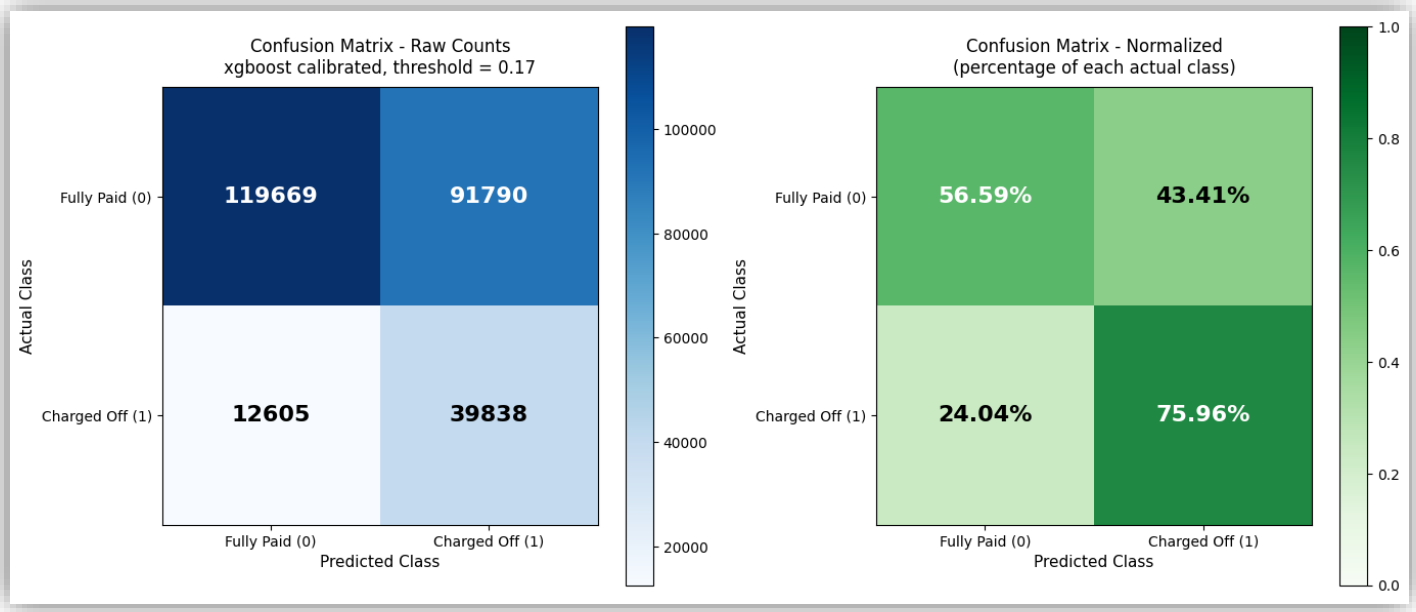

Figure 5.12: Confusion Matrix of the Hero Model on the Validation Set

The four cells of the confusion matrix tell the full story of the model. The hero model correctly caught 39,838 real defaulters and missed 12,605 of them. It correctly approved 119,669 fully paid loans and incorrectly rejected 91,790. The right panel shows the normalized view: of every 100 real defaulters, the model catches about 76 and misses about 24. Of every 100 fully paid borrowers, the model correctly approves about 57 and incorrectly rejects about 43.

The 91,790 false positives in the confusion matrix is a number that requires explanation. At first glance it looks like a problem, since the model is rejecting almost 44 percent of safe borrowers. But this is a known and accepted price for our recall-first strategy. In Section 5 and again in Phase 1 of this section, we explained that accepting a person who defaults is much worse than rejecting a safe applicant. So the model is designed to be cautious: when it sees risky signals, it flags the applicant. The model is a decision-support tool, not an automated reject system. The Streamlit web app shows the loan officer a calibrated probability and the SHAP explanation behind it, and the loan officer makes the final call. So a high false positive count translates into more applications that get a second human review, not more rejections.

With Phase 12 finished, Section 6 was complete. The hero model artifacts were saved to Google Drive and were ready to be loaded by the Streamlit web app. Section 7 evaluates the hero model one final time on the test set, which we have kept locked since Phase 3 of Section 5

7- Final Evaluation, SHAP, and Streamlit Preparation

Section 6 ended with our hero model picked: the calibrated ***XGBoost*** at threshold 0.17. All of the comparisons and selections so far were done on the validation set. The test set, which is 20 percent of the original data, has been locked since Phase 3 of Section 5 and we have not touched it for any decision. Section 7 is where we use it for the first and only time, then we prepare everything the Streamlit web app needs to serve predictions.

This section has seven phases. Phase 1 evaluates the hero model on the test set and compares the result to the validation numbers. This is the most important phase in the section because it tells us whether the model generalizes to data it has never seen. Phase 2 sets up the **SHAP explainer** that we will use to explain individual predictions. Phase 3 uses the explainer to walk through five representative applicants from the test set. Phases 4, 4.5, 5, and 6 are setup work that bridges this section into the Streamlit web app: a feature mapping for the user interface, a lookup pool of past applicants for the "similar borrowers" feature, a JSON schema and sample input files, and a final production artifact bundle.

Phase 1: Final Test Set Evaluation

The test set is 20 percent of the original data and contains 263,902 applicants. It was held out in the 60/20/20 stratified split from Phase 3 of Section 5. Since that split we have never used it. The training set was used for tuning and for calibration cross-validation. The validation set was used for the threshold sweep and the hero selection. The test set was completely off-limits. Phase 1 is the first and only time we use it. The numbers from this phase are the official performance numbers for the project, and they are what we report as the model's generalization performance.

The test set has the same class balance as the validation set: 52,443 defaulters out of 263,902 applicants, which is 19.87 percent. This match is expected because the original split was stratified. We loaded the locked hero model and the locked test set from Google Drive, generated calibrated probabilities for every test applicant, and applied the 0.17 threshold to convert probabilities into "default" or "fully paid" decisions. We then computed the six metrics (accuracy, precision, recall, F1, F2, AUC) on both the validation and test sets, side by side. The full comparison is in Table 5.19.

Table 5.19: Validation vs Test Set Performance of the Hero Model

METRIC	VALIDATION	TEST	GAP (TEST − VAL)
ACCURACY	0.6044	0.6045	+ 0.0001
PRECISION	0.3027	0.303	+ 0.0004
RECALL	0.7596	0.7614	+ 0.0018
F1 SCORE	0.4329	0.4335	+ 0.0006
F2 SCORE	0.5835	0.5846	+ 0.0011
AUC	0.73	0.7309	+ 0.0009

Every metric is within 0.002 of the validation value, and every gap is positive, meaning the test set numbers are slightly *better* than the validation numbers. The recall gap of +0.0018 is the most important one because recall is the metric we optimized for. A gap this small confirms that the model generalizes from validation to test almost perfectly. There is no sign of overfitting to the validation set decisions we made during threshold tuning. The model is doing the same thing on both unseen pieces of data, which is exactly what we wanted.

The test set confusion matrix tells the same story in absolute counts: 39,932 real defaulters were correctly caught, 12,511 were missed, 119,605 fully paid loans were correctly approved, and 91,854 were incorrectly rejected. In total the model flagged 131,786 applicants as default risks (about 50 percent of the test set) and approved 132,116 (about 50 percent). These counts are almost identical to the validation confusion matrix from Figure 5.12, which is consistent with the tight metric gaps in Table 5.19. As we explained in Phase 12 of Section 6, the high rejection rate is a deliberate consequence of the recall-first strategy. The model's role is to flag risky cases for a human review, not to auto-reject them.

With Phase 1 complete, the test set has done its job and we set it aside. The rest of Section 7 is about preparing the hero model for deployment. Phase 2 sets up the SHAP explainer so we can later explain individual predictions to a loan officer.

Phase 2: SHAP Setup

The hero model gives every applicant a calibrated probability of defaulting, but it does not tell the loan officer *why*. A loan officer who sees "this applicant has 80 percent risk" needs to know which factors drove that number. The literature gap we identified in Chapter 2 was exactly this kind of black-box problem, and our project promised to solve it with **SHAP** (SHapley Additive exPlanations). Phase 2 sets up the SHAP explainer that produces those reasons.

SHAP comes with several types of explainers, each one tuned for a different kind of model. We used ***KernelExplainer***, which is **model-agnostic**. This means ***KernelExplainer*** treats the model as a black box function from inputs to outputs. We gave it a small Python function that takes a row of features and returns the calibrated probability, and ***KernelExplainer*** explains that function directly. This is important for our case because our hero is a calibrated ***XGBoost***, which means the raw model's output is passed through a second isotonic regression step from Phase 9 of Section 6. ***KernelExplainer*** explains the full calibrated pipeline as one unit, so the SHAP values it produces sum to exactly the calibrated probability the loan officer sees on the screen, with no math gap.

KernelExplainer works by sampling. For each applicant we want to explain, it generates many perturbations of the applicant's data, gets the calibrated probability for each one, and then estimates the SHAP value of each feature from the differences between those probabilities. The number of perturbations is controlled by the ***nsamples*** parameter. We picked ***nsamples=2000*** as a balance between accuracy and computation time. As we will show in the sanity check below, 2000 perturbations is enough to make the math exact for our 40-feature model.

KernelExplainer also needs a **background sample**. The background is a small reference dataset that ***KernelExplainer*** uses to simulate "missing" features when it generates perturbations. We used 100 random rows from the training set as the background. We fixed the random seed to 42 for reproducibility. The mean predicted probability across these 100 background applicants is called the **expected value**. We computed it directly from the explainer and got 0.1973. This is the baseline probability that every SHAP explanation starts from. SHAP values then push this baseline up or down to reach the final probability for a specific applicant. The 0.1973 baseline is very close to the actual default rate of 19.87 percent in the data, which confirms the 100-row background is representative of the training distribution.

Before we used the explainer on real cases in Phase 3, we ran a sanity check. The fundamental property of SHAP is that the expected value plus the sum of the SHAP values for an applicant must equal the model's predicted probability for that applicant. We picked one applicant from the validation set, computed the SHAP values with ***nsamples=2000***, and verified the equation. The model's calibrated probability for the applicant was 0.189933. The expected value was 0.197282. The sum of the 40 SHAP values was −0.007349. Adding the expected value and the SHAP sum gave 0.189933, an exact match to the model probability to six decimal places. The reconstruction difference was 0.000000.

This sanity check confirmed two things. First, our explainer was set up correctly. Second, ***nsamples=2000*** is enough for ***KernelExplainer*** to produce mathematically exact reconstructions on our 40-feature model. With the explainer ready and verified, we saved the pieces the Streamlit web app will need to rebuild the same explainer at startup: the 100-row background sample, the expected value of 0.1973, the list of 40 feature names, and the ***nsamples*** value. These are bundled into ***shap_setup.pkl*** and used in Phase 3 to explain five real test set applicants.

Phase 3: SHAP Local Explanations on Five Cases

Phase 2 confirmed our SHAP setup worked correctly. Phase 3 uses the explainer to walk through five individual applicants from the test set and show how the model reasoned about each one. We picked these five cases from the test set because the test set is the only data the model has never seen, which makes every explanation completely honest. We did not cherry-pick favorable cases. Each case was picked because it represented a specific outcome category, and we let the data choose the actual applicant within that category.

The five cases were chosen to cover every corner of the confusion matrix from Phase 1, plus one tricky borderline case. Case 1 is a **true positive** with very high probability. Case 2 is a **true negative** with very low probability. Case 3 is a **borderline** applicant whose probability sat right at the 0.17 threshold. Case 4 is a **false negative**, which is the most costly mistake category for our problem because it is a defaulter the model approved. Case 5 is a **false positive**, which is the opportunity-cost mistake category because it is a safe borrower the model rejected. Together these five cases honestly show where the model is right and where it is wrong. The summary of all five is in Table 5.20.

Table 5.20: Summary of the Five Test Set Cases Used in Phase 3

CASE	TYPE	PROBABILITY	DECISION	ACTUAL OUTCOME	STATUS
1	True Positive	0.9165	REJECT	Defaulted	Correct
2	True Negative	< 0.0001	APPROVE	Paid	Correct
3	Borderline	0.17	REJECT	Paid	Wrong
4	False Negative	0.0035	APPROVE	Defaulted	Wrong
5	False Positive	0.8634	REJECT	Paid	Wrong

For each case we generated a SHAP waterfall plot. A waterfall plot reads from bottom to top. The bottom shows the **expected value** E[f(x)] = 0.197, the baseline probability across the 100 background applicants from Phase 2. The waterfall then stacks the SHAP contribution of each feature, where red bars push the probability up and blue bars push it down. The top of the waterfall is f(x), the model's final calibrated probability for that applicant. The feature values shown on the left of each row are the standardized values that the model actually sees, so a positive number means the feature is above its training average and a negative number means it is below.

Case 1: High Confidence Default (True Positive). This applicant had a calibrated probability of 0.9165, which is far above the 0.17 threshold. The model rejected and the applicant did default. Figure 5.13 shows the waterfall. The four largest red bars are ***term***, ***int_rate***, ***credit_history_months***, and ***dti***. The applicant chose a long-term loan, which historically correlates with higher default rates. The interest rate was well above average, which means the lender's own internal credit scoring already saw this applicant as risky. The debt-to-income ratio was high and the credit history was short. Together these four signals pushed the probability from 0.197 all the way up to 0.917. This is exactly the kind of pattern the model should catch, and the waterfall confirms it caught the right signals.

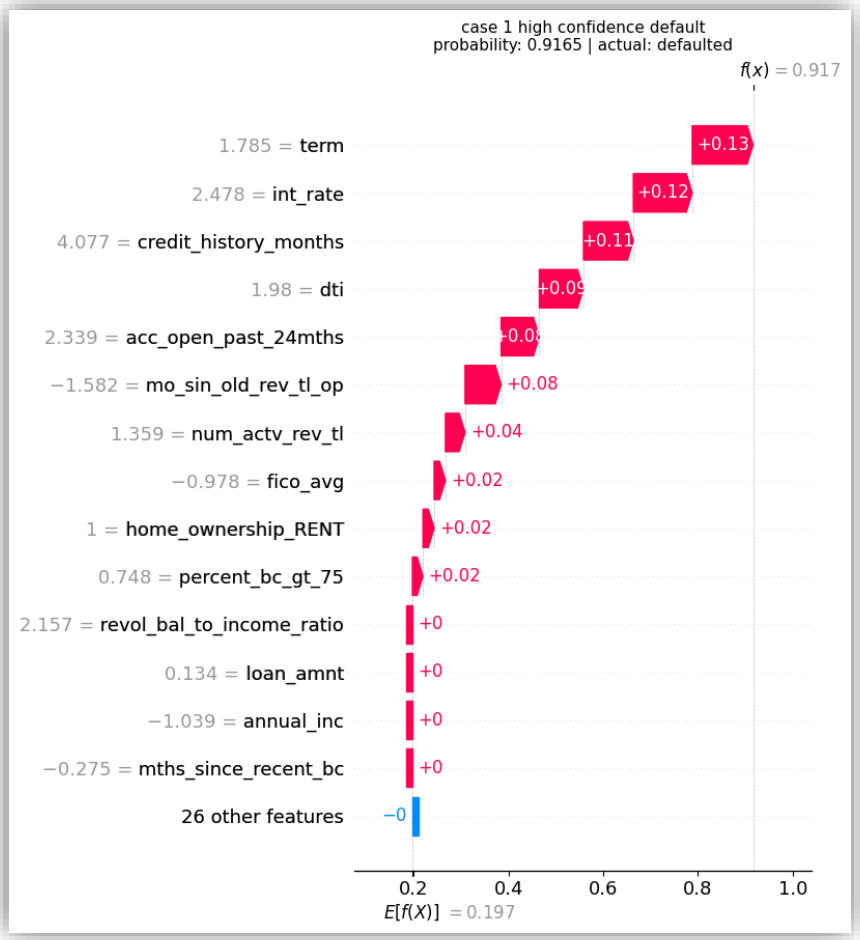

Figure 5.13: SHAP Waterfall - Case 1, High Confidence Default

Case 2: High Confidence Approval (True Negative). This applicant had a calibrated probability below 0.0001, which is the lowest range the model produces. The model approved and the applicant did pay. Figure 5.14 shows the waterfall. The four largest blue bars are ***int_rate***, ***fico_avg***, ***installment_to_income_ratio***, and ***loan_amnt_to_total_credit***. The interest rate was very low, which means the lender already classified this applicant as low risk. The FICO score was well above average. The monthly installment was small relative to the applicant's income, and the loan amount was small relative to the applicant's total available credit. Every feature in the top 10 of the waterfall pushed the probability *down*. The model is reading the same signals a careful loan officer would read, just much faster.

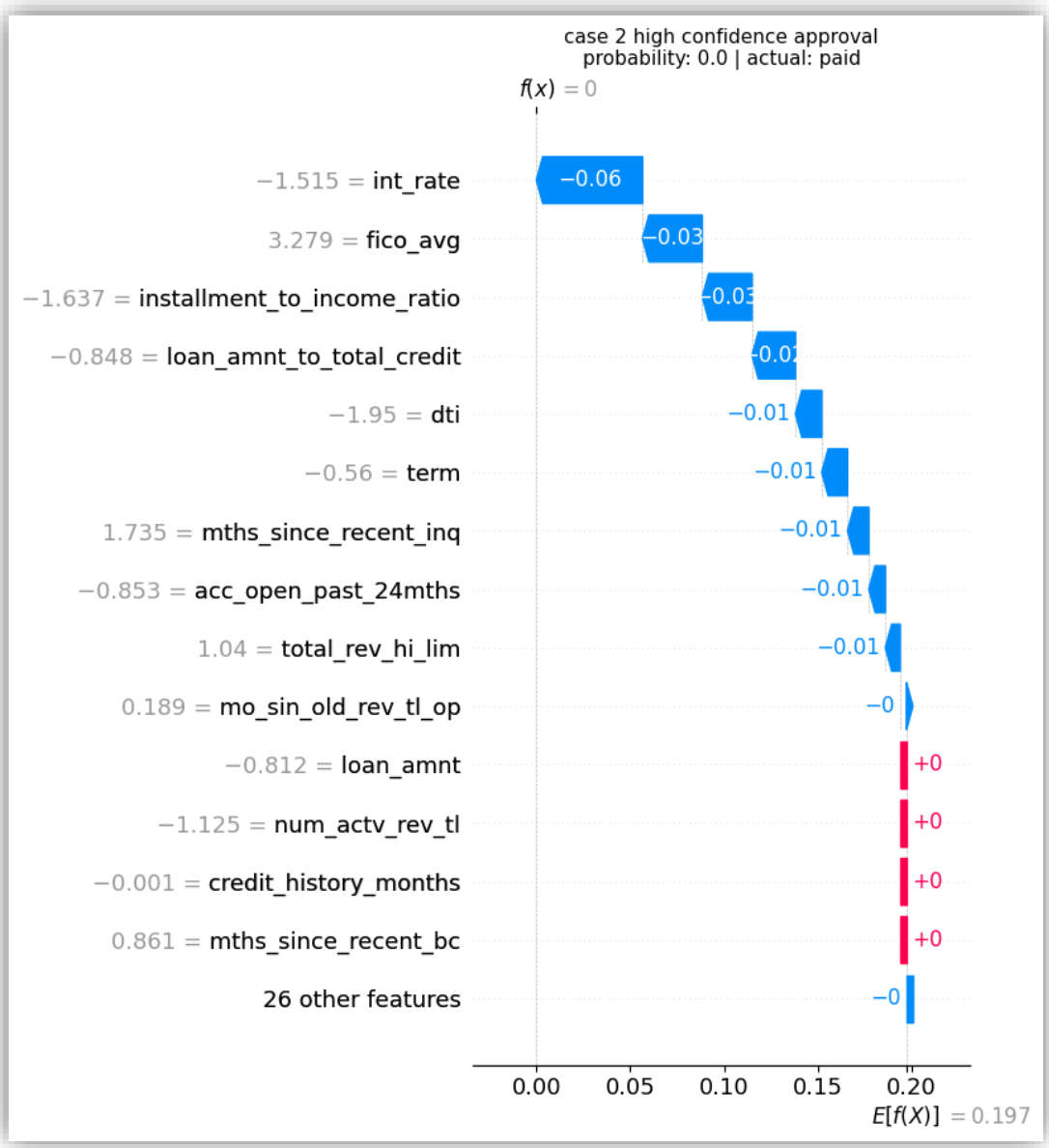

Figure 5.14: SHAP Waterfall - Case 2, High Confidence Approval

Case 3: Borderline. This applicant had a probability of 0.1700, which is the threshold itself. The model rejected and the applicant actually paid. Figure 5.15 shows the waterfall. The single biggest contributor was ***term*** with a red bar of +0.07, by far the largest push in the case. Every other feature pushed the probability down slightly, but together they could not overcome the term signal. This case shows why borderline applicants are genuinely hard to call: most of the signals say "safe", but one strong signal says "risky", and the model has to take a side. Cases like this are exactly where the loan officer's judgement matters, and exactly why the SHAP explanation is part of the Streamlit app. The officer can see that the model is making the call almost entirely on the loan term and can decide whether that single signal is enough to override the rest.

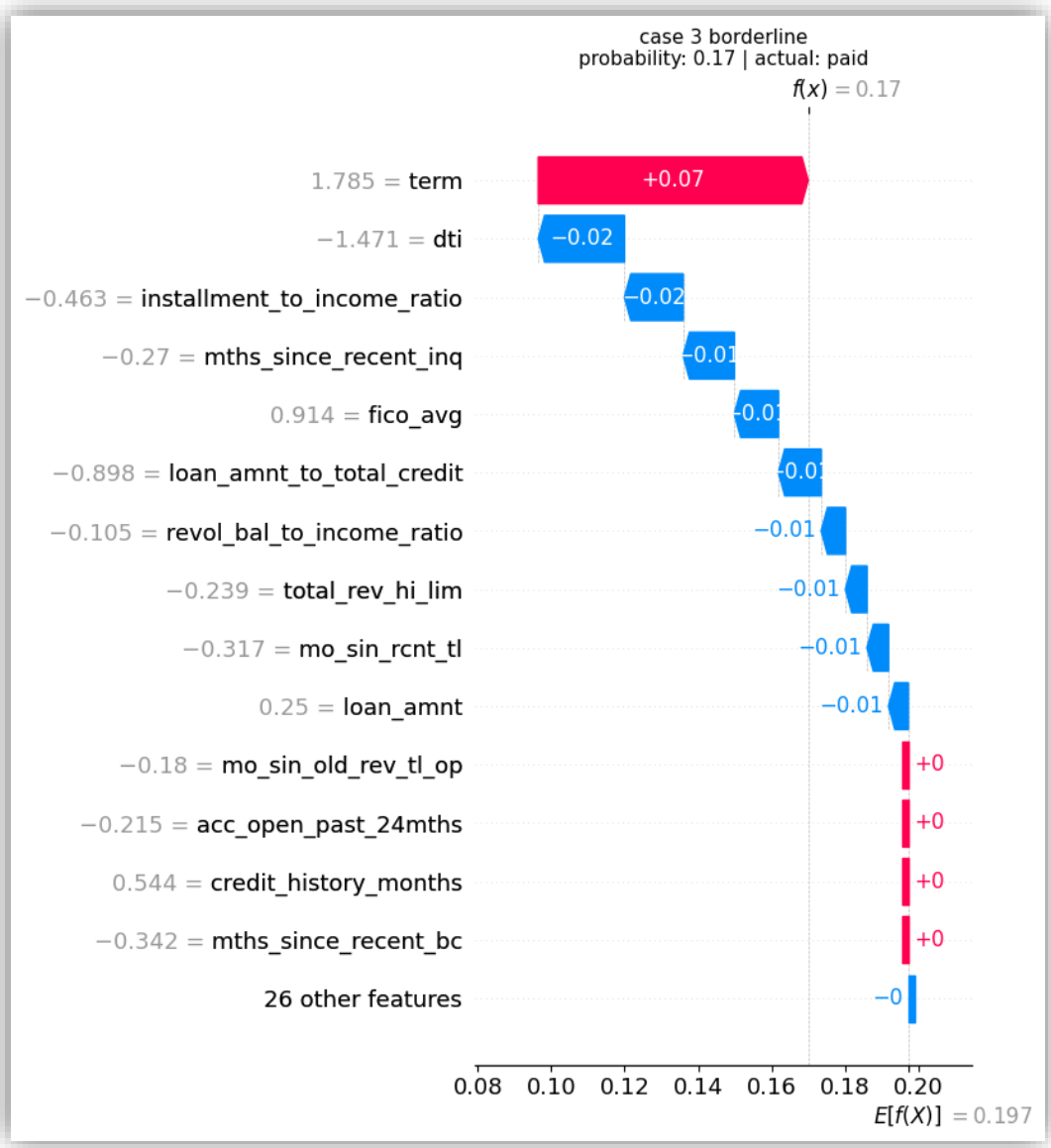

Figure 5.15: SHAP Waterfall - Case 3, Borderline

Case 4: False Negative. This applicant had a probability of 0.0035, well below the threshold. The model approved but the applicant actually defaulted. This is the costliest type of mistake for our problem. Figure 5.16 shows the waterfall. Every feature in the top 10 is blue, pushing the probability down. ***int_rate*** is the largest blue bar, followed by ***fico_avg***, ***installment_to_income_ratio***, and ***dti***. From the model's point of view this looked like a perfectly safe applicant: low interest rate, high FICO, low installment ratio, low DTI. But the applicant defaulted anyway. The reason is that the 40 features we used cannot capture every possible cause of default. Real-world factors like sudden job loss, medical emergencies, or family events are not in the data. The waterfall is being honest here: it tells the loan officer that based on the available features, there was no warning sign. This is also a reminder of why the model is a decision-support tool, not an oracle.

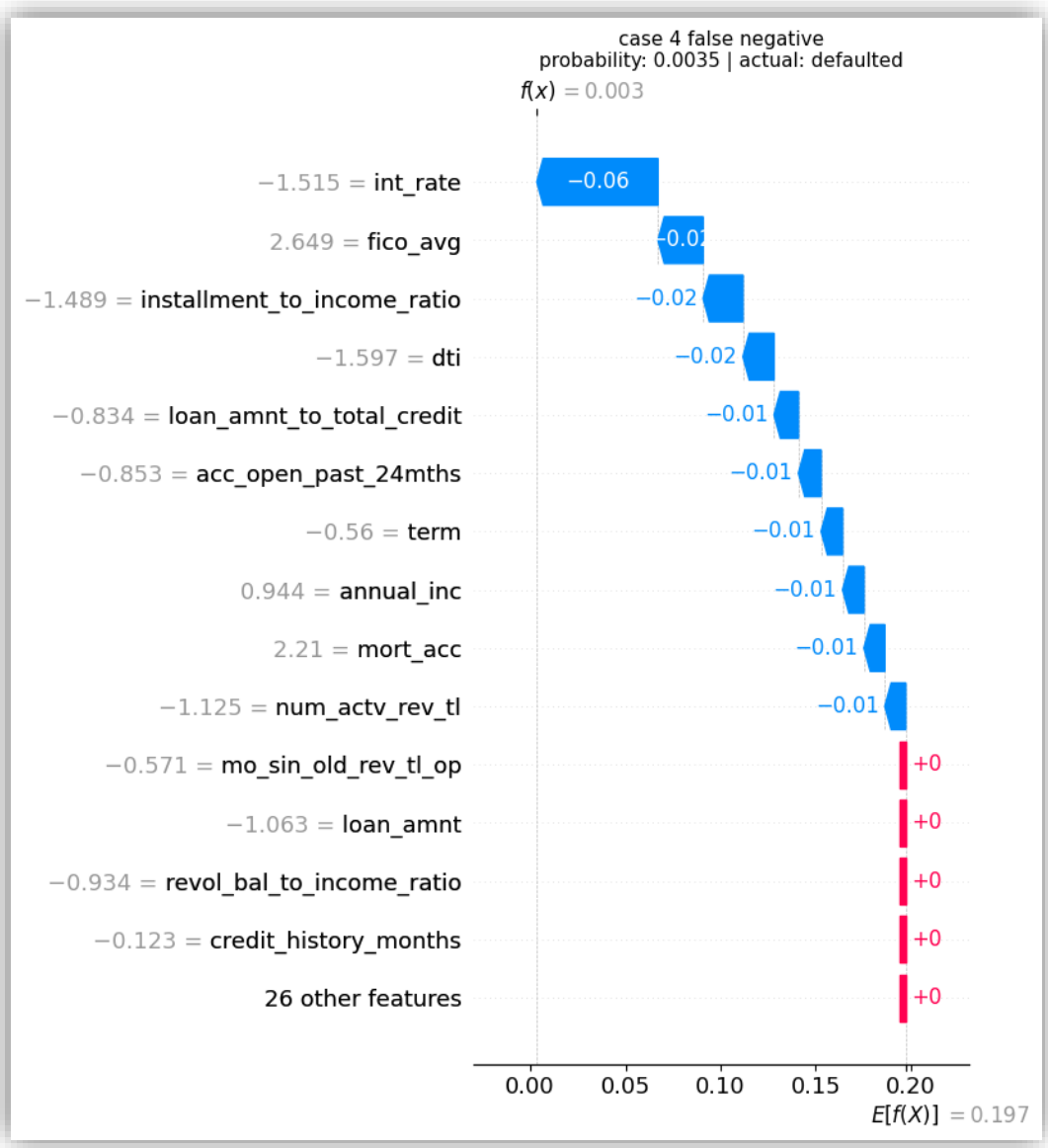

Figure 5.16: SHAP Waterfall - Case 4, False Negative

Case 5: False Positive. This applicant had a probability of 0.8634. The model rejected but the applicant actually paid. This is the opportunity-cost mistake. Figure 5.17 shows the waterfall. The top three red bars are ***term***, ***int_rate***, and ***mo_sin_old_rev_tl_op***. The applicant chose a long-term loan with a high interest rate (just like Case 1), and the months-since-old-revolving-account signal was unusually low. The model also added a red push for ***emp_length_was_missing***, which means this applicant did not provide an employment length on the application. From the model's point of view this combination looked highly risky. But the applicant actually paid the loan back. This shows that the model is conservative by design. Following the recall-first strategy we explained in Section 5 and again in Section 6, the model would rather reject some safe applicants than miss real defaulters. The SHAP waterfall makes the conservative reasoning visible, which lets the loan officer challenge the decision based on factors the model cannot see.

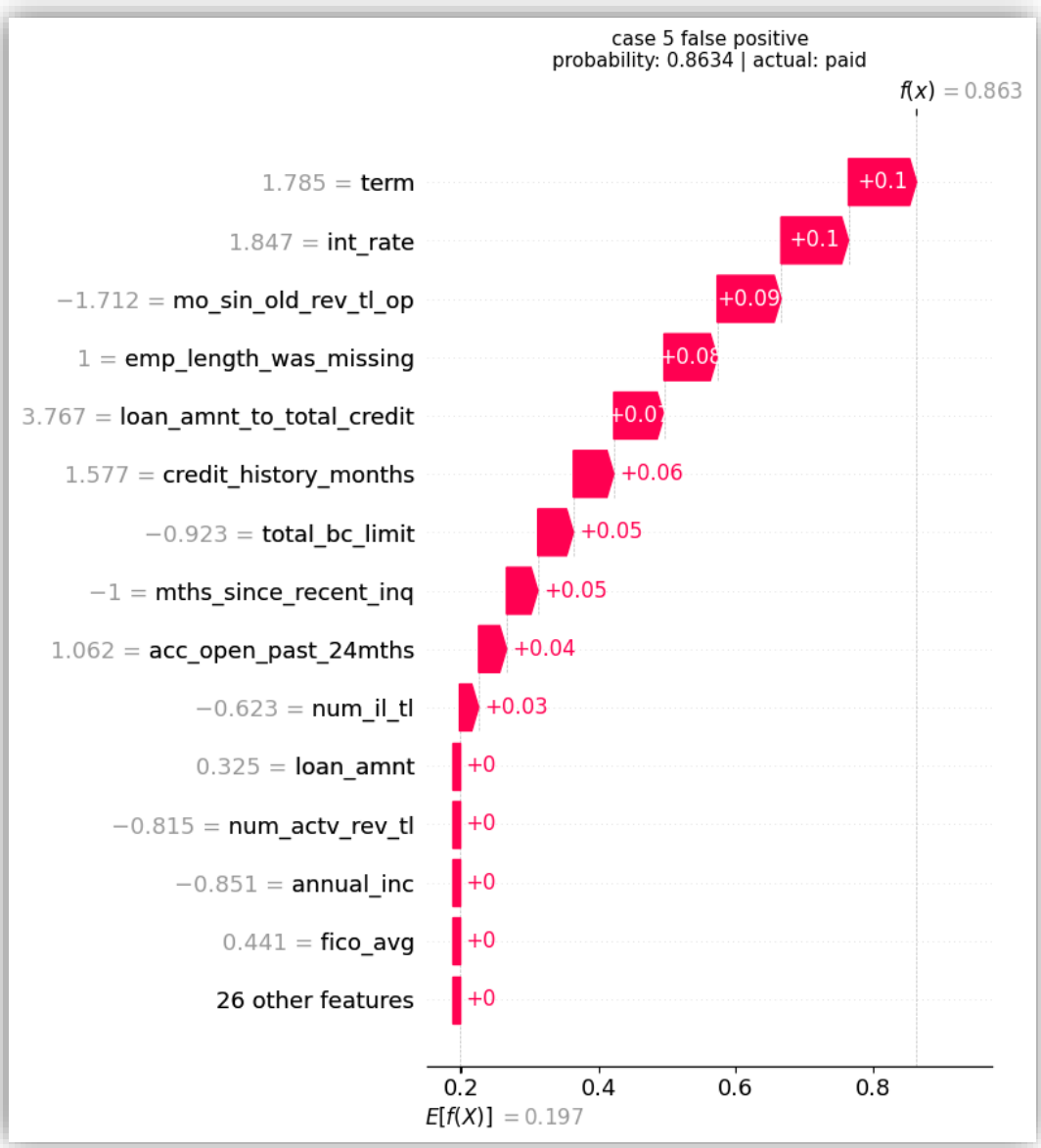

Figure 5.17: SHAP Waterfall - Case 5, False Positive

Across the five cases, two patterns are worth noting. First, the same few features dominate most explanations: ***int_rate***, ***term***, ***fico_avg***, ***dti***, ***credit_history_months***, and the engineered ratios from Phase 2 of Section 5. This matches the cross-algorithm feature importance ranking from Phase 10 of Section 5, where ***int_rate*** was the top feature in three of the four algorithms. Second, the model produces honest explanations even for its mistakes. In Case 4 (the false negative), the waterfall does not invent risk signals that were not there. It correctly says "every feature pointed to safe". This is the kind of transparency that solves the black-box problem we identified as the research gap in Chapter 2. With the explainer working and the five cases documented, the rest of Section 7 prepares the model for deployment in the Streamlit web app.

Phase 4: Streamlit Feature Mapping and Demo Cases

Phase 4 prepared two things the Streamlit web app needs: a clean mapping of the 40 model features to user-facing inputs, and a curated set of demo cases that the app uses for its live demonstration tab. The first task was the feature mapping. The model expects 40 specific features in a specific order. But the loan officer using the app cannot be expected to type 40 separate values, and some features (like the engineered ratios from Phase 2 of Section 5) are computed from other inputs anyway. So we split the 40 features into three sources based on where each one comes from. The first source is **12 direct form fields** that the loan officer types or selects on the screen, such as the loan amount, interest rate, term, annual income, and employment length. The second source is **17 JSON fields** that come from the bank's credit report system, such as the FICO score, revolving balance, total credit history, and recent inquiries. These are values a loan officer would not type by hand because they live in upstream systems. The third source is **11 app-computed features**, which the Streamlit app calculates internally before sending the row to the model. These include the seven engineered ratios from Phase 2 of Section 5 (***installment_to_income_ratio***, ***debt_to_credit_ratio***, ***dti***, ***revol_util***, ***bc_util***, ***loan_amnt_to_total_credit***, and ***revol_bal_to_income_ratio***), three one-hot expansions of the two categorical form dropdowns (***home_ownership*** and ***purpose***), and one missing-value flag (***emp_length_was_missing***). The math adds up: 12 + 17 + 11 = 40 features, which matches the locked feature list from Phase 11 of Section 5. We saved this mapping to ***streamlit_features.pkl*** so the Streamlit app loads it on startup.

The second task was the demo cases. The Streamlit app has a tab that shows pre-loaded example applicants so the loan officer (or the defense committee) can see the model in action without typing data. The five Phase 3 cases were designed for the academic narrative of this section, so for the live demo we picked a different set of five cases tailored to tell a different story. The five demo cases are: a **Strong Borrower** with very low risk who actually paid, a **Moderate Risk** applicant below the threshold who actually paid, a **High Risk** applicant who actually defaulted (so the model called it right), a **Borderline** applicant near the 0.17 threshold, and a **False Positive** case where the model rejected but the applicant paid. The High Risk case was deliberately filtered to be an applicant who really defaulted, because otherwise the demo would show the model being wrong twice in a row, which would hurt the live presentation. We computed SHAP values for all five demo cases ahead of time and bundled them into ***streamlit_demo_cases.pkl***, so the demo tab can show the explanations instantly without recomputing them on every click.

With the feature mapping and the demo cases saved, the Streamlit app now has everything it needs to convert a loan officer's form inputs into a calibrated prediction and a SHAP explanation. The remaining phases of Section 7 are smaller setup tasks: Phase 4.5 builds a pool of past applicants for the "similar borrowers" feature, Phase 5 builds a JSON schema and sample input files, and Phase 6 bundles every artifact from Sections 5, 6, and 7 into a single production package.

Phase 4.5: Lookup Pool for the Similar Borrowers Feature

The Streamlit app has a "similar borrowers" feature that complements the SHAP explanation. When a loan officer reviews a new applicant, the app finds the 10 most similar past applicants from a pre-built pool and shows their actual outcomes. This gives the officer a real-world reference point for the model's prediction. For example, if the model predicts a moderate risk of 0.35 for a new applicant, the officer can see that of the 10 most similar past applicants, 4 actually defaulted and 6 actually paid, which is consistent with the model's probability. This pattern matching grounds the calibrated probability in observable data, which helps the officer trust or challenge the prediction.

Phase 4.5 built this pool. We sampled 100,000 applicants at random from the validation set with a fixed random seed of 42 for reproducibility. We chose the validation set, not the training set, because the model never used validation data for fitting or calibration. The 791,706 training rows were used for hyperparameter tuning and for the isotonic calibration cross-validation in Phase 9 of Section 6, so any "similar borrower" from the training pool would have already been seen by the model in some way. The validation set is the largest data we have that is truly unseen during fitting. The test set would be even cleaner, but we kept the test set reserved for the final evaluation in Phase 1 of this section and did not touch it for any operational purpose.

The 100,000 chosen rows include 20,062 defaulters (20.06 percent) and 79,938 fully paid loans (79.94 percent), which matches the population default rate of 19.87 percent within sampling error. For each row we stored the 40 model features, the actual outcome (paid or defaulted), and the calibrated probability the hero model assigned to that applicant. The Streamlit app loads this pool at startup and computes feature-distance to find the 10 nearest neighbours for each new applicant. The full pool is saved as ***lookup_pool.pkl***.

Phase 5: JSON Schema and Sample Inputs

The Streamlit web app supports two ways to feed applicant data into the model. The first is the manual form from Phase 4, where the loan officer types the 12 form fields directly. The second is a JSON file upload, where the loan officer drops a file containing the 17 JSON fields that come from the bank's credit report system. Phase 5 built both the schema that defines the JSON format and a small set of sample input files that the app offers as downloadable examples.

The schema was saved as ***input_schema.json***. It documents the expected types, ranges, and options for each form field and lists every JSON field by name. The Streamlit app loads this schema on startup and uses it both to validate uploaded files and to render the form. By centralizing the schema in one file, we made sure the form and the upload feature stay in sync if a future maintainer needs to add or remove a field.

The sample input files were built from real test set rows so that the probabilities the user sees when they try the samples are exactly the same probabilities the model produced during evaluation. We picked five sample rows by their calibrated probability, each one chosen to represent a different decision band: a clear approval, a threshold case, a low-risk rejection, a mid-risk rejection, and a high-risk rejection. For each picked row, we inverse-transformed the standardized feature values back to raw human-readable values using the saved scaler from Phase 7 of Section 5, then rounded each value appropriately (integers for counts, two decimals for floats). The five files were saved with descriptive names so users can pick the right one for a quick test. Table 5.21 lists the five samples.

Table 5.21: The Five Sample JSON Input Files

File Name	Target Probability	Actual Outcome	Test Row Index
sample_approve_low.json	0.07	Paid	65,194
sample_threshold.json	0.17	Paid	72,769
sample_reject_low.json	0.22	Defaulted	6,500
sample_reject_mid.json	0.35	Defaulted	216,941
sample_reject_high.json	0.65	Paid	185,593

The five samples cover the practical decision range of the model. The first sample is a clear approve at probability 0.07. The second sits at exactly the 0.17 threshold, which the model rejects by a hair. The remaining three samples are firm rejects at increasing risk levels of 0.22, 0.35, and 0.65. The last sample is interesting because it has a high risk score of 0.65 but the applicant actually paid the loan back, which is the same false positive pattern we explained for Case 5 in Phase 3. This was an intentional choice. Including a real false positive in the sample set tells the user that the model is not always right, which sets honest expectations for the live demo. With the schema and the samples saved, the Streamlit upload feature has everything it needs to validate and process incoming files.

Phase 6: Production Artifact Bundle

Phase 6 was the closing phase of Section 7. Its job was to collect every artifact built across Sections 5, 6, and 7 and package them into one master file that the Streamlit web app loads at startup. Without this bundling step, the Streamlit app would need to load a dozen separate pickles in a specific order, which would be slow at startup and fragile against missing files. With the bundle, the app loads one file and has access to everything.

The bundle is a Python dictionary organized into seven sections that match the way the Streamlit app uses them. The **model** section contains the calibrated ***XGBoost*** hero model, the model name, and the decision threshold of 0.17, all from Phase 11 of Section 6. The **preprocessing** section contains the fitted scaler from Phase 7 of Section 5, the outlier clip caps from Phase 5, the multicollinearity drop list from Phase 8, and the ***continuous_cols*** list that tells the app which columns the scaler was fit on. These transformers let the Streamlit app process a new applicant's data in the exact same way the model was trained. These transformers let the Streamlit app process a new applicant's data in the exact same way the model was trained. The **features** section contains the locked 40-feature list from Phase 11 of Section 5 and the form/JSON/computed mapping from Phase 4 of this section. The **shap** section contains the explainer setup from Phase 2 of this section: the 100-row background sample, the expected value of 0.1973, the feature names, and the ***nsamples*** value. The **demos** section contains the five demo cases from Phase 4 with their precomputed SHAP values. The **lookup_pool** section contains the 100,000 past applicants from Phase 4.5 for the similar borrowers feature. Finally, the **metrics** section contains both the validation set metrics from Phase 11 of Section 6 and the test set metrics from Phase 1 of this section, which the Streamlit app uses to power its performance dashboard tab.

The bundle also carries a small **metadata** entry with the model version, the build date, the total feature count (40), and the row counts of the training, validation, and test sets. This metadata is not used by any feature, but it lets the loan officer (or a future maintainer) verify quickly which model and which dataset slice the app is running on. The whole package was saved as ***production_artifacts.pkl***. With this file written, Section 7 is complete and 5.2.5 Graphical User Interface Description picks up by building the Streamlit web app around this bundle.

5.2.4 Encountered Problems and Solutions

During the implementation, we faced some unexpected problems and we had to find practical solutions for each of them. The problems and how we solved them are described below.

Problem 1: Google Colab RAM Crash

When we first tried to load the full Lending Club dataset on the free version of Google Colab, the runtime crashed because the data was too big to fit in the available RAM of about 12 GB. The dataset has over 2.2 million rows and 151 columns, which is too much for a free Colab session to handle.

Solution: We first tried a quick fix by using the ***nrows*** parameter in ***pd.read_csv()*** to load only the first 500,000 rows. This worked but it meant we were losing a lot of data. So later we upgraded to Google Colab Pro with the High-RAM runtime, which gives around 50 GB of RAM. This let us load the full dataset without any memory problems.

Problem 2: Class Imbalance in the Target Variable

After we filtered ***loan_status*** to keep only Fully Paid and Charged Off rows, we found that the two classes were not balanced. About 80 percent of the loans were Fully Paid and only 20 percent were Charged Off. This is a problem because a model can reach 80 percent accuracy just by predicting Fully Paid for everyone, without learning to find the real defaulters. In a real bank, missing a defaulter costs much more than rejecting a safe applicant, so the model must learn the minority class well.

Solution: We first thought about using ***SMOTE*** because some related papers used it on similar loan datasets. But after reading more about SMOTE, we found two problems for our pipeline. First, normal SMOTE creates new samples by mixing two rows together. This produces values like 0.58 in our one-hot encoded columns, which has no meaning because a borrower cannot be 58 percent renter and 42 percent owner. Second, the ***SMOTE-NC*** version that handles categorical columns picks the most common value from neighbors, which can make the synthetic defaulters look the same as non-defaulters. So we decided not to use SMOTE. Instead, we used the class weight option that is built into each algorithm. For ***XGBoost***, we set ***scale_pos_weight*** to about 4 (the imbalance ratio). For ***LightGBM***, ***Random Forest***, and ***Logistic Regression***, we set ***class_weight='balanced'***. This makes each model give about 4 times more weight to the minority class during training, without creating any fake data. This way we solved the imbalance problem and kept the test set evaluation realistic.

Problem 3: Colab Disconnects During Long Hyperparameter Tuning Runs

Hyperparameter tuning with ***Optuna*** took a long time, especially for ***Random Forest*** and ***XGBoost*** on our large training set. A single tuning run could take more than one hour. Google Colab has a habit of disconnecting after a period of inactivity or when the runtime times out. Every disconnect would erase all the tuning progress and force us to restart from zero, which wasted hours of compute time.

Solution: We added a checkpointing step that saves the Optuna study and the best model artifacts to Google Drive after every tuning phase. This way, if Colab disconnected in the middle of a run, we could reconnect, load the saved study from Drive, and continue from where we stopped without losing any progress. We also saved the cross-validation results to Drive so we did not need to retrain the model just to see the metrics again.

Problem 4: The Default 0.50 Decision Threshold Was Wrong

All four of our tuned models produced calibrated probabilities, but using the default 0.50 threshold to convert those probabilities into APPROVE/REJECT decisions gave us very low recall. For example, the calibrated XGBoost at threshold 0.50 caught only 10 percent of the real defaulters in the validation set. The default threshold was wrong for our problem because the class imbalance and the recall-first strategy together push the optimal decision boundary much lower than 0.50.

Solution: We swept 25 different thresholds between 0.10 and 0.50 on the validation set and measured F2 (a recall-weighted metric) at each one. The sweep showed that the F2 peak sits at threshold 0.10, but choosing the peak alone would catch 92 percent of defaulters at a precision of only 0.25. So we did not pick the F2 peak directly. Instead we set a recall floor of 0.75 and picked the highest precision among all thresholds that met that floor. Threshold 0.17 was the sweet spot for every calibrated model. We saved 0.17 as the production threshold alongside the hero model.

5.2.5 Graphical User Interface Description

The Machine Learning Loan Default Prediction system is delivered as a web application built with Streamlit and deployed on Streamlit Cloud. The web interface is the front-end that loan officers use to interact with the hero model from Section 6. It loads the production artifact bundle from Phase 6 of Section 7 at startup and uses it for every prediction. The interface is organized into four main views: Home, About, New Application, and Result. Navigation between pages occurs through dedicated buttons placed within each view, providing a guided flow. Each view has one clear purpose, which keeps the experience simple for a loan officer who is not expected to have any technical background.

The **Home** view is the entry point shown in Figure 5.18. It displays the MLLDP logo, the system title, and two large cards that send the user to the two main workflows. The left card, **Start Assessment**, takes the user to the New Application page to evaluate a new applicant. The right card, **About the Project**, takes the user to the About page to learn about the methodology and the team. The Home page deliberately keeps the choice short and visual so that a first-time user is not overwhelmed by options.

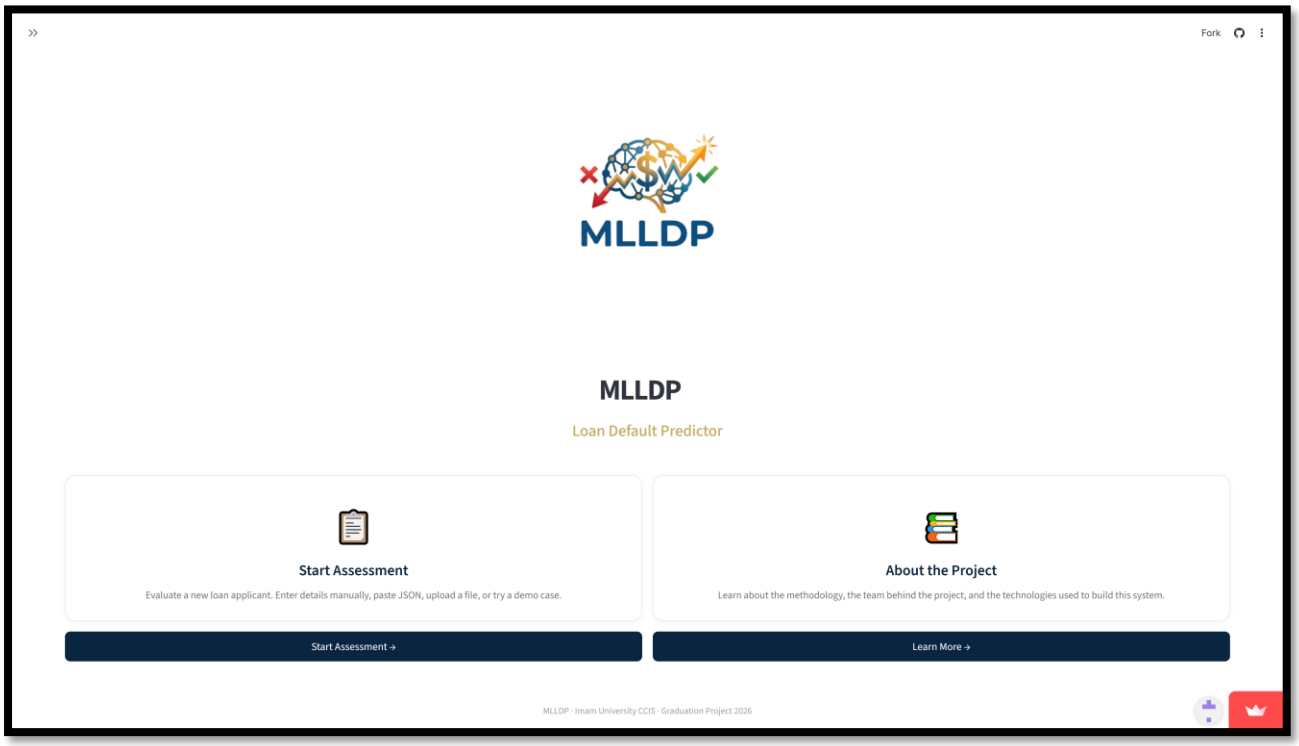

Figure 5.18: Home page of the MLLDP web application

The **About** view, shown in Figure 5.19 and 5.20, gives the user the context behind the system. It is divided into five panels. The **Project Overview** panel explains that MLLDP is a machine learning system that predicts the probability of loan default at the application stage and that goes beyond a single binary decision by explaining each prediction and showing similar past borrowers. The **Methodology** panel summarizes the technical approach: the LendingClub dataset, the calibrated XGBoost model from Section 6, SHAP for feature-level explanations, and a K-nearest-neighbours search for finding similar historical cases. The **Team** panel lists the two project members and the supervisor. The **Institution** panel identifies the university, the college, and the group number. The **Technologies** panel lists the open-source Python stack used to build the system: Python 3.13, Streamlit, XGBoost, LightGBM, scikit-learn, SHAP, pandas, NumPy, and matplotlib. This page acts as documentation embedded inside the app, so a reviewer or a future maintainer can understand what the system does without opening any external file.

Figure 5.19: About page of the MLLDP web application

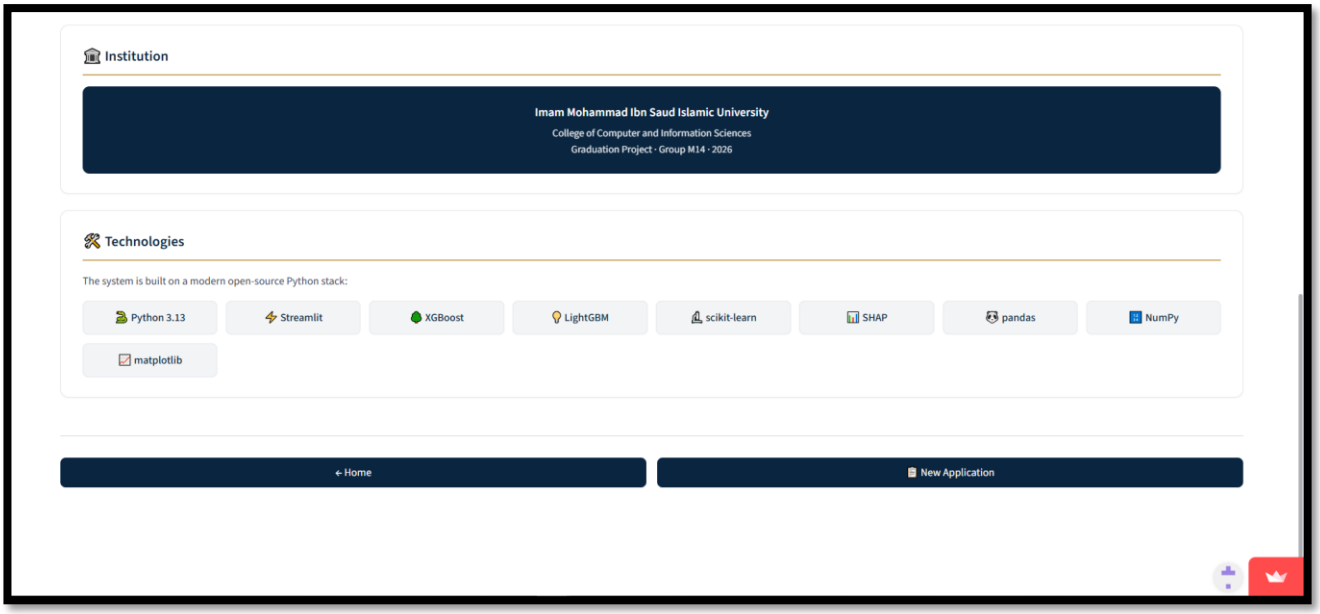

Figure 5.20: About page of the MLLDP web application (2)

The **New Application** view is where the loan officer actually evaluates an applicant. Because real loan officers work in different ways, the page offers four input methods through a tab bar at the top: **Manual Entry**, **Paste JSON**, **Upload File**, and **Demo Cases**. All four methods feed into the same prediction pipeline; they only differ in how the data reaches the model. The Manual Entry tab, shown in Figure 5.21, is the default. It splits the 14 user-visible application fields into three labelled sections to make the form easie=r to read. **Loan Details** holds five fields about the loan itself: loan amount, monthly installment, interest rate, loan purpose, and loan term. **Applicant Information** holds four fields about the borrower: annual income, home ownership, employment length, and number of mortgage accounts. **Credit Profile** holds five fields about the borrower's existing credit accounts. The 17 credit bureau fields are not asked in the form because they would slow down the loan officer and they are normally pulled from upstream systems anyway; instead the Manual Entry tab provides a small JSON upload box at the bottom of the form for the credit bureau report.

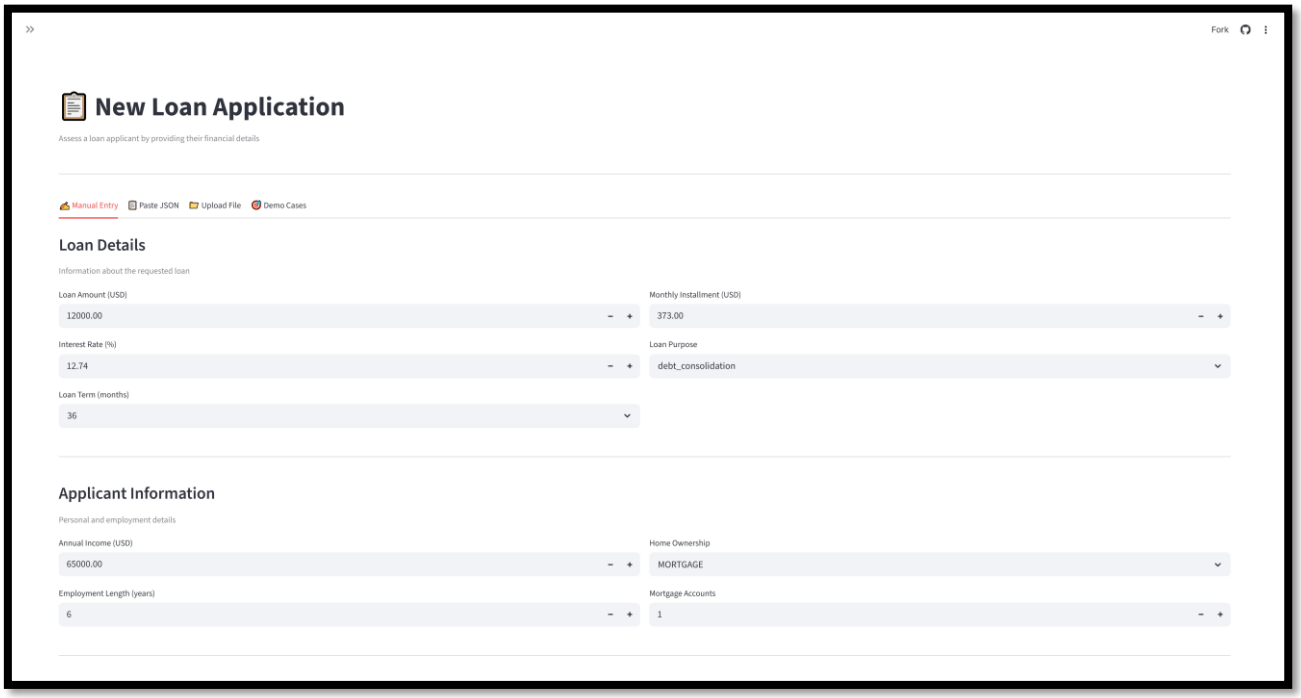

Figure 5.21: New Application page - Manual Entry tab

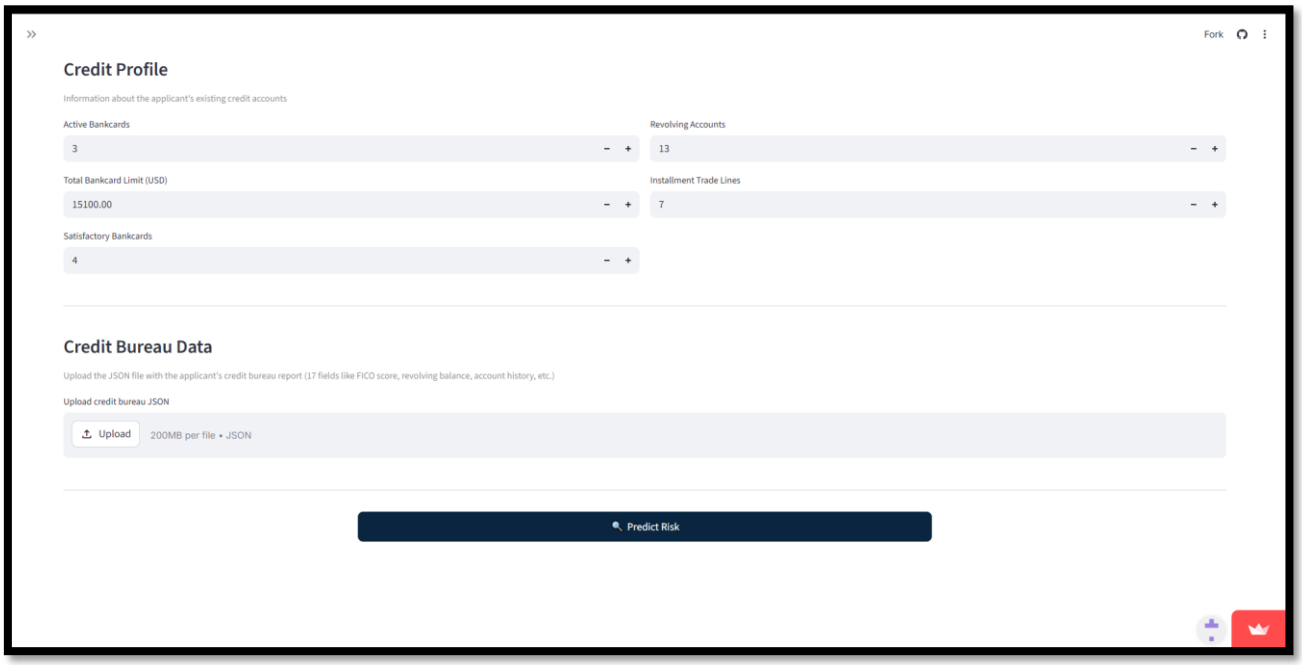

Figure 5.22: New Application page - Manual Entry tab (2)

The Paste JSON tab, shown in Figure 5.23, is the second input method. It is for users who already have the full 31-field applicant data in JSON format (14 application fields plus 17 credit bureau fields) and want to skip the form entirely. The tab presents a single text area where the user pastes the complete JSON object. A short instruction at the top reminds the user of the required field count. The Upload File tab works the same way but reads the JSON from a file the user selects from disk rather than from clipboard text. Both methods are useful for testing the system with the five sample JSON files we generated in Phase 5 of Section 7 (***sample_approve_low.json***, ***sample_threshold.json***, ***sample_reject_low.json***, ***sample_reject_mid.json***, and ***sample_reject_high.json***).

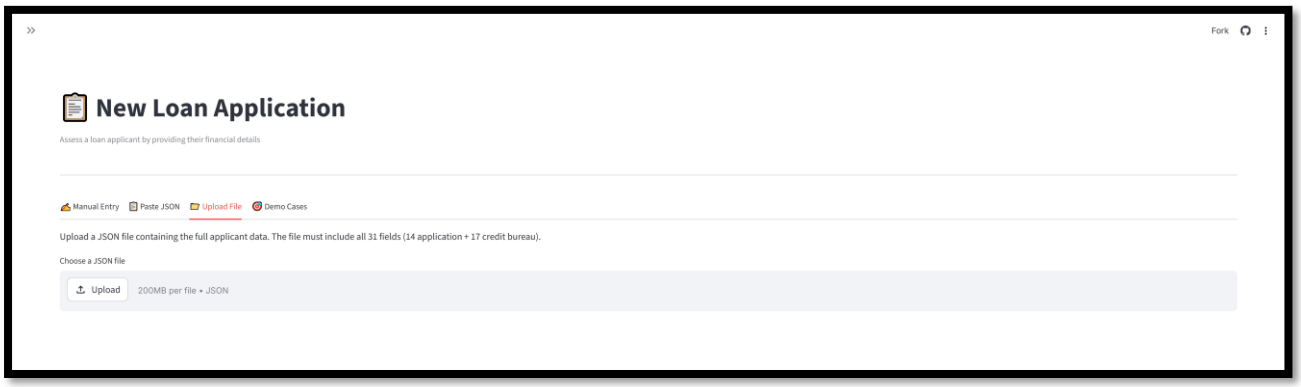

Figure 5.23: New Application page - Paste JSON tab

The fourth input method is the **Demo Cases** tab shown in Figure 5.24. This tab gives the user instant access to the five demo cases we picked in Phase 4 of Section 7. Selecting a case from the dropdown loads the applicant data into the pipeline without the user typing or pasting anything. A short description below the dropdown explains the story behind each case for example, "Strong Borrower Low risk profile, very likely to repay". This tab exists for two reasons. First, it lets a demo audience see the system in action with realistic examples without filling out a form. Second, it gives the loan officer a way to quickly compare the model's behaviour on different risk profiles.

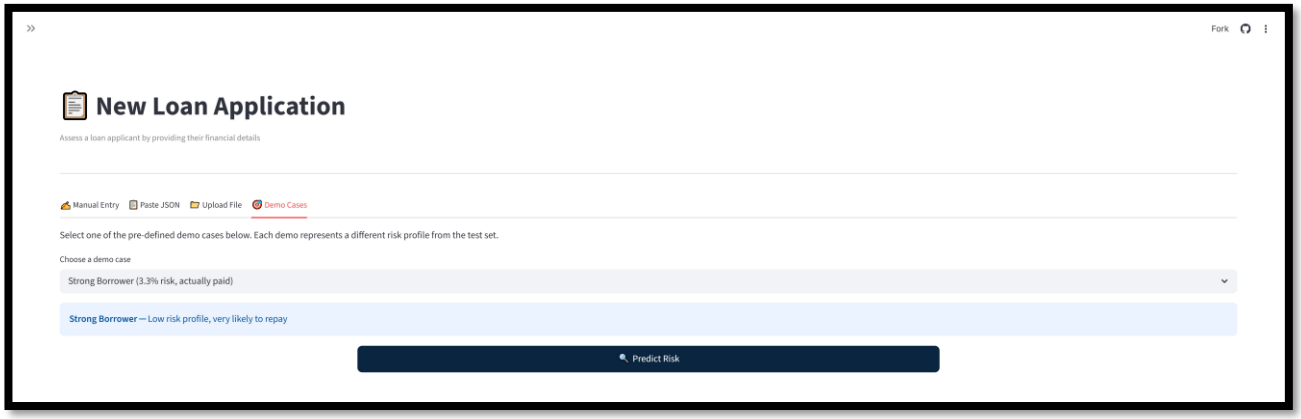

Figure 5.24: New Application page - Demo Cases tab

After the user submits an applicant through any of the four input methods, the system runs the inference pipeline described in Phase 6 of Section 7 and lands on the **Result** view. The result is split into three panels stacked vertically. The first panel, shown in Figure 5.25, is the decision banner at the top of the page. It is the most prominent visual on the page because it is the answer the loan officer needs first. The banner has four possible states driven by the applicant's risk zone. It is colored **green** and labelled **APPROVED** when the score is clearly below the 0.17 threshold. It is colored **yellow** and labelled **REVIEW REQUIRED** when the score lands within a ±0.05 epsilon band around the threshold, which signals that the case is on the decision boundary and deserves a closer human look. It is colored **orange** and labelled **REJECTED** when the score sits between the threshold and the 0.50 high-risk cap. It is colored **red** and labelled **REJECTED** when the score is at or above 0.50, signalling extreme risk. Below the banner, the same probability is shown three more times in different formats: as a large 0-to-100 score (for example, '3.3 / 100' for the Strong Borrower demo case), as a horizontal gauge with three colored zones, and as a row of three small metric cards showing the default probability, the decision threshold, and the risk zone. The horizontal gauge splits the 0-to-100 range into three zones (**APPROVE** for 0 to 17, **HIGH RISK** for 17 to 50, and **REJECT** for 50 to 100) and places an arrow marker at the applicant's score. The model's underlying decision is still binary at the 0.17 threshold, but the four-state banner and the three-zone gauge together turn the simple binary outcome into a richer visual signal that helps the loan officer understand the strength of the model's recommendation.

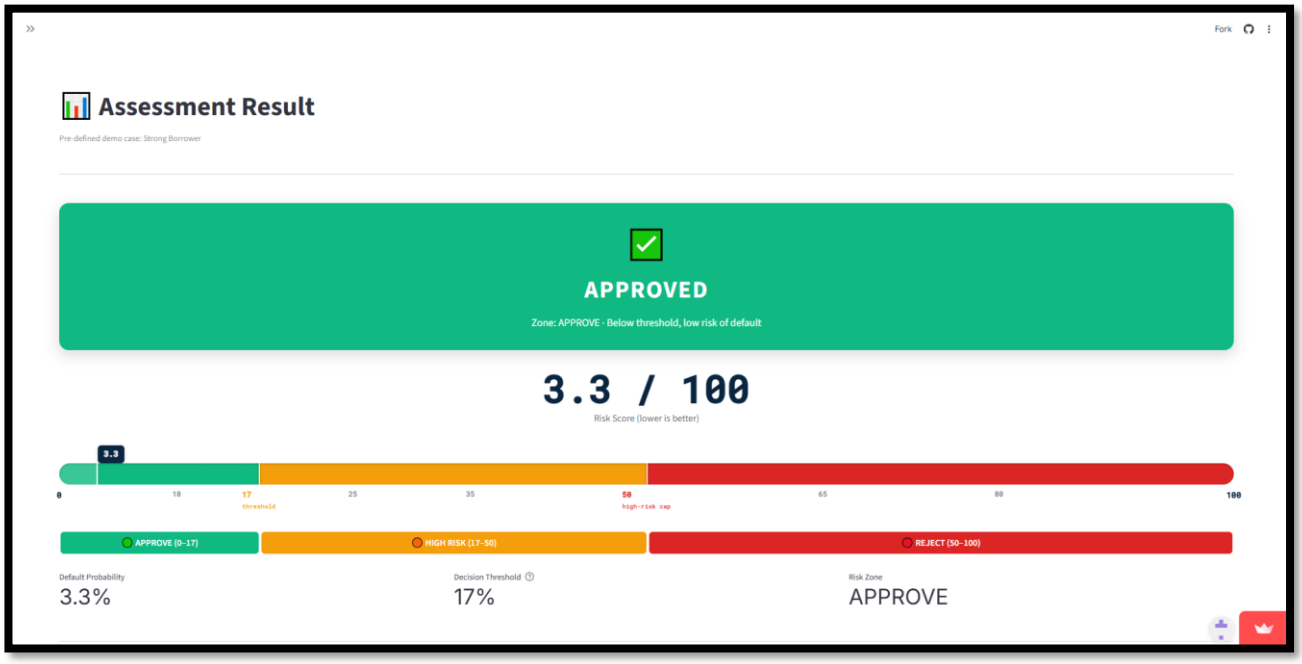

Figure 5.25: Assessment Result page - decision banner and risk score gauge

The second panel of the Result view, shown in Figure 5.26, is the **Top Risk Factors** explanation. This is the simplified user-facing version of the SHAP waterfalls we showed in Phase 3 of Section 7. It lists the seven features that pushed the applicant's score the most, with red circles for features that pushed risk up and green circles for features that pulled risk down. The contributions are shown as percentage points of default probability, which is easier to read for a non-technical user than raw SHAP values. For the Strong Borrower demo case, all seven features pulled the risk down, led by the interest rate at −6.4%, the loan term at −1.7%, and accounts opened in the past 24 months at −1.6%. A legend at the bottom of the panel reminds the user how to interpret the colors. Below the simplified view, an expandable section labelled **View Detailed SHAP Waterfall** reveals the full waterfall plot, which is useful for technical reviewers but hidden by default to keep the main view uncluttered.

Figure 5.26: Assessment Result page - top risk factors

The third panel, shown in Figure 5.27, is the **Similar Past Borrowers** section. This is the user-facing surface of the lookup pool we built in Phase 4.5 of Section 7. When a new applicant is submitted, the system computes the feature distance between that applicant and every row in the 100,000-applicant validation pool, then returns the 10 nearest neighbours. The panel displays three summary statistics at the top (the number of similar borrowers, the count of defaulters among them, and the count of fully paid loans), followed by a table that lists each of the 10 neighbours with their actual outcome (Paid or Defaulted) and the model's risk score for them. For the Strong Borrower demo case shown in the figure, all 10 nearest neighbours actually paid their loans, which is a strong real-world confirmation of the model's low-risk prediction. This panel turns the model's calibrated probability into something the loan officer can verify against historical evidence, which is a key trust-building feature that no SHAP plot alone can provide.

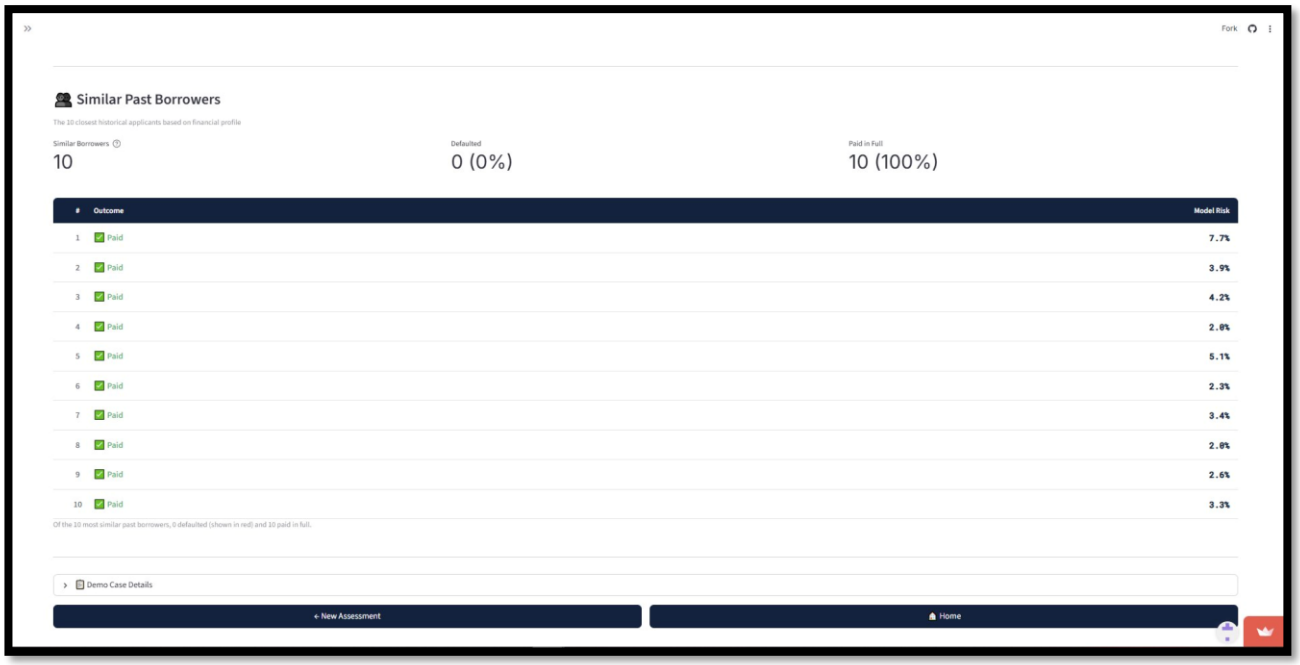

Figure 5.27: Assessment Result page - similar past borrowers

Two navigation buttons at the bottom of the Result page complete the workflow. The **New Assessment** button takes the user back to the New Application page to start over with a different applicant. The **Home** button returns to the Home view. Together, the four views and their internal panels deliver every functional requirement from Chapter 3: data ingestion through the form and upload widgets, loan status prediction through the decision banner, prediction explanation through the SHAP panel and the similar borrowers panel, and full transparency through the About page documentation. The Streamlit interface is the final piece of the MLLDP pipeline and the surface through which all of the work in Sections 5, 6, and 7 reaches the loan officer.

6 .Testing Chapter

6.1 Introduction

The testing phase of the project was conducted to verify and validate the functionality, correctness, and reliability of the Machine Learning Loan Default Predictor (MLLDP) system. The testing objectives were to ensure that the system met the requirements and specifications defined in the analysis and design phases, and to identify and resolve any defects or errors that occurred during the implementation phase. The testing approach combined three software testing techniques: path testing to verify complete user workflows, condition testing to validate critical decision logic, and code testing to confirm that individual components produce the expected output. The testing environment consisted of the following components:

A Streamlit web application built with Python, hosted both locally and remotely on Streamlit Community Cloud. The application provides four input modes for loan officers to assess applicants and displays results through a dashboard composed of a decision banner, a risk score bar, top contributing factors, a SHAP waterfall chart, and a table of similar past borrowers.

A calibrated XGBoost classifier trained on the LendingClub dataset, packaged together with all preprocessing transformers, SHAP background data, demo cases, and a lookup pool in a single production artifact file.

A PC with Visual Studio Code, a Chrome browser to access the Streamlit interface, and a GitHub repository for version control and deployment.

6.2 Test Cases

Table:6.1 TC001

Test Case Name/ID	**TC001**
Description	Testing the manual form input flow for a low-risk applicant who should be approved.
Steps to Reproduce	1. Open the MLLDP web application. 2. Click "Start Assessment " on the welcome page. 3. Select the "Manual Entry" tab. 4. Keep all 14 form fields at their default values. 5. In the Credit Bureau Data section, upload the sample_approve_low.json file. 6. Click the "Predict Risk" button. 7. Verify the result dashboard displays the prediction.
Expected Result	The system should preprocess the 31 raw inputs, run the calibrated XGBoost model, and display a probability below the 17% threshold. The decision banner should appear in green with "APPROVED", the zone label should read "APPROVE", and the SHAP top risk factors and similar borrowers sections should populate correctly.
Actual Result	The system correctly displayed a default probability of 10.3%, classified the applicant in the APPROVE zone, and showed a green "APPROVED" banner. The top risk factors and similar borrowers sections rendered successfully without any errors
Test Status	Passed

Table:6.2 TC002

Test Case Name/ID	**TC002**
Description	Testing the manual form input flow for a high-risk applicant who should be rejected.
Steps to Reproduce	1. Open the MLLDP web application. 2. Click "Start Assessment " on the welcome page. 3. Select the "Manual Entry" tab. 4. Update the form manual entry fields with high-risk values: - Loan Amount: 35000 - Interest Rate: 28 - Loan Term: 60 - Annual Income: 26000 - Employment Length: 0 - Home Ownership: RENT - Loan Purpose: small_business 5. In the Credit Bureau Data section, upload the sample_reject_high.json file. 6. Click the "Predict Risk" button. 7. Verify the result dashboard displays the rejection.
Expected Result	The system should preprocess the inputs, run the model, and display a probability well above the 17% threshold. The decision banner should appear in red or orange with "REJECTED", and the zone should be classified as HIGH RISK or REJECT. The SHAP top risk factors section should display several factors in red, indicating features that pushed the risk upward.
Actual Result	The system correctly displayed a high probability of default and classified the applicant in the appropriate risk zone with a red/orange "REJECTED" banner. The top risk factors clearly showed which features contributed to the rejection decision.
Test Status	Passed

Table:6.3 TC003

Test Case Name/ID	**TC003**
Description	Testing the borderline case handling when an applicant's risk score lands exactly on the decision threshold of 17%.
Steps to Reproduce	1. Open the MLLDP web application. 2. Click "Start Assessment " on the welcome page. 3. Select the "Demo Cases" tab. 4. Choose the "Borderline" demo from the dropdown. 5. Click the "Predict Risk" button. 6. Verify how the system handles the borderline case.
Expected Result	The system should detect that the score is exactly at the threshold (17.0) and trigger the special "AT THRESHOLD" zone. The decision banner should appear in yellow with "REVIEW REQUIRED", and the zone message should read "Exactly at the decision boundary, borderline case", allowing the loan officer to give such cases additional review rather than an automatic rejection.
Actual Result	The system correctly identified the borderline case with a score of 17.0, displayed a yellow "REVIEW REQUIRED" banner, and classified the applicant in the AT THRESHOLD zone with the appropriate borderline message.
Test Status	Passed

Table:6.4 TC004

Test Case Name/ID	**TC004**
Description	Testing the JSON paste input validation when the provided data is missing required fields.
Steps to Reproduce	1. Open the MLLDP web application. 2. Click "Start Assessment " on the welcome page. 3. Select the "Paste JSON" tab. 4. Clear the default JSON skeleton in the text area. 5. Paste a partial JSON object containing only 10 fields (instead of 31). 6. Click the "Predict Risk" button. 7. Verify the system's response.
Expected Result	The system should detect that the JSON is incomplete, prevent the prediction from running, and display a clear error message indicating how many fields are missing along with the names of some missing fields. The user should not be navigated to the result page.
Actual Result	The system correctly identified the missing fields, displayed a red error message stating "JSON is missing 21 required fields" along with the names of the first few missing ones, and prevented navigation to the result page.
Test Status	Passed

Table:6.5 TC005

Test Case Name/ID	**TC005**
Description	Testing the JSON paste input validation when the provided data is not valid JSON format.
Steps to Reproduce	1. Open the MLLDP web application. 2. Click "Start Assessment " on the welcome page. 3. Select the "Paste JSON" tab. 4. Clear the default JSON skeleton in the text area. 5. Paste an invalid string that does not conform to JSON syntax (random text(6. Click the "Predict Risk" button. 7. Verify the system's response.
Expected Result	The system should detect that the input is not valid JSON, prevent the prediction from running, and display a clear error message indicating the parsing failure. The user should remain on the JSON paste tab and not be navigated to the result page.
Actual Result	The system correctly caught the JSON parsing exception, displayed a red error message starting with "Invalid JSON:" followed by the specific syntax error, and prevented any further processing.
Test Status	Passed

Table:6.6 TC006

Test Case Name/ID	**TC006**
Description	Testing the complete file upload workflow for assessing a loan applicant through a JSON file containing all 31 required fields.
Steps to Reproduce	1. Open the MLLDP web application. 2. Click "Start Assessment " on the welcome page. 3. Select the "Upload File" tab. 4. Click "Browse files" and upload the test_high_risk.json file. 5. Verify that the system displays a success message and a preview of the uploaded data. 6. Click the "Predict Risk" button. 7. Verify that the result dashboard appears with the appropriate risk assessment.
Expected Result	The system should accept the uploaded JSON file, validate that all 31 required fields are present, display a success message along with an expandable preview of the data, and upon clicking Predict Risk, navigate to the result dashboard with a HIGH RISK classification including a complete breakdown of the prediction.
Actual Result	The system successfully loaded the JSON file, displayed the success message "File loaded successfully. Found 31 fields", showed the data preview when expanded, and after clicking Predict Risk navigated to the result page displaying a HIGH RISK zone with the corresponding probability and SHAP explanation..
Test Status	Passed

Table:6.7 TC007

Test Case Name/ID	**TC007**
Description	Testing whether the application produces predictions that exactly match the probabilities stored in the production artifact bundle for the demo cases.
Steps to Reproduce	1. Open the MLLDP web application. 2. Click "Start Assessment " on the welcome page. 3. Select the "Demo Cases" tab. 4. Select the "High Risk" demo from the dropdown. 5. Note the expected probability shown in the dropdown label (41.3%). 6. Click the "Predict Risk" button. 7. Compare the displayed probability on the result page with the expected value. 8. Repeat for the remaining four demo cases (Strong Borrower, Moderate Risk, Borderline, False Positive).
Expected Result	For each of the five demo cases, the application should display a probability that matches the stored value exactly (within rounding tolerance of 0.1%). The decision and zone classification should also match the stored values.
Actual Result	All five demo cases produced predictions matching their stored probabilities precisely: Strong Borrower 3.3%, Moderate Risk 9.9%, High Risk 41.3%, Borderline 17.0%, and False Positive 86.3%. Each case was classified into the correct zone with the appropriate banner.
Test Status	Passed

Table:6.8 TC008

Test Case Name/ID	**TC008**
Description	Testing the input validation in the manual form to ensure that the system rejects negative numbers and non-numeric text in numeric fields.
Steps to Reproduce	1. Open the MLLDP web application. 2. Click "Start Assessment " on the welcome page. 3. Select the "Manual Entry" tab. 4. Attempt to enter a negative number -5000 into the Annual Income field. 5. Verify the system's response. 6. Attempt to type a non-numeric text "abc" into the Loan Amount field. 7. Verify the system's response. 8. Repeat for other numeric fields such as Monthly Installment and Total Bankcard Limit.
Expected Result	The system should not accept negative values or non-numeric text in any numeric field. Negative inputs should be automatically clamped to the minimum allowed value, and non-numeric characters should be ignored entirely, preventing malformed data from reaching the model.
Actual Result	The system correctly rejected all invalid inputs. Negative numbers were either prevented from being entered or automatically replaced with the minimum allowed value, and non-numeric characters were ignored as expected. The form values remained within their defined valid ranges at all times.
Test Status	Passed

Table:6.9 TC009

Test Case Name/ID	**TC009**
Description	Testing the system's behaviour when the user attempts to predict without uploading the required credit bureau JSON file in the manual form.
Steps to Reproduce	1. Open the MLLDP web application. 2. Click "Start Assessment " on the welcome page. 3. Select the "Manual Entry" tab. 4. Fill in the 14 form fields with any valid values, or leave the defaults. 5. Do not upload any JSON file in the Credit Bureau Data section. 6. Look for the "Predict Risk" button. 7. Verify the system's response.
Expected Result	The system should not allow the user to proceed with prediction without the credit bureau data. The Predict Risk button should remain hidden until a valid JSON file is uploaded, ensuring that all 31 required fields are available before the model is invoked.
Actual Result	The system correctly prevented prediction by keeping the Predict Risk button hidden until a valid JSON file was uploaded in the Credit Bureau Data section. Once the file was provided, the button became visible and the prediction proceeded normally.
Test Status	Passed

Table:6.10 TC010

Test Case Name/ID	**TC010**
Description	Testing the navigation flow between pages and verifying that the application correctly resets between assessments.
Steps to Reproduce	1. Open the MLLDP web application. 2. From the welcome page, click "Start Assessment " and verify navigation to the New Application page. 3. Select the "Demo Cases" tab, choose any demo, and click "Predict Risk". 4. On the result page, click "New Assessment" and verify that the user returns to the New Application page with a fresh state. 5. Repeat the assessment, then click "Home" instead. 6. From the welcome page, click "Learn More " and verify navigation to the About page. 7. From the About page, click " Home" to return to the welcome page.
Expected Result	All navigation buttons should correctly route the user to their target pages. After clicking " New Assessment" or "Home", the application should reset properly so that the next assessment starts fresh without showing results or data from the previous one. Each page transition should occur smoothly without errors.
Actual Result	All navigation links worked as expected. The application correctly reset between assessments, ensuring that the previous applicant's results did not appear when starting a new one. All page transitions occurred smoothly and the website remained fully functional throughout.=
Test Status	Passed

7. Conclusion and Future Work

7.1 Introduction

This chapter closes the report by tying together the work we presented in the previous six chapters. It begins with a summary of the overall project from problem to deliverable. It then describes how the final system meets the requirements set out in Chapter 3. After that, it discusses the principal shortfalls we identified during the work, and proposes future work to address each of them. The chapter ends with a short concluding paragraph.

7.2 Summary of the Work

The project began with a simple but serious problem. Banks lose money when borrowers default on their loans, and traditional rule-based credit decisions are too rigid to handle the complex patterns hidden in modern lending data. Chapter 1 framed this as the core technical challenge: how can we build an end-to-end machine learning pipeline that predicts loan default reliably, explains its decisions, and gives loan officers a tool they can actually use.

Chapter 2 reviewed the related work and confirmed that ensemble methods such as Random Forest, XGBoost, and LightGBM consistently outperform traditional classifiers on credit data, while also identifying a clear research gap: most existing studies rely on black-box models that cannot explain why a loan is flagged for default. Chapter 3 translated these findings into a concrete set of user and system requirements, organized into functional and non-functional categories. Chapter 4 proposed a Machine Learning Pipeline Architecture with a Training Pipeline, a Shared Storage layer, and an Inference Pipeline that connects to a Streamlit web application.

Chapter 5 covered the actual implementation. The data preparation work was organized into 11 phases that took the raw LendingClub dataset of 2.2 million rows and 151 columns and turned it into a clean, leak-free training set of 791,706 rows with 40 locked features. The hyperparameter tuning work was organized into a further 12 phases that tuned four algorithms (***LightGBM***, ***XGBoost***, ***RandomForest***, and ***LogisticRegression***), built a stacking ensemble, calibrated the four candidates with isotonic regression, swept 25 decision thresholds, and finally locked in calibrated ***XGBoost*** at threshold 0.17 as the hero model. The final phases set up the SHAP explainer using ***KernelExplainer*** so the explanations would match the calibrated probability the user sees, walked through five real test set cases, and prepared every artifact the web application would need.

Chapter 5.2.5 described the final deliverable: a Streamlit web application with four views (Home, About, New Application, and Result) and four input methods (Manual Entry, Paste JSON, Upload File, and Demo Cases). The application loads a production artifact bundle at startup and produces a calibrated risk score, a top-risk-factors explanation, and a list of 10 similar past borrowers for every applicant the loan officer submits. Chapter 6 then verified the system through ten test cases covering path testing, condition testing, and code testing. All ten test cases passed, confirming that the application handles valid inputs, invalid inputs, edge cases, and navigation flows correctly. The end-to-end pipeline from raw CSV to live prediction is complete and verified.

7.3 Meeting the Requirements

The final system delivers on the requirements set out in Chapter 3. On the functional side, the web application loads the hero model and the production artifact bundle at startup, accepts applicant data through the user interface, returns a clear APPROVE or REJECT decision, and generates a visual SHAP explanation for every prediction. The application also offers three additional input methods (Paste JSON, Upload File, and Demo Cases) and a Similar Past Borrowers panel that complements the SHAP explanation with real historical evidence.

On the non-functional side, the system meets the efficiency target because the inference pipeline produces a decision and an explanation in a few seconds after the user clicks Predict Risk. It meets the dependability target with a test set recall of 76.14%, a precision of 30.30%, an F1 of 0.4335, an F2 of 0.5846, and an AUC of 0.7309, which are stable across the validation and test sets within 0.002 on every metric. It meets the usability target because the four-tab New Application page guides the loan officer step by step and does not require any technical background. It meets the security target because the application processes the applicant data inside the session and does not store sensitive details after the session ends. It meets the development and operational targets because the system is built entirely in Python with open-source libraries and runs on the free Streamlit Cloud service without requiring specialized high-performance hardware., it meets the ethical interpretability target because every prediction comes with a SHAP explanation and a similar-borrowers table, which together remove the black-box problem identified as the research gap in Chapter 2.

7.4 Principal Shortfalls

Although the system meets the requirements, it is not perfect. We discuss the main shortfalls below.

The first shortfall is the precision of the model. The hero model catches 76% of real defaulters but its precision is only 30%. This means that for every 100 applicants the model flags as risky, about 70 of them actually pay their loans in full. This is a deliberate consequence of the recall-first strategy we explained in Chapter 5, since accepting a defaulter is much more costly than rejecting a safe applicant. But it is still a real cost. A bank using this system as an auto-reject tool would lose business by turning away many safe borrowers. The model is best understood as a decision-support tool that flags applicants for a closer human review, not as an automated reject system.

The second shortfall is that the model was trained on US data from LendingClub. The Saudi market has different lending practices, different regulatory environments, and different borrower profiles. We made an early design choice to drop US-specific columns such as ***grade***, ***sub_grade***, ***addr_state***, and ***zip_code*** so that the trained model would generalize as much as possible, but the underlying behaviour of the borrowers in our training set is still American. A bank in Saudi Arabia could not deploy this exact model in production without retraining it on local data.

The third shortfall is that the model only sees 40 features taken from a single snapshot of the applicant at application time. Real-world events that often cause default, such as sudden job loss, medical emergencies, divorce, or family events, are completely invisible to the model. This was visible in Case 4 of Section 7 Phase 3, where a real defaulter looked perfectly safe to the model because every feature pointed in the right direction. The 40 features capture financial behaviour but not life events.

7.5 Future Work

We identified three directions of future work that map directly to the shortfalls discussed in the previous section.

Saudi-specific data integration. The single biggest improvement would be retraining the pipeline on local Saudi lending data, for example from SAMA or a partner bank. The entire pipeline we built is generalizable, so most of the work would be data collection and feature engineering rather than rebuilding the system from scratch. This would directly address the second shortfall and make the system usable in a real Saudi bank.

Cost-sensitive learning. The current model uses F2 as its optimization target, which weights recall twice as much as precision. But a real bank has concrete dollar costs for false positives (lost business) and false negatives (unrecovered loans). A future version of the pipeline could let the bank input these two cost values and tune the model and the decision threshold directly on the expected dollar loss. This would address the first shortfall by giving the bank a precise way to balance the recall-precision trade-off based on its own risk appetite, instead of forcing a one-size-fits-all setting.

Fairness audits. Credit decisions can unintentionally disadvantage specific groups of borrowers. A future version of the system should include a fairness audit that compares model decisions across demographic groups and flags any disparate impact. The audit would also check that the SHAP explanations do not consistently penalize the same groups for the same kinds of features. This addresses an ethical concern that the current system does not explicitly check for, even though Chapter 3 listed interpretability as an ethical requirement.

7.6 Concluding Remarks

The MLLDP project successfully delivered an end-to-end machine learning pipeline for loan default prediction, from raw data ingestion all the way to a live web application. The final system uses a calibrated ***XGBoost*** hero model with a recall of 76% on unseen test data, and pairs every prediction with a SHAP-based explanation and a list of similar past borrowers. Together, these features directly address the research gap identified in Chapter 2 by replacing a black-box decision with a transparent and verifiable one. The system is not perfect and the shortfalls in Section 7.4 are real, but the foundation is solid and the future work in Section 7.5 provides a clear path forward. We hope this work serves as a starting point for a more complete and locally-adapted credit risk system in the Saudi market.

Poster

M14

MLLDP

Saad ALmugrin Thamer ALshamrani

Supervisor: Dr. Mostafa Elsayed Ahmad Ibrahim

Introduction

- Banks lose money when borrowers default on their loans
- Traditional rule-based credit checks are slow and miss complex patterns
- Most existing ML solutions act as "black boxes" they predict but cannot explain why
- **Our goal:** an accurate AND explainable loan default prediction system

Background

- Recent studies show ensemble models (Random Forest, XGBoost, LightGBM) work best on credit data
- Class imbalance is the main technical challenge (defaults are rare)
- Interpretability is the main research gap in the literature

Objectives

- Build an end-to-end ML pipeline using a public lending dataset
- Compare multiple algorithms and select the best one
- Pair every prediction with a clear visual explanation
- Deploy a user-friendly web interface for loan officers

Methodology

- **Dataset:** LendingClub 2007–2018 (1.34M loans, 151 columns)
- **Split:** 60/20/20 stratified (train / validation / test)
- **Cleaning:** 11 phases remove leakage, engineer features, lock 40 final inputs
- **Tuning:** 12 phases tune 4 algorithms + stacking ensemble + calibration
- **Imbalance:** native class weighting (not SMOTE)
- **Threshold:** F2-based sweep, locked at 0.17
- **Explainability:** SHAP for feature contributions

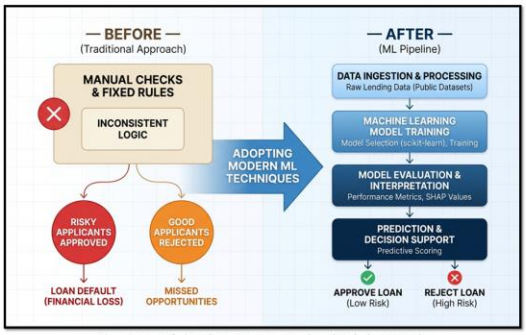

Figure 1: Loan Default Prediction System: From Traditional Methods to ML Pipeline

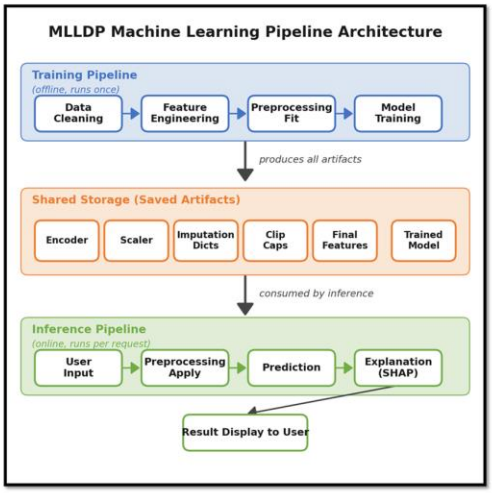

Figure 2: MLLDP pipeline architecture

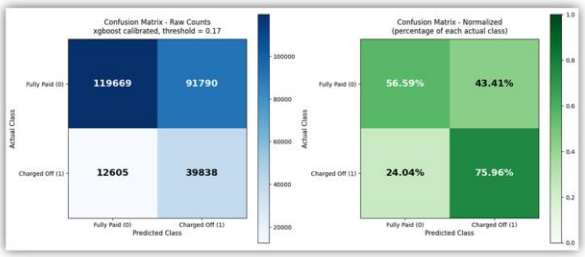

Figure 3: Confusion matrix on test set (calibrated XGBoost)

Figure 4: Assessment result with decision banner and risk score

Conclusion

- Delivered an end-to-end ML pipeline from raw data to live web app
- Final model catches **76% of real defaulters** on completely unseen test data
- Every prediction is **explainable** through SHAP and verifiable through similar past borrowers
- The system directly addresses the **black-box research gap** identified in the literature

Future Work

- **Saudi-specific data:** Retrain on local lending data for direct use in the Saudi market
- **Cost-sensitive learning:** Let banks plug in their own false-positive and false-negative dollar costs
- **Fairness audits:** Check that model decisions and SHAP explanations don't disadvantage specific groups

Limitations

- High recall comes with low precision (30%) system is best used as decision support, not auto-reject
- Model was trained on US data, so direct Saudi deployment requires retraining
- 40 features capture financial behavior but not life events like job loss

References

[1] Akinjole et al., *Mathematics*, 2024
[2] Nguyen et al., *American Journal of Applied Sciences*, 2025
[3] Núñez Mora et al., *Revista Mexicana de Economía y Finanzas*, 2023
[4] Lai, *Proc. CCNS*, 2020

Acknowledgments

We thank Imam Mohammad Ibn Saud Islamic University and the College of Computer and Information Sciences for the academic environment and resources that made this project possible.

CS Department

1447, Semester # 2

References

[1] S. Chauhan, "Machine Learning Models for Loan Default Forecasting: Accuracy Comparison," in *Proc. 2024 Second International Conference on Computational and Characterization Techniques in Engineering & Sciences (IC3TES)*, Lucknow, India, 2024, pp. 1-5. doi: 10.1109/IC3TES62412.2024.10877523.

[2] J. C. Alejandrino, J. P. Bolacoy Jr., and J. V. B. Murcia, "Supervised and unsupervised data mining approaches in loan default prediction," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 13, no. 2, pp. 1837-1847, Apr. 2023. doi: 10.11591/ijece.v13i2.pp1837-1847.

[3] N. Deborah R, et al., "An Efficient Loan Approval Status Prediction Using Machine Learning," in *Proc. 2023 International Conference on Advanced Computing Technologies and Applications (ICACTA)*, 2023, pp. 1-6. doi: 10.1109/ICACTA58201.2023.10392691.

[4] V. Padimi, V. S. Telu, and D. D. Ningombam, "Applying Machine Learning Techniques To Maximize The Performance of Loan Default Prediction," *Journal of Neutrosophic and Fuzzy Systems*, vol. 2, no. 2, pp. 44-56, Jan. 2022. doi: 10.54216/JNFS.020204.

[5] N. Robinson and N. Sindhwani, "Loan Default Prediction Using Machine Learning," in *Proc. 2024 12th International Conference on Reliability, Infocom, Technologies and Optimization (ICRITO)*, 2024, pp. 1-5. doi: 10.1109/ICRITO61523.2024.10522232.

[6] X. Zhang, et al., "Data-Driven Loan Default Prediction: A Machine Learning Approach for Enhancing Business Process Management," *Systems*, vol. 13, no. 7, Art. no. 581, Jul. 2025. doi: 10.3390/systems13070581.

[7] A. Akinjole, O. Shobayo, J. Popoola, O. Okoyeigbo, and B. Ogunleye, "Ensemble-Based Machine Learning Algorithm for Loan Default Risk Prediction," *Mathematics*, vol. 12, no. 21, Art. no. 3423, Oct. 2024. doi: 10.3390/math12213423.

[8] S. K. Shaheen and E. ElFakharany, "Predictive analytics for loan default in banking sector using machine learning techniques," in *Proc. 2018 28th International Conference on Computer Theory and Applications (ICCTA)*, Alexandria, Egypt, 2018, pp. 66-71. doi: 10.1109/ICCTA45985.2018.9499147.

[9] L. Lai, "Loan Default Prediction with Machine Learning Techniques," in *Proc. 2020 International Conference on Computer Communication and Network Security (CCNS)*, 2020, pp. 5-9. doi: 10.1109/CCNS50731.2020.00009.

[10] W. J. Wu, "Machine Learning Approaches to Predict Loan Default," *Intelligent Information Management*, vol. 14, pp. 157-164, Sep. 2022. doi: 10.4236/iim.2022.145011.

[11] Q. G. Nguyen, et al., "Enhancing Credit Risk Management with Machine Learning: A Comparative Study of Predictive Models for Credit Default Prediction," *The American Journal of Applied Sciences*, vol. 7, no. 1, pp. 21-30, Jan. 2025. doi: 10.37547/tajas/Volume07Issue01-04.

[12] J. A. Núñez Mora, P. Moncayol, C. Franco, P. Madrazo-Lemarroy, and J. Beltrán, "Loan Default Prediction: A Complete Revision of LendingClub," *Revista Mexicana de Economía y Finanzas, Nueva Época*, vol. 18, no. 3, p. e886, Jul.-Sep. 2023. doi: 10.21919/remef.v18i3.886.

[13] P. Khalili, M. Kargari, M. A. Rastegar, and M. Alavi, "A Hybrid Method for Online Loan Default Prediction using Machine Learning Techniques," in *Proc. 2025 11th International Conference on Web Research (ICWR)*, 2025, pp. 503-508. doi: 10.1109/ICWR65219.2025.11006211.

[14] Y. Qiu and J. Wang, "Credit Default Prediction Using Time Series-Based Machine Learning Models," *Artificial Intelligence and Applications*, vol. 3, no. 3, pp. 284-294, Mar. 2025. doi: 10.47852/bonviewAIA52023655.